Algorithmique et programmation

par

David Célestin FAYE

UFR DE SCIENCES APPLIQUEES ET DE TECHNOLOGIE
UNIVERSITÉ GASTON BERGER

Saint-Louis, Sénégal, 26 novembre 2007

Table des matières

Table des matières **ii**

Liste des tableaux **x**

Liste des figures **xi**

1 Ordinateur et Informatique **1**

1.1 Généralités 1

1.2 Architecture d'un ordinateur 2

2 Notions d'algorithme et de programme **5**

2.1 Introduction à l'Algorithmique 5

2.1.1 Qu'est-ce qu'un algorithme ? 5

2.1.2 Propriétés d'un algorithme 6

2.1.3 Place de l'algorithme dans la résolution d'un problème informatique 7

2.2 Notion de pseudo-langage 8

2.3 Elaboration d'un algorithme 9

2.4 Programmation 10

2.5 Objets simples, types et actions élémentaires 10

2.5.1 Type d'un objet 10

2.5.2 Les objets 11

2.5.3 Actions élémentaires 12

2.5.3.1 Opérateurs sur les types simples 12

2.5.3.2 Affectation 12

2.5.3.3 Lecture et écriture 13

2.5.3.4 Commentaires 13

3 Le langage Pascal **15**

3.1 Historique 15
3.2 Programmer en Pascal 15
3.3 Les constituants élémentaires du Pascal 16
3.3.1 L'Alphabet 16
3.3.2 Les mots du langage 17
3.3.2.1 Les mots réservés 17
3.3.2.2 Les identificateurs 17
3.3.2.3 Les identificateurs standards 18
3.4 Le Langage PASCAL 18
3.4.1 Caractéristiques globales 18
3.4.2 Structure globale d'un programme PASCAL 18
3.4.2.1 En-tête 19
3.4.2.2 Partie déclarative 19
3.4.2.3 Bloc d'instructions 20
3.4.3 Déclarations de constantes 21
3.4.4 Déclaration de types 22
3.4.4.1 Les types standards 22
3.4.4.2 Les types scalaires et non standards 25
3.4.5 Déclaration de variables 26
3.4.6 Instructions composées 28
3.4.6.1 Définition 28
3.4.6.2 Instruction d'affectation 28
3.4.7 Opérateurs et Fonctions arithmétiques 29
3.4.7.1 Opérateurs disponibles 29
3.4.7.2 Expressions 30
3.4.7.3 Fonctions arithmétiques 31
3.4.7.4 Fonctions logiques 31
3.5 Entrées / Sorties 31
3.5.1 Sortie (Ecriture) 32
3.5.2 Entrées (Lecture) 34
3.5.3 Lecture directe du clavier 35

4 Structures de contrôle **36**
4.1 Introduction 36
4.2 La structure séquentielle 36
4.2.1 La notion de séquence 36
4.2.2 Les instructions composées 38
4.3 Les structures alternatives 39
4.3.1 Le choix simple 39
4.3.2 La sélection multiple : le CAS... 44
4.3.3 Le concept de condition 48
4.4 Les structures répétitives 50
4.4.1 Définition 50
4.4.2 Boucle à bornes définies (POUR...FAIRE) 51
4.4.3 Boucles à bornes non définies 52
4.4.4 Exemple comparatif 57

5 Les tableaux et les chaînes de caractères **60**
5.1 Tableaux à 1 dimension 60
5.1.1 Définition 60
5.1.2 Déclaration d'un tableau 60
5.2 Tableaux à deux dimensions 63
5.2.1 Définition 63
5.3 Tableaux multidimensionnels 65
5.4 Manipulations élémentaires de tableaux 65
5.4.1 Création et affichage d'un tableau 65
5.4.2 Maximum et Minimum d'un tableau 66
5.4.3 Recherche séquentielle d'un élément dans un tableau 67
5.4.4 Recherche dichotomique d'un élément dans un tableau ordonné . 68
5.4.5 Recherche d'un élément dans une matrice 69
5.4.6 Initialisation d'une matrice unité 70
5.4.7 Fusion de deux tableaux ordonnés 71
5.5 Les chaînes de caractères 72
5.5.1 Définition 72
5.5.2 Opérateurs et fonctions 73

6 Les sous programmes **76**
6.1 La programmation modulaire 76
6.1.1 Définition 76
6.1.2 Les différents types de modules 78
6.2 Les Procédures 78
6.3 Syntaxe et déclarations 78
6.3.1 Exemples 80
6.4 Fonctions 83
6.4.1 Utilité des fonctions 83
6.4.2 Définition 83
6.5 Différence entre procédure et fonction 85
6.6 Variables globales et variables locales 87
6.6.1 Déclarations 87
6.6.2 Portée des variables 88
6.6.3 Locale ou globale ? 89
6.7 Paramètres 93
6.7.1 Définition 93
6.7.2 Paramètres formels et paramètres effectifs 95
6.7.3 Passage de paramètre par valeur 95
6.7.4 Passage de paramètre par adresse 98
6.7.5 Bon réflexes 101
6.7.6 Cas des fonctions 101

7 Comp **103**

8 Fichiers **104**

9 La récursivité **106**
9.1 Définition 106
9.2 Récursivité simple 107
9.3 Récursivité multiple 108
9.4 Récursivité mutuelle 108
9.5 Récursivité imbriquée 109
9.6 Exemples d'algorithmes récursifs 109
9.6.1 Somme des N premiers entiers 109

9.6.2 Puissance d'un entier . . . 111
9.6.3 Conversion binaire récursive . . . 113
9.6.4 Tour de Hanoï . . . 114
9.6.5 Dessin en abyme . . . 116
9.7 Les Algorithmes de Tri récursif . . . 117
9.7.1 Tri avec insertion récursif . . . 117
9.7.2 Diviser pour régner . . . 119

10 Algorithmes de tri **123**
10.1 Le problème du tri . . . 123
10.2 Spécification du tri . . . 123
10.3 Méthodes de tri . . . 124
10.3.1 Le tri par sélection . . . 124
10.3.2 Tri par insertion . . . 125
10.4 Tri par comptage . . . 125
10.4.1 Tri avec insertion récursif . . . 126
10.4.2 Tri par fusion . . . 127
10.4.3 Tri rapide . . . 127
10.5 Conclusion . . . 127

11 Structures de données élémentaires **128**
11.1 Rappels Les pointeurs . . . 128
11.1.1 Notion de pointeur . . . 128
11.1.2 Pointeurs et arguments . . . 130
11.1.3 Pointeurs et tableaux . . . 130
11.1.4 Arithmétique des pointeurs . . . 131
11.1.5 Copie de tableau . . . 132
11.1.6 Allocation dynamique . . . 133
11.1.7 Pointeurs et tableaux . . . 134
11.1.7.1 Pointeurs et tableaux à une dimension . . . 134
11.1.7.2 Pointeurs et tableaux à plusieurs dimensions . . . 136
11.1.8 Pointeurs et chaînes de caractères . . . 138
11.1.9 Pointeurs et structures . . . 141
11.1.9.1 Pointeur sur une structure . . . 141
11.2 Listes chaînées . . . 143

11.2.1 Definition 143
11.2.2 Algorithmes de manipulation de listes chaînées 146
11.2.2.1 Création d'une liste vide 146
11.2.2.2 Test de la liste vide 146
11.2.2.3 Tête de liste 147
11.2.2.4 Ajout en tête 147
11.2.2.5 Ajout en tête (version fonction) 148
11.2.2.6 Ajout en queue 148
11.2.2.7 Ajout au milieu 150
11.2.2.8 Concaténation de deux listes 151
11.2.2.9 Insertion d'un élémént à la k-ieme place 153
11.2.2.10 Recherche d'un élément 153
11.2.2.11 Impression des éléments d'une liste 155
11.2.2.12 Nombre d'élements d'une liste 157
11.2.2.13 Suppression d'un élément 158
11.2.2.14 Image miroir d'une liste 161
11.2.2.15 Suppression de la totalité d'une liste 163
11.2.2.16 Suppression de toutes les occurences d'un élément dans une liste 164
11.3 Structures auxilaires : Piles, files d'attentes 165
11.3.1 Piles 165
11.3.1.1 Eléments de base 165
11.3.1.2 Pile : représentation par tableau 167
11.3.2 Files 168
11.3.2.1 Eléments de base 168
11.3.2.2 File d'attente : représentation par liste chaînée gardée . 169
11.3.2.3 File d'attente : représentation par tableau 171
11.3.2.4 File circulaire 172
11.4 Application :evaluation des expressions arithmétiques postfixées 173
11.5 Anneaux et listes chaînées bidirectionnelles 178
11.5.1 Anneaux 178
11.5.2 Manipulations sur les anneaux 179
11.5.3 Listes chaînées bidirectionnelles 180
11.5.3.1 Manipulations sur les chaînes bidirectionnelles 180

12 Les arbres **183**
12.1 Terminologie 183
12.2 Opérations élémentaires sur les arbres 186
12.2.0.2 Test de l'arbre vide 186
12.2.0.3 Racine d'un arbre 186
12.2.0.4 Sous-arbre gauche 186
12.2.0.5 Sous-arbre droit 187
12.2.0.6 Construction d'un nouvel arbre 187
12.2.0.7 Nombre de nœuds d'un arbre 187
12.2.0.8 Hauteur d'un arbre 187
12.2.0.9 Recherche d'un élément 188
12.3 Parcours d'abres 188
12.3.0.10 ***Parcours préfixé*** 189
12.3.0.11 *Parcours postfixé* 189
12.3.0.12 *Parcours infixé* 190
12.4 Arbres binaires de recherche 190
12.4.1 Définition 190
12.4.2 Opérations sur les arbres binaires de recherche 191
12.4.2.1 recherche d'un élément 191
12.4.2.2 Insertion 192
12.4.2.3 Suppression 192
12.4.2.4 Minimum d'un ABR 194
12.4.2.5 Maximum d'un ABR 195
12.5 Arbres équilibré 195
12.6 Arbres rouge et noir 195

13 Exercices **196**
13.0.1 Eléments de base 197
13.0.2 Les structures conditionnelles et itératives 197
13.0.2.1 Tableaux 202
13.0.2.2 Fonctions 204
13.0.3 Récursivité 205
13.0.4 Enregistrements 206
13.0.5 Complexité 211

13.0.5.1 LISTES 214
13.0.5.2 Arbres 217

Conclusion **221**

Annexe A Titre **222**
A.1 Articulation d'un programme 222
A.2 Procédure 222
A.3 Mots réservés 223
A.3.1 Mots réservés 223
A.3.2 Unité Crt 223
A.3.3 Unité DOS 223
A.3.4 Unité graph 223

Bibliographie **228**

Liste des tableaux

2.1 Opérateurs sur les types simples 12

3.1 Opérateurs disponibles 29
3.2 Fonctions arithmétiques 32
3.3 Fonctions logiques 32

A.1 Mots réservés 223
A.2 Unité Crt 223
A.3 Unité DOS 224
A.4 Unité graph 224
A.5 Types de variables 225

Liste des figures

1.1 traitement information. 1
1.2 Architecture. 3

2.1 Les différentes étapes du processus de programmation. 7
2.2 Traitement des données. 8
2.3 Notion de pseudo langage. 9

3.1 Cycle de la programmation en Pascal. 16

4.1 Algorigramme de la structure séquentielle. 38
4.2 Organigramme de la sélection sans alternative 41
4.3 Organigramme de la sélection avec alternative 41
4.4 Organigramme de la structure avec IF imbriqués 45
4.5 Organigramme de la structure TANTQUE... FAIRE 53
4.6 Organigramme de la structure REPETER...JUSQUE 55

5.1 Tableau à une dimension (vecteur) 62
5.2 Tableau à deux dimensions (matrice) 64

6.1 Diagramme de description syntaxique de l'entête d'une procédure 79
6.2 Diagramme de description syntaxique de la liste de paramètres 79

9.1 Dessin en abyme. 116
9.2 Mode de tracé de P_1 sans jamais lever la plume : 117
9.3 Tri rapide 120
9.4 Tri fusion 121

10.1 Tri par sélection 125

11.1 Exemple de liste chaînée. 144
11.2 représentation graphique de la liste doublement chaînée. 144
11.3 Représentation graphique de la liste circulaire. 145
11.4 Ajout en tête de liste. 147
11.5 Ajout en queue. 149
11.6 Ajout au milieu. 150
11.7 Concaténation de deux listes par `concatL`. 152
11.8 Concaténation de deux listes par `concatL2`. 152
11.9 Suppresion de la cellule contenant e (en milieu). 158
11.10Représentation d'une Pile 166
11.11file d'attente implémentée par une liste chaînée gardée. 169
11.12Représentation graphique de la liste circulaire. 179
11.13représentation graphique de la liste doublement chaînée. 180

12.1 Exemple d'arbre. 183
12.2 Exemple d'arbre binaire. 185
12.3 Exemple d'arbre binaire. 188
12.4 Exemple d'arbre binaire de recherche. 191

13.1 Représentation d'une structure de données d'une discothèque. 216

Chapitre 1

Ordinateur et Informatique

1.1 Généralités

Apparue, il y a une cinquantaine d'années, l'informatique a connu et connaît une évolution extrêmement rapide. Etymologiquement, le mot "informatique" veut dire : traitement automatique de l'information. Ceci signifie que l'ordinateur n'est capable de fonctionner que s'il y a apport d'information par l'utilisateur. Le traitement automatique implique un traitement qui suit des règles qui peuvent être identifiées et également programmées dans un ordinateur L'utilisateur fournit des données à l'ordinateur (entrées

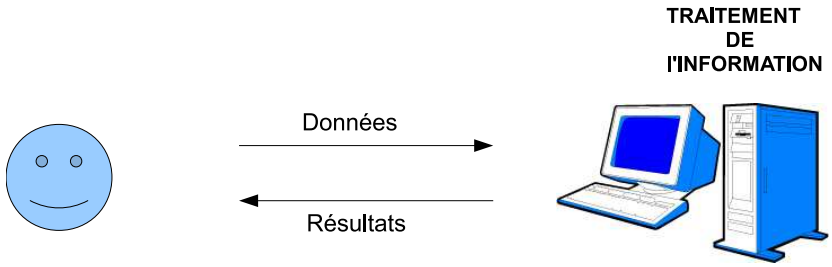


FIGURE 1.1 – traitement information.

ou *Input*), qui traite ces informations, puis renvoie les résultats ou réponses à l'utilisateurs (sorties ou *Output*). Le but de l'informatique est donc de faire accomplir par l'ordinateur ce que fait l'homme, avec des gains de :

- rapidité
- précision
- efficacité

Avec l'informatique, la machine utilisée (l'ordinateur) nécessite l'utilisation d'un programme (ou logiciel). Un ordinateur sans logiciel est inutile. En revanche, il est possible d'utiliser plusieurs logiciels très différents, avec le même ordinateur (calculs, gestion, jeux, Bureautique...). Aujourd'hui, l'ordinateur est donc une machine capable d'effectuer des fonctions aussi diverses que puissantes, grâce à ces fameux programmes ou logiciels. Jusqu'aux années 90, les ordinateurs dans l'entreprise étaient essentiellement des systèmes de grands constructeurs, très coûteux, très fermés, peu conviviaux. Ils portaient le nom de MainFrames pour les plus puissants ou de mini-ordinateurs. Désormais, les micro-ordinateurs (de type PC pour Personal Computer) remplacent petit à petit ces anciens systèmes .

1.2 Architecture d'un ordinateur

Un ordinateur est une machine de traitement de l'information capable :

– d'acquérir et de stocker des informations

– d'effectuer des traitements et de restituer des informations.

L'architecture générale d'un ordinateur est la suivante :

– une unité centrale appelée CPU (contenant le processeur),

– des composants matériels autour (mémoires, cartes diverses),

– des organes périphériques (écran, clavier, imprimante...)

– une couche logicielle basse, appelée système d'exploitation

– les logiciels d'applications (Bureautique, jeux, calcul, gestion...)

On pourrait schématiser le noyau central d'un ordinateur de la façon suivante :

Le CPU (Central Process Unit ou Unité Centrale) est le cœur de l'ordinateur. Il comporte les registres et la mémoire centrale Les registres :

– contiennent les instructions exécutées ;

– gardent la trace des informations courantes.

L'Unité Arithmétique et Logique (UAL) effectue les instructions arithmétiques et logiques en travaillant sur les données. L'Unité de Contrôle (UC) entre l'UAL et les registres et décode les instructions. La mémoire centrale contient le programme (instructions à traiter) ainsi que les données au moment de l'exécution. Le processeur et la mémoire sont reliés par un bus.

Il existe deux types de mémoire centrale :

– la mémoire vive, ou encore RAM pour Random Access Memory (Mémoire à Accès

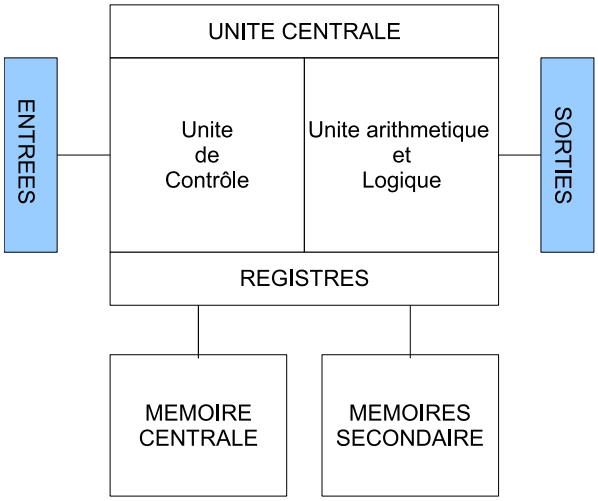


FIGURE 1.2 – Architecture.

Aléatoire : lecture ou écriture) qui contient les données et les programmes dits "volatiles' ; son contenu s'efface dès que l'alimentation en énergie est coupée.

– la mémoire morte, ou ROM : Read Only Memory (Mémoire à Lecture Seule) contient des données et des programmes figés, son contenu est fixé à la fabrication de l'ordinateur et ne peut jamais être modifié.

Les mémoires secondaires, quant à elles, contiennent des programmes et des données de façon permanente, puisqu'elles conservent l'information même si la machine est éteinte. Il peut s'agir de : disques durs, disquettes, CD-ROMs ou DVD-ROMs. L'unité centrale peut être étendu avec des composants supplémentaires (composants matériels). Il s'agit par exemple de mémoires (mémoire centrale), de cartes sonores, cartes vidéo, cartes réseau... Les organes périphériques permettent surtout la communication avec l'utilisateur. Le clavier, la souris, le CD-ROM ou DVD-ROM sont des périphériques d'entrée. Ils permettent la collecte d'informations. L'écran, les imprimantes, la table traçante sont des périphériques de sortie. Ils permettent la diffusion d'informations Le système d'exploitation est un programme sans lequel il n'est pas possible de communiquer avec l'ordinateur. C'est un programme qui interprète les diverses informations données par l'utilisateur, en les convertissant en langage machine. Le système d'exploitation (Operating System) est un ensemble de programmes dont la fonction est de gérer le système informatique. Le rôle d'un système d'exploitation est donc de créer un environnement pour faire fonctionner des applications diverses :

– gestion des ressources de l'ordinateur
– gestion du disque et des fichiers
– gestion des opérations effectuées par l'utilisateur
– pilotage des organes périphériques
– Utilisation d'un langage de commande.

Actuellement, les OS les plus connus sont DOS, Windows (3, 9.X, NT ou XP), MacOS, OS/2 ou encore UNIX. Le langage machine, le seul directement compréhensible par la machine, est en fait une succession de zéros et de uns définissant des opérations précises à effectuer : toute information étant donc codée avec uniquement des 0 et des 1. Ces chiffres binaires 0 et 1 à partir desquels on construit des nombres plus grands sont appelés bits (pour binary digit). Toutes les données manipulées sont ainsi représentées par des séquences de bits :

– un caractère : 8 bits (code entre 0 et 255)
– un entier : 32 bits
– un réel en virgule flottante (32 ou 64 bits)
– sons : décomposés en échantillon
– image décomposée en pixels

Un Octet (ou Byte) est un ensemble de 8 bits.

Les longueurs couramment utilisées sont des ensembles de 16, 32 ou 64 bits.

Un Kilo (ou 1 K) correspond à 1024 bits, soit 210 bits.

Un Méga (ou 1 M) correspond à 1000 K.

Un Giga est un ensemble de 1000 Mégas.

Ces unités de mesures sont fréquemment utilisées pour indiquer des tailles (ou capacités) de mémoires. Par exemple la taille des ordinateurs PC d'aujourd'hui est souvent comprise entre 4 et quelques centaines de Mega-octets. Enfin, la couche logicielle haute est composée de programmes spécifiques applicatifs usuels. Il peut être question de jeux, de langages de programmation, de logiciels de calculs, des systèmes de gestion de fichiers et de bases de données, d'utilitaires divers, de logiciels de Bureautique (traitements de textes, tableurs, présentation graphique...).

Chapitre 2

Notions d'algorithme et de programme

2.1 Introduction à l'Algorithmique

L'algorithmique est une science très ancienne. Son nom vient d'un mathématicien arabe du IXème siècle EL KHAWRISMI. Des mathématiciens grecs comme Euclide ou Archimède en ont été les précurseurs (calcul du PGCD de 2 nombres, calcul du nombre π).

2.1.1 Qu'est-ce qu'un algorithme ?

Il existe plusieurs définitions possibles de l'Algorithme :

Definition 1 *Spécification d'un schéma de calcul, sous forme d'une suite finie d'opérations élémentaires obéissant à un enchaînement déterminé*

Definition 2 *Ensemble de règles opératoires dont l'application permet de résoudre un problème donné au moyen d'un nombre fini d'opérations*

Definition 3 *Etant donné un traitement à effectuer, un algorithme du traitement est l'énoncé d'une séquence d'actions primitives réalisant ce traitement*

2.1.2 Propriétés d'un algorithme

Tout algorithme décrit un traitement sur un nombre fini de données est la composition d'un nombre fini d'étapes, chaque étape étant formée d'un nombre fini d'opérations dont chacune est définie de façon rigoureuse et non ambiguë, effective, c'est-à-dire pouvant être effectivement réalisée par une machine Quelque soit la donnée sur laquelle il travaille, un algorithme doit toujours se terminer et fournir un résultat. Un algorithme est déterministe : étant donné un algorithme, toute exécution de celui-ci sur les mêmes données donne lieu à la même suite d'opérations et aboutit au même résultat. Il existe une relation étroite entre la notion de programme informatique et celle d'algorithme. Un programme informatique est écrit dans un langage de programmation et s'exécute sur un ordinateur (processeur, mémoire et organes d'Entrées-Sorties). En résumé, un algorithme doit être

$\Longrightarrow$ PRECIS : Il doit indiquer :
- l'ordre des étapes qui le constituent
- à quel moment il faut cesser une action
- à quel moment il faut en commencer une autre
- comment choisir entre différentes possibilités

$\Longrightarrow$ DETERMINISTE
- Une suite d'exécutions à partir des mêmes données doit produire des résultats identiques.

$\Longrightarrow$ FINI DANS LE TEMPS
- c'est-à-dire s'arrêter au bout d'un temps fini.

Exemple 4 *Résolution de l'équation du premier degré $A\ X + B = 0$*

```
Lire les coefficients A et B
Si A est non nul Alors
                    affecter à X la valeur - B / A
                    afficher à l'écran la valeur de X
                 Sinon
                    Si B est nul   Alors
                                       écrire "tout réel est solution"
                                   Sinon
                                       écrire "pas de solution"
```

2.1.3 Place de l'algorithme dans la résolution d'un problème informatique

Un algorithme doit être exprimé dans un langage de programmation pour être compris et exécuté par un ordinateur§.Le programme constitue le codage d'un algorithme dans un langage de programmation donné, et qui peut être traité par une machine donnée. L'écriture d'un programme n'est qu'une étape dans le processus de programmation, comme le montre le schéma suivant :

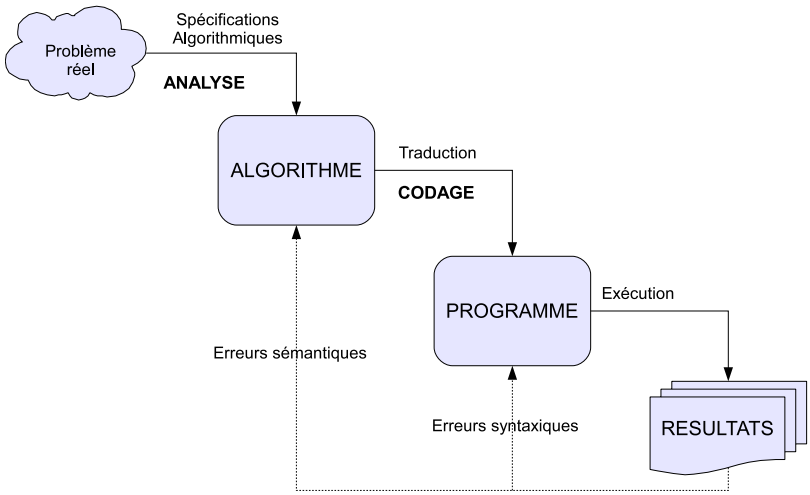


FIGURE 2.1 – Les différentes étapes du processus de programmation.

L'analyse du problème consiste à définir les différentes étapes de sa résolution. Elle permet de définir le contenu d'un programme en terme de données et d'actions. Le problème initial est décomposé en sous problèmes plus simples à résoudre auxquels on associe une spécification formelle ayant des conditions d'entées et le(s) résultat(s) que l'on souhaiterait obtenir. L'ensemble de ces spécifications représente l'algorithme. La phase suivante consiste à traduire l'algorithme dans un langage de programmation donné tout en respectant strictement la syntaxe du langage. Lors de l'étape d'exécution, soit des erreurs syntaxiques sont signalées, ce qui entraîne des corrections en général simples à effectuer, soit des erreurs sémantiques plus difficiles à déceler. Dans le cas d'erreurs syntaxiques les retours vers le programme peuvent être fréquents. Dans le cas d'erreurs

sémantiques, le programme produit des résultats qui ne correspondent pas à ceux escomptés : les retours vers l'analyse (l'algorithme) sont alors inévitables.

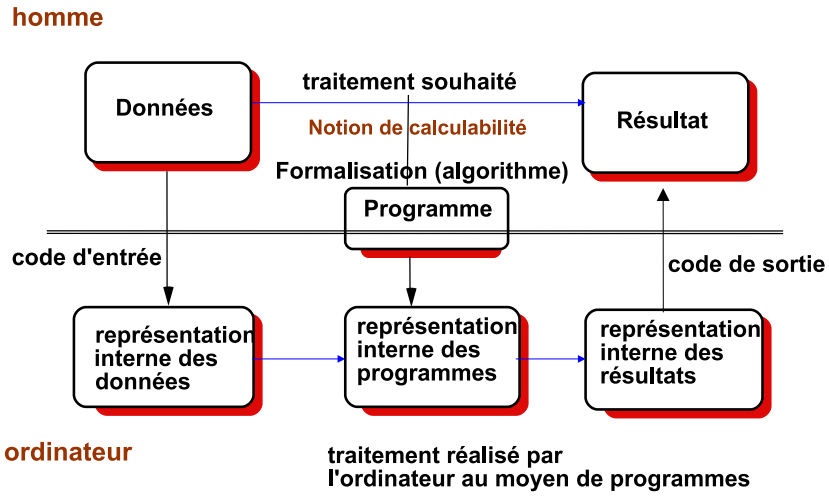


FIGURE 2.2 – Traitement des données.

2.2 Notion de pseudo-langage

Certains langages très connus du grand public sont largement utilisés dans des domaines très divers (PASCAL, C, LISP, ADA, COBOL, etc.). On distingue généralement les langages de bas niveau (proches de la machine : Assembleur) et les langages évolués (dits de haut niveau). De tout temps, les chercheurs ont essayé de mettre au point des langages permettant de se détacher le plus possible de la machine sur laquelle les programmes seront exécutés. D'un autre côté, un algorithme n'a d'intérêt que s'il peut être compris et utilisé par un grand nombre de programmeurs. Il a donc fallu élaborer un langage de description suffisamment formel, pour permettre des implantations dans différents langages de programmation peu fastidieuses, et d'un niveau suffisant pour qu'il soit un outil de communication efficace. Un tel langage s'appelle pseudo-langage. En résumé, l'avantage du pseudo-langage est qu'il permet d'écrire tout algorithme de façon formelle, c'est-à-dire suffisamment précise, tout en restant compréhensible pour l'ensemble des informaticiens. La phase de programmation se trouvera nécessairement allégée, puisqu'elle se résumera à adapter l'ensemble des opérations décrites aux spécificités du langage utilisé.

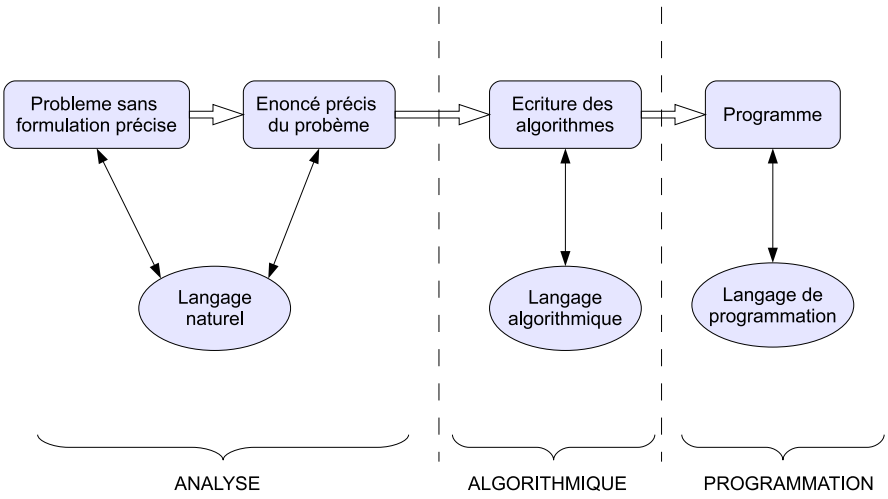


FIGURE 2.3 – Notion de pseudo langage.

2.3 Elaboration d'un algorithme

On peut distinguer quatre phase principales dans l'élaboration d'un algorithme.

- Analyse du problème
- Expression d'une solution en langage courant
- Expression d'une solution en pseudo-langage
- Tests et Vérification de l'adéquation de la solution

Analyse du problème :L'analyse consiste à bien comprendre l'énoncé du problème : Il est inutile et dangereux de passer à la phase suivante si vous n'avez pas bien discerné le problème.

Expression du raisonnement : Bien souvent, quelques lignes écrites en langage courant suffisent pour décrire succinctement l'essentiel du problème. L'intérêt de cette étape est qu'elle permet de vérifier rapidement que l'on se trouve sur la bonne voie. De plus, ces quelques lignes seront un support efficace lors de l'écriture de l'algorithme.

Expression d'une solution en pseudo-langage : Il peut arriver que plusieurs solutions répondent à un problème donné. Il faudra choisir la solution la plus judicieuse et rester cohérent jusqu'au bout.

Tests et Vérification de l'adéquation de la solution : Vérifier l'exactitude du comportement de l'algorithme, son bon déroulement. Si l'algorithme ne répond pas

parfaitement à toutes les requêtes exprimées dans l'énoncé du problème, retournez à la phase n°1.

2.4 Programmation

Il existe différents logiciels réalisés par d'autres programmeurs et prêts à l'emploi :
- jeux,
- Bureautique (traitements de textes, tableurs...),
- de gestion (fichiers, bases de données...),
- retouche de son ou d'images...

Cependant, il arrive fréquemment que l'on doive concevoir soi-même un logiciel, pour une tâche spécifique, inédite. Il est alors indispensable de programmer. On utilise dans ce cas un langage de programmation.

De nos jours, il existe de nombreux langages informatiques, dédiés à la programmation. Les plus connus sont FORTRAN, COBOL, BASIC, C, ADA, C++, LISP, PROLOG, JAVA... Dans le cadre de ce cours, nous utiliserons le langage PASCAL, très pédagogique, et bien adapté pour les débutants, car il donne de bonnes bases en programmation structurée.

2.5 Objets simples, types et actions élémentaires

Un algorithme manipule des objets au sens informatique. Ces objets pourront être des données qui seront fournies en entrée, des résultats produits par l'algorithme ou des outils nécessaires au bon déroulement de l'algorithme.

2.5.1 Type d'un objet

En mathématiques, lorsque l'on utilise des objets, on précise leur type.

Exemple 5 $x \in \mathbb{R}, \quad i \in \mathbb{Z}$

x et i appartiennent à des types dont on connaît les propriétés

En informatique, les objets manipulés par un algorithme doivent appartenir à un type connu au moment de leur utilisation. Tout objet peut être caractérisé par un type qui indique :

- les ensembles de valeurs que peut prendre l'objet.
- les actions autorisées sur cet objet.

Les objets simples sont :

- des nombres (par exemple 3 , 7 , 3.141592 , 1996).
- des caractères ou des chaînes de caractères (par exemple : 'A' , '45' , 'BONJOUR').
- des valeurs booléennes (VRAI , FAUX)

On distingue généralement :

- les types scalaires qui sont par définition totalement ordonnés
- les types structurés qui regroupent sous un même nom une collection d'objets élémentaires qui peuvent être de même type (type homogène) ou de type différents (type hétérogène). Ces types structurés seront vus ultérieurement.

Les différents types simples sont les suivants :

- **Type entier** : prend ses valeurs dans un sous-ensemble des entiers relatifs. C'est un ensemble fini dans lequel chaque élément possède un successeur et un prédécesseur.
- **Type réel** : prend ses valeurs dans un sous-ensemble de réels décimaux signés. Dans la plupart des langages, cet ensemble n'est pas un ensemble fini. On ne peut trouver de successeur ou de prédécesseur à un réel donné.
- **Type caractère** : prend ses valeurs dans l'ensemble des caractères de la table ASCII.
- **Type chaîne de caractère** : se compose d'une suite de symboles de type caractère
- **Type booléen** : type logique qui peut prendre les valeurs VRAI ou FAUX.

En résumé,
objet $\Longrightarrow$ donnée ou algorithme
algorithme $\Longrightarrow$ procédure ou fonction
donnée $\Longrightarrow$ constante ou variable
constante ou variable $\Longrightarrow$ type scalaire ou type structuré
type structuré $\Longrightarrow$ type homogène ou hétérogène

2.5.2 Les objets

Pour désigner ces différents objets, on utilisera des chaînes de caractères qui seront appelées les identificateurs des objets. Pour différentier un identificateur d'un nombre, un identificateur commence par une lettre et ne comporte pas d'espace. De plus, on essaiera toujours de choisir des noms explicites afin de faciliter la relecture et l'éventuelle

maintenance de l'algorithme par un tiers.

Un objet peut être :

- une constante ou une variable s'il s'agit d'une donnée
- au sens large, une fonction (ou une procédure) s'il s'agit d'un algorithme

2.5.3 Actions élémentaires

Les actions élémentaires sur une donnée dépendent évidemment du type de cette donnée et de sa catégorie (variable ou constante).

2.5.3.1 Opérateurs sur les types simples

Opérateur	Notation	Type des opérandes	Type du résultat
+ et - unaires	+ -	entier ou réel	celui de l'opérande
négation logique	NON	booléen	booléen
Puissance	↑	entier ou réel	entier ou réel
Multiplication	*	entier ou réel	entier ou réel
Division entière	DIV	entier	entier
Division	/	entier ou réel	réel
Reste(modulo)	MOD	entier	entier
Addition	+	entier ou réel	entier ou réel
Soustraction	-	entier ou réel	entier ou réel
Comparaison	$<$, $\leq$, $>$, $\geq$, $<>$, $=$	tout type	booléen
ET logique	ET	booléen	booléen
OU logique	OU	booléen	booléen

TABLEAU 2.1 – Opérateurs sur les types simples

Dans le tableau 2.1, les opérateurs sont classés par ordre de priorité décroissante mais attention dans l'utilisation des priorités car il existe souvent des différences d'un langage de programmation à l'autre. En absence de parenthèses, l'évaluation se fait de gauche à droite.

2.5.3.2 Affectation

L'affectation a pour rôle d'attribuer une valeur, résultat d'une évaluation, à un objet. La valeur doit être compatible avec le type de la valeur à gauche de l'affectation. Le symbole utilisé pour l'affectation est ← ou :=

Exemple 6

```
X <-- 1       ( X prend la valeur 1 )
X <-- 2*3+5   ( X prend la valeur du résultat de l'opération 2*3+5)
D <-- D+1     ( D augmente de 1)
prix_total <- nb_kg * prix_du_kg  (Si nb_kg est de type entier et
                                   prix_du_kg est de type  réel alors
                                   prix_total doit être de type réel)
```

2.5.3.3 Lecture et écriture

Ces actions permettent d'assurer l'interface entre l'environnement externe (l'utilisateur) et l'algorithme.

```
Lire(Valeur1, Valeur2 ...);
Ecrire(Valeur3, Valeur4 ...);
Ecrire ('Le résultat du calcul est : ',prix_total);
```

Exemple 7

```
ALGORITHME Epicier;
VAR prix_total , prix_du_kg : REEL;
    nb_kg : ENTIER;
DEBUT
    Ecrire('Entrez le prix d'un kilogramme de choux : ');
    Lire(prix_du_kg);
    Ecrire ('Entrez le nombre de kilogramme de choux : ');
    Lire (nb_kg);
    prix_total <- prix_du_kg * nb_kg;
    Ecrire ('Le prix total de l''achat est :',prix_total);
FIN .
```

2.5.3.4 Commentaires

Afin d'améliorer la lisibilité d'un algorithme, on peut utiliser des commentaires. Un commentaire est une suite de caractères quelconques encadrée par les symboles (* et *). Cette suite de caractère ne sera pas exécutée. Elle sert seulement à documenter le programme pour le rendre lisible par un tiers.

Exemple 8 *(* ceci est un commentaire *)*

Chapitre 3

Le langage Pascal

3.1 Historique

Le langage de programmation Pascal a été conçu au début des années 70 par N. Wirth. Depuis l'utilisation de ce langage s'est développé dans les universités et la communauté scientifique. Son succès toujours croissant a montré qu'il s'agit du langage qui durant les années 80, a détrôné les langages tels que FORTRAN, les dérivés de ALGOL. Le Pascal est facile à enseigner et à apprendre. Il permet d'écrire des programmes très lisibles et structurés. Il dispose entre autres de facilités de manipulation de données.

3.2 Programmer en Pascal

- **Programme**

  Un programme est une suite d'instructions destinées à l'ordinateur. Or le langage le langage de l'ordinateur est un langage machine qui n'utilise que deux symboles 0 et 1. On utilise donc un lange de programmation, ici le langage Pascal permettant de produire des programmes lisibles et facilement modifiables. Ces programmes sont traduits en langage machine par un compilateur

- **Code source**

  Un programme pascal (prog par exemple) peut être écrit avec un simple éditeur de texte. Le programme ainsi réalisé est stocké sous forme de fichier avec l'extension .pas (prog.pas).

- **Compilation et édition de liens**

La version en langage machine d'un programme s'appelle aussi le code objet. L'éditeur de liens est un programme qui intègre, après la compilation du fichier source, le code machine de toutes les fonctions utilisées dans le programme et non définies à l'intérieur. A partir du code objet du programme l'éditeur de liens génère un fichier exécutable d'extension .exe(prog.exe). Le fichier exécutable renferme alors le programme qui peut être chargée dans la mémoire pour être exécuté.

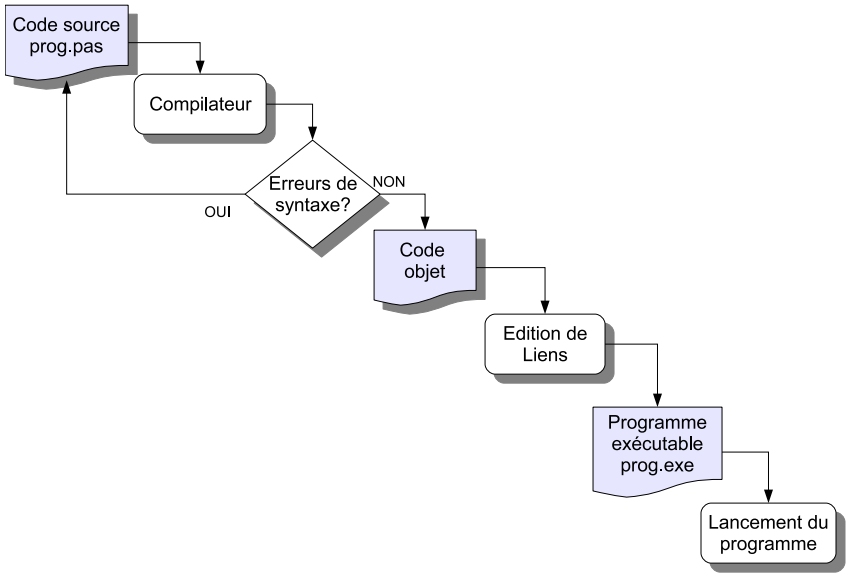


FIGURE 3.1 – Cycle de la programmation en Pascal.

3.3 Les constituants élémentaires du Pascal

3.3.1 L'Alphabet

L'alphabet Pascal est constitué des éléments suivants :

- Les majuscules : **A, B,..., Z** (26 caractères)
- Les minuscules : **a, b,..., z** (26 caractères)
- Le caractère « blanc »
- Les chiffres : **0, 1,..., 9**
- Les symboles spéciaux :
- Les opérateurs :
  - arithmétiques : + - * /

- relationnels : $<$; $<>$; $=$; $<=$; $>=$; $<>$
- Les séparateurs : ( ) ; { } ; [ ] ;(* *)
- Le signe« pointeur » : ^
- Les signes de ponctuation : . , ; : ' !?

Remarque **:**
Le Pascal n'est pas sensible à la casse. Il ne fait pas de distinction entre majuscule et minuscule. L'écriture begin est correcte ainsi que les écritures suivantes BEGIN ou Begin

3.3.2 Les mots du langage

Un mot est une suite de caractères encadrés par des espaces ou des caractères spéciaux.

3.3.2.1 Les mots réservés

- Ne peuvent être redéfinis par l'utilisateur
- Ont une signification précise
- Aident à la construction syntaxique des instructions.

Exemples de mots réservés :

```
AND     BEGIN   END     CASE    PROCEDURE
REPEAT  WHILE   VAR     TYPE    FUNCTION
CONST   FOR     GOTO    IF      UNTIL
WITH    TYPE    ARRAY   DO      ELSE    ... ... ...
```

3.3.2.2 Les identificateurs

Un identificateur est un nom donné à un élément du programme introduit par l'utilisateur (constante, variable, fonction, procédure, programme).

Remarque **:**

- Un identificateur est une suite alphanumérique commençant nécessairement par une lettre de l'alphabet et ne comportant pas d'espaces.
- Possibilité de lier plusieurs mots à l'aide de "_ ".
- Existence d'une limite au nombre de caractères. (dépend du compilateur)

- ***Exemples d'identificateurs légaux***
  `TERME  terme  TRES_LONG_TERME  X23  Z1Z2`
- ***Exemples d'identificateurs non légaux*** :
  `3MOT  U.T.C.  MOT-BIS  A!8  $PROG  AUTRE MOT`

3.3.2.3 Les identificateurs standards

Exemples d'identificateurs standards :

- Fonctions :

  `COS SIN EXP SQR SQRT SUCC PRED`

- Constantes :

  `MAXINT TRUE FALSE`

- Types :

  `INTEGER REAL BOOLEAN CHAR`

- Procédures :

  `READ WRITE NEW RESET REWRITE`

- Fichiers d'entrée-sortie :

  `INPUT OUTPUT`

3.4 Le Langage PASCAL

3.4.1 Caractéristiques globales

- **C'est un Langage Typé**
  - Toutes les variables doivent être pré-déclarées, ce qui évite les erreurs
  - Leur type doit être explicitement défini, ce qui enlève toute ambiguïté
  - Il s'agit d'un langage très 'pédagogique' et bien adapté pour les débutants
- **C'est un Langage Structuré**
  - Organisation du programme en "blocs d'instructions" emboîtés,
  - Utilisation d'identificateurs pour spécifier les blocs,
  - Utilisation d'indentations pour visualiser l'architecture du programme.
- **C'est un Langage Récursif**
  - Programmation concise et efficace

3.4.2 Structure globale d'un programme PASCAL

- EN-TETE
- DECLARATIONS

1. UNITE
2. ETIQUETTE
3. CONSTANTES
4. TYPES
5. VARIABLES
6. FONCTIONS / PROCEDURES

– BLOC D'INSTRUCTIONS EXECUTABLES

3.4.2.1 En-tête

Il s'agit de la première ligne d'un programme PASCAL. L'en-tête commence par le mot réservé PROGRAM suivi d'un identificateur, éventuellement suivi d'une liste de paramètres situés entre parenthèses. La ligne est terminée par un point-virgule.
SYNTAXE : `PROGRAM identificateur (id1,id2, ..., idn) ;`
Exemples d'en-tête :

```
PROGRAM  second_degre ;
PROGRAM  second_degre (input, output);
```

3.4.2.2 Partie déclarative

La zone de déclaration comprend :

– Déclarations d'unité avec `USES`
– Déclarations d'étiquette avec `LABEL`
– Déclarations de constante avec `CONST`
– Déclarations de type avec `TYPE`
– Déclarations de variables avec `VAR`
– Déclaration des fonctions et/ou de procédures respectivement avec `FUNCTION` et `PROCEDURE`

L'ordre indiqué doit être impérativement respecté. Les clauses indiquées sont optionnelles et dépendent des besoins du programmeur. Seuls l'entête et le corps du programme sont obligatoire.

3.4.2.3 Bloc d'instructions

Une instruction est une phrase du langage représentant un ordre ou un ensemble d'ordres qui doivent être exécutés par l'ordinateur. On distingue :

- Les instructions simples :
  - affectation, appels, branchement
- Les instructions structurées
  - instructions composées
  - instructions itératives
  - instructions conditionnelles

  Le bloc d'instructions principal commence par 'BEGIN' et se termine par 'END.'

Exemple 9 *Nous présentons ci-dessous un programme qui donne la moyenne de N nombres entrés au clavier par l'utilisateur, qui précisera le nombre de données qu'il va taper. A ce stade du cours, il n'est pas nécessaire de comprendre le contenu de ce programme. Il suffit simplement de reconnaître l'architecture globale décrite précédemment (déclarations de variables, blocs, indentations, begin...end).*

```
program MOYENNE ;                                    {En-tête}
USES crt;
var     DONNEE, SOMME, MOYENNE : real;
              I, N : integer ;                       {partie déclarative}
begin                                                {début du bloc d'instructions}
    clrsrc;
    writeln('entrer le nombre de données');
    readln(N);                                       {instruction}
    if N > 0 then
                begin
                    SOMME := 0;
                    for I := 1 to  N  do
                        begin
                           read(DONNEE);
                           SOMME := SOMME + DONNEE;
                        end;
                    MOYENNE := SOMME / N;
                    writeln('moyenne =',MOYENNE);
```

```
            end
        else
            writeln('pas de donnees');
end.                                          {Fin du bloc d'instructions}
```

3.4.3 Déclarations de constantes

En Pascal, tout symbole utilisé dans un programme doit être explicitement déclaré.

Une constante est désignée par un identificateur et une valeur, qui sont fixées en début de programme, entre les mots clés `CONST` et `VAR`. La valeur **ne peut pas être** modifiée, et **ne peut pas** être une expression.

Syntaxe

```
identificateur = valeur_constante;
```

ou

```
identificateur : type = valeur_constante;
```

Dans la première forme, le type est sous-entendu (si il y a un point, c'est un réel, sinon un entier ; si il y a des quotes, c'est un caractère (un seul) ou une chaîne de caractères (plusieurs).

L'utilisation de constantes en programmation est vivement conseillée. Elles permettent :

- une notation plus simple
  Exemple : `PI` à la place de `3,141592653` `CONST pourcent : real = 33.3;`
- La possibilité de modifier simplement la valeur spécifiée dans la déclaration au lieu d'en rechercher les occurrences, puis de modifier dans tout le programme.

Exemple 10 *Déclaration de constantes*

- *Déclarations de type numérique*

```
const    DEUX = 2;
         PI = 3.14;
```

– *Déclarations de type booléen*

```
VRAI = true;
FAUX = false;
```

– *Déclarations de type caractère*

```
CARA  = 'A';
```

– *Déclarations de type chaîne de caractères*

```
PHRASE = 'Vaut mieux une fin effroyable qu''un effroi sans fin';
```

3.4.4 Déclaration de types

Un type est un ensemble de valeurs que peut prendre une donnée.

3.4.4.1 Les types standards

Un type standard est un type qui est normalement connu de tout langage Pascal et qui n'a donc pas été déclaré par l'utilisateur. Les différents types standards :

Le type entier : `integer`

- Sur$n$ bits, il sera possible d'utiliser des entiers compris entre -2^{n-1} et $(2^{n-1}-1)$
- Donc sur 16 bits, les entiers seront compris entre -32768 et 32767 (-2^{15} et $+2^{15}-1$).
- Donc sur 32 bits, les entiers seront compris entre -2147483648 et $+2147483647$ (-2^{31} et $+2^{31}-1$).
- Opérateurs sur les entiers :

`abs(x)`	valeur absolue de x
`pred(x)`	$x - 1$
`succ(x)`	$x + 1$
`odd(x)`	`true` si x est impair, `false` sinon.
`sqr(x)`	le carrée de x.
`+ x`	identité.
`- x`	signe opposé
`x + y`	addition
`x - y`	soustraction.
`x * y`	multiplication.
`x / y`	division, fournissant un résultat de type réel.
`x div y`	dividende de la division entière de x par y.
`x mod y`	reste de la division entière, avec y non nul.

Remarques

– Attention, les opérateurs `/`, `div` et `mod`, produisent une erreur à l'exécution si y est nul.

– Lorsqu'une valeur (ou un résultat intermédiaire) dépasse les bornes au cours de l'exécution, on a une erreur appelée débordement arithmétique.

Le type réel : `real`

- Utilisable pour représenter les rééls et les entiers élevés. Le domaine de définition dépend de la machine et du compilateur utilisés.
- 0.0 ; $-21.4E3$ $(= -21,4 \times 10^3 = -21400)$; $1.234E-2$ $(= 1,234 \times 10^{-2})$
- Opérateurs sur un argument x réel : `abs(x)`, `sqr(x)`, `+x`, `-x`. Si l'un au moins des 2 arguments est réel, le résultat est réel pour : `x - y`, `x + y`, `x * y`.
- Résultat réel que l'argument soit entier ou réel : `x / y` (y doit être non nul) ; fonctions `sin(x)`, `cos(x)`, `exp(x)`, `ln(x)`, `sqrt(x)` (*square root*, racine carrée).
- Fonctions prenant un argument réel et fournissant un résultat entier : `trunc(x)` (partie entière), `round(x)` (entier le plus proche). Si le résultat n'est pas représentable sur un `integer`, il y a débordement.

Le type booléen : `boolean`

- Définit les valeurs logiques
- Constantes : `true` (vrai) et `false` (faux)
- Ce type permet la manipulation avec des opérateurs logiques
- Opérateurs booléens : `not` (négation), `and` (et), `or` (ou).

x	y	not x	x and y	x or y
true	true	false	true	true
true	false	false	false	true
false	true	true	false	true
false	false	true	false	false

- Opérateurs de comparaison (entre 2 entiers, 2 réels, 1 entier et 1 réel, 2 chars, 2 booléens) : `<, >, <=, >=, =` (égalité, à ne pas confondre avec l'affectation `:=`), `<>` (différent).
  Le resultat d'une comparaison est un booléen. On peut comparer 2 booléens entre eux, avec la relation d'ordre `false < true`.
- En mémoire, les booléens sont codés sur 1 bit, avec 0 pour false et 1 pour true. De là les relations d'ordre. Les opérateurs booléens `not`, `and`, `or` s'apparentent approximativement à $(1 - x)$, $\times$, $+$.

Le type caractère : `char`

- C'est un unique caractère entouré d'apostrophes
- Ensemble des valeurs de ce type :
  - alphanumériques : `'a' . . 'z'  'A' . . 'Z'  '0' . . '9'`
  - caractère blanc : ' '
  - Le choix et l'ordre des 256 caractères possible dépend de la machine et de la langue. Sur PC, on utilise le code ASCII, où `'A'` est codé par 65, `'B'` par 66, `'a'` par 97, ' ' par 32, `'{'` par 123, etc.
  - Les opérateurs sur les chars sont :

    `ord(c)` numéro d'ordre dans le codage ; ici ¡ code ascii ¿.

    `chr(a)` le résultat est le caractère dont le code ascii est a.

    `succ(c)` caractère suivant `c` dans l'ordre ascii , `chr(ord(c)+1)`

    `prec(c)` caractère précédent `c` dans l'ordre ascii.

  - On peut remplacer `chr(32)` par `#32`, mais pas `chr(i)` par `#i`.

Le type chaîne de caractère : `string` ou `varying`

- Ce type n'est pas disponible sur tous les compilateurs
- Possibilité d'accéder à un caractère particulier de la chaîne, en indiçant la variable qui y fait référence

Exemple 11

```
phrase  <---  ' il fait beau'
phrase[5] <--- 'a'
```

3.4.4.2 Les types scalaires et non standards

Principe : on fabrique les types dont on a besoin par l'intermédiaire du mot réservé `TYPE`. Les types **scalaires standard** sont les **entiers** et les **caractères**.

☛LE TYPE ÉNUMÉRÉ

Un type énuméré est une séquence ordonnée d'identificateurs.

SYNTAXE : `TYPE identificateur = (id1, id2,..., idn) ;`

Exemple 12
```
Type COULEUR  =  (jaune, vert, rouge, bleu, marron);
     SEMAINE=(lundi, mardi, mercredi, jeudi, vendredi, samedi, dimanche);
     SEXE =(masculin, féminin);
     VOYELLE = (A, E, I, O, U);
```

Le mot réservé `TYPE` ne doit être écrit qu'une seule fois.

Avec un type énuméré, on peut donc donner la liste des valeurs possibles les unes après les autres. Les autres types énumérés déjà définis sont les `boolean`, les `char`, mais *aussi* les `integer`. Tous ces types se comportent comme des entiers, et la déclaration qu'on a faite dans l'exemple précédent est traduite sous la forme `lundi = 0`, `mardi = 1` et ainsi de suite. L'avantage est qu'il existe des fonctions définies sur tous les types énumérés :

- la fonction `ORD`, qui donne le numéro dans la liste de déclaration (`ord(mardi)=1`, on commence à `0`) ;
- les fonctions `SUCC` et `PRED`, qui donnent le successeur et le prédecesseur dans la liste : `succ(mardi)= mercredi`, `pred(samedi)=vendredi`. Attention, `pred(lundi)` et `succ(dimanche)` ne sont pas définis. On ne sait pas ce qu'ils valent ;
- les opérateurs de comparaison
- les boucles for ;

Il faut faire attention, les fonctions `readln` et `writeln` ne marchent pas avec ces types.

D'autre part, deux types énumérés différents ne peuvent contenir le même identificateur. $\Longrightarrow$ Ensembles énumérés disjoints

☛LE TYPE INTERVALLE

Un type intervalle est un sous-type d'un type scalaire déjà défini.

SYNTAXE : `TYPE identificateur = [borne inf].. [borne sup] ;` Points importants :

- Valeurs autorisées : toutes celles de l'intervalle
- Deux sortes de type intervalle :
  - les types issus d'un type énuméré standard
  - les types issus d'un type énuméré déclaré.

Exemple 13 *Nous présentons ci-dessous des exemples issus d'un type standard.*

- *Intervalle d'entiers :*

```
Type DECIMAL =  0 .. 9   ;
     OCTAL   =  0 .. 7   ;
     AGE     =  0 .. 150 ;
```

- *Intervalle de caractères :*

```
Type ABC = 'A' .. 'C' ;
     MAJ = 'A' .. 'Z' ;
```

- *A présent, voici quelques exemples issus d'un type non-standard*

```
Type OUVRABLE = lundi  .. vendredi ;
     WEEK-END  = samedi .. dimanche ;
     LETTRES   = 'A' .. 'Z' ;
```

Ordre ascendant requis : borne-inf doit être placé avant borne-sup dans le type énuméré source.

Exemple 14 *Déclarations de type intervalle incorrectes :*

```
Type OCTAL    = 7 .. 0 ;
     OUVRABLE = vendredi .. lundi ;
```

Pas de type intervalle issu du type réel (non scalaire) !

3.4.5 Déclaration de variables

Une variable est une donnée manipulée par un programme et pouvant être modifiée. Elle peut être :

– une donnée d'entrée ;
– le résultat final d'un calcul ;
– un résultat intermédiaire de calcul.

Déclarer une variable, c'est définir l'ensemble des valeurs qu'elle peut prendre. Toutes les variables utilisées dans un programme doivent être déclarées. Une variable représente un objet d'un certain type ; cet objet est désignée par un identificateur. Toutes les variables doivent être déclarées après le `VAR`.

Deux façons pour déclarer une variable :
– à l'aide d'un type standard ou d'un type pré-déclaré
– par une déclaration explicite et spécifique à cette variable de l'ensemble des valeurs qu'elle peut prendre.

  SYNTAXE : `VAR identificateur : type ;`
– `VAR` ne peut apparaître qu'une seule fois
– Possibilité de grouper plusieurs variables pour le même type
– Séparation des variables par une virgule

Exemple 15 *Déclarations de variables*

– *avec référence à un type existant*

```
var    JOUR    :  semaine ;
       A, B, C :  real    ;
       I, J, K :  integer ;
       CONGE   :  week-end ;
       VIVANT  :  boolean ;
```

– *avec déclaration locale explicite :*

```
var  LETTRE :  'A'  . .  'Z' ;
     FEUX   :  (vert, orange, rouge) ;
```

Exemple 16 *Déclaration de constante, type et variable*

```
const  JOUR_MAX =  31 ;
       AN_MIN   =  1901 ;
       AN_MAX   =  2000 ;
type   SIECLE   =  AN_MIN  . .  AN_MAX ;
```

```
    SEMAINE  = (LUNDI, MARDI, MERCREDI,JEUDI,VENDREDI,SAMEDI, DIMANCHE) ;
    ANNEE    = (JANVIER, FEVRIER, MARS, AVRIL, MAI,JUIN, JUILLET,  AOUT,
                SEPTEMBLE, OCTOBRE , NOVEMBRE, DECEMBRE);
var  MOIS       : année ;
     JOUR       : semaine ;
     N_JOUR     : 1 .. jour_max ;
     AN         : siecle ;
     OUVRABLE   : lundi .. vendredi ;
     I, J       : integer ;
     N_ETUDIANT : 1 ..  maxint ;
```

3.4.6 Instructions composées

3.4.6.1 Définition

Les instructions composées permettent de regrouper, dans un même bloc, un ensemble d'instructions qui seront exécutées au même niveau.

Syntaxe : Séquence de deux ou plusieurs instructions comprises entre BEGIN et END et séparées par des points virgules

3.4.6.2 Instruction d'affectation

`<VARIABLE>:=<expression>`

– Evaluation de l'expression (calcul)

– Puis affectation (rangement) dans la variable (identificateur)

Nécessité d'avoir des types compatibles (les mélanges de types sont interdits) Ne pas confondre " :=", l'opérateur d'affectation et "=", l'opérateur de test.

Exemple 17 *Soit* `X` *une variable de type integer et on lui donne comme valeur* `10` `X := 10` *signifie que l'on affecte la valeur* `10` *à la variable* `X` *donc* `X` *vaut* `10`.
On peut tester si `X` *est égal à une certaine valeur avant d'effectuer un calcul :*

```
Si X = 3 alors X := X / 2
```

Ici `X` *vaut toujours 10 car le test* `X = 3` *n'est pas vérifié (puisque la valeur* `10` *a été placée dans* `X`*)*

A la déclaration, les variables ont une valeur indéterminée. On initialise les variables juste après le `BEGIN` pricncipal (on ne peut pas le faire dans la déclaration). Utiliser la

valeur d'une variable non initialisée est une erreur grave !

Exemple 18

```
VAR
a, b, c : integer;
BEGIN
    { Partie initialisation }
    b := 5;
    { Partie principale }
    a := b + c; { ERREUR, c n'a pas été initialisée' }
END.
```

L'opération `identificateur := expression;` est une affectation. On n'a pas le droit d'écrire `id1 := id2 := expr` , ni `expr := id` ni `expr1 := expr2` .

3.4.7 Opérateurs et Fonctions arithmétiques

3.4.7.1 Opérateurs disponibles

Opérateur	Description
+	somme
-	soustraction
*	multiplication
/	division
DIV	division entière (5 div 3 = 1)
MOD	modulo (5 mod 3 = 2)

TABLEAU 3.1 – Opérateurs disponibles

Exemple 19

```
var A, B, C, D : real;
    I, J, K : integer;
begin
   A := 7.4 ; B := 8.3 ;
   C := A + B ;
   D := A / B + C ;
   I  :=  42 ; J := 9 ;
   K := I  mod  J ;     { K vaut 6 }
end.
```

3.4.7.2 Expressions

Une expression désigne une valeur, exprimée par composition d'opérateurs appliquées à des opérandes, qui sont : des valeurs, des constantes, des variables, des appels de fonction ou des sous-expressions.

Exemple 20 *Etant donnée une variable* `x`*, une constante* `max` *et une fonction cos(), chaque ligne contient une expression :*

```
5
x + 3.14
2 * cos(x)
(x < max) or (cos(x-1) > 2 * (x+1))
2.08E3 * x
(x>2) OR (x<8)
```

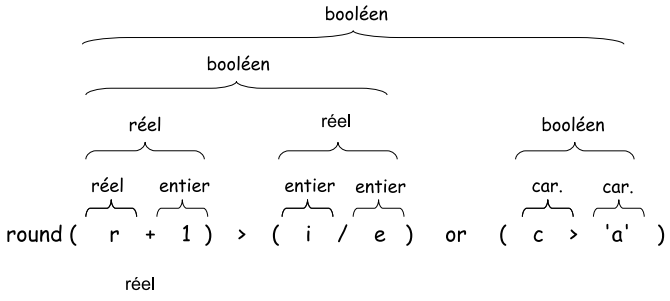
Type des expressions bien formées

Une expression doit être bien formée pour que l'on puisse trouver sa va- leur. Par exemple, `3 * 'a' - true` n'est pas bien formée, et la compilation Pascal échouera. L'expression bien formée a un type, qui dépend des règles d'évaluation de l'ex- pression.

Soit r un réel, i un entier, e une constante entière, `c` un caractère. L'expression

`(round(r+1) > (i/e)) or (c < 'a')`

est bien formée, et sont type est booléen comme on le montre ici :



Remarque Le fait qu'une expression soit bien formée n'implique pas que son évaluation est sans erreur, ce qui peut être le cas ici si `e` est nul.

Règles d'évaluation

L'expression `a + b * c` est évaluée `a + (b * c)` et non pas `(a + b) * c` : ceci parce que le `*` est prioritaire par rapport à `+`. On classe les différents opérateurs par ordre de priorité, les opérateurs de plus forte priorité étant réalisés avant ceux de plus faible priorité. Lorsque deux opérateurs sont de priorité égale, on évalue de gauche é droite. Par exemple `a + b - c` est évaluée `(a + b) - c`, et non pas `a + (b - c)`. Voici la table des priorités classées par ordre décroissant, les opérateurs sur une même ligne ayant une priorité égale.

`() fonction()`	primaire
`+ - not`	unaire
`* / div mod and`	multiplicatif
`+ - or`	additif
`= <> < <= >= >`	relation

Remarque Est-ce que l'expression `a < b or c <= d` est bien formée ? Quel est son type ?

Réponse : non ! Ecrire une telle expression booléenne sans parenthèses est une erreur classique. En effet dans la table de priorités, l'opérateur or a une priorité plus élevée que les opérateurs `<` et `<=`, et donc l'expression sera évaluée `a < (b or c) <= d` , ce qui est faux. L'expression bien formée est ici `(a < b) or (c <= d)` .

3.4.7.3 Fonctions arithmétiques

3.4.7.4 Fonctions logiques

3.5 Entrées / Sorties

Ce sont des échanges d'informations entre la mémoire (variables et constantes) et les périphériques (clavier, écran ...). Types autorisés : réel, booléen, caractères et chaînes de caractères.

Le clavier et l'écran sont gérées comme des fichiers particuliers : ce sont des fichiers texte, toujours ouverts et sans fin ; ils sont désignés par les variables prédéfinies `input` et `output` (dont on ne se sert quasiment jamais).

Pour transmettre des données saisies au clavier à un programme (entrées) ou pour afficher à l'écran les données par un programme (sorties), il faut faire appel à un ensemble

Fonction	Description
ABS (X)	valeur absolue de X
ARCTAN (X)	arctangente de X
CHR (X)	caractère dont le numéro d'ordre est X
COS (X)	cosinus de X
EXP (X)	exponentielle de X
LN (X)	logarithme népérien de X
ORD (X)	numéro d'ordre dans l'ensemble de X
PRED (X)	prédécesseur de X dans son ensemble
ROUND (X)	arrondi de X
SIN (X)	sinus de X
SQR (X)	carré de X
SQRT (X)	racine carrée de X
SUCC (X)	successeur de X dans son ensemble
TRUNC (X)	partie entière de X

TABLEAU 3.2 – Fonctions arithmétiques

Fonction	Description
EOF (X)	vrai si la fin de fichier X est atteinte
EOLN (X)	vrai si fin de ligne du fichier
ODD (X)	vrai si X est impair, faux sinon

TABLEAU 3.3 – Fonctions logiques

de fonctions appartenant à l'unité d'entrées/sorties. Il faut donc faire apparaître en début de programme l'instruction suivante :

```
USES crt ;
```

3.5.1 Sortie (Ecriture)

On utilise la fonction `write` ou `writeln` pour l"affichage formaté des données ; Formaté signifie que l'on contrôle la forme et le format des données La fonction admet la syntaxe suivante :

```
Write(argument1, argument2,..., argumentn)
```

ou

```
Writeln(argument1, argument2,..., argumentn)
```

avec `argument1,..., argumentn` : les arguments à afficher

Exemple 21 `write('bonjour ')`

```
writeln('monsieur')
a :=2+3 ;
writeln('la somme de 2 + 3 donne :',a) ;
```

La fonction `write` *écrit ici à l'écran les arguments (chaîne de caractères, constante, variable) La fonction* `writeln` *écrit la même chose. La seule différence est que à la fin de l'écriture du dernier argument, il y a un passage à la ligne suivante*

⟶ Trois écritures équivalentes :

```
writeln (a, b, c, d);
write (a, b, c, d); writeln;
write(a); write(b); write(c); write(d); writeln;
```

Le résultat de l'affichage dépend du type du paramètre :

Exemple 22

```
VAR e : integer; c : char; b : boolean; r : real; s : string[32];
BEGIN
e := 12; c := 'A'; b := true; r := 23.0; s := 'toto';
writeln (e, '|', c, '|', b, '|', r, '|', s);
END.
```

affiche : `12|A|TRUE|2.300000E+01|toto`

☛***Formatage de l'inpression des variables***

- Soit `v` un entier, un booléen, un caractère ou un string. `write(v:8)` dit à `write` d'afficher `v` sur au moins 8 caractères. Si le nombre de caractères (signe éventuel compris) est `>` 8, `v` est complétement affiché ; si il est `<` 8, des espaces sont rajoutés à gauche pour compléter.
  Ainsi `writeln (e:5, '|', c:3, '|', b:5, '|', s:6);` affiche :
  ␣␣␣`12|`␣␣`A|`␣`TRUE|`␣␣`toto`
- Soit `r` un réel.
  `write(r:10);` dit à `write` d'afficher `r` en notation scientifique, sur au moins `10` caractères, signes de la mantisse et de l'exposant compris. Cette fois c'est d'abord le nombre de chiffres après la virgule qui change de `1` à `10`, puis au besoin des espaces sont ajoutés à gauche. De plus le dernier chiffre de la mantisse affichée est arrondi.
  ```
  r := 2 / 3;
  writeln (r:8, '|', r:10, '|', r:18 );
  ```

affiche :

␣6.7E-01|␣6.667E-01|␣␣6.6666666667E-01

– Autre formattage de `r` réel. `write(r:8:4);` dit à `write` d'afficher `r` en notation simple, sur au moins `8` caractères, dont 4 chffres après la virgule (le dernier étant arrondi).

Ainsi `writeln (r:8:4);` affiche :

␣␣␣0.6667

3.5.2 Entrées (Lecture)

On utilise la fonction `readln` pour la saisie des données depuis le clavier La fonction admet la syntaxe suivante :

`readln(argument1, argument2,..., argumentn)`

avec `argument1,..., argumentn` : les arguments. Le programme va lire ce que l'utilisateur a tapé et va stocker les valeurs dans les variables `argument1,..., argumentn`

Exemple 23

```
Write('Entrez un nombre entier : ') ;
Readln(a) ;
Writeln('vous avez entré la nombre ',a) ;
Write('Entrez 3 nombre réels : ') ;
Readln(b,c,d) ;
```

La fonction `Readln` lit des valeurs sur le périphérique d'entrée standard (clavier) les interprète dans le format de la variable et les range dans les arguments spécifiés. A chaque valeur saisie il faut valider par la touche entrée pour que la saisie soit prise en compte.

Remarque :

`Readln(...);`

$\Longrightarrow$ passage à la ligne suivante en ignorant ce qui reste sur la ligne

`Readln` ; peut être employé sans paramètre

La procédure `read()` permet de lire un ou plusieurs paramètres. `readln()` fait la même chose puis fait un `readln` ;

$\longrightarrow$Trois écritures équivalentes :

```
readln (a, b, c, d);
```

```
read (a, b, c, d); readln;
read(a); read(b); read(c); read(d); readln;
```

Remarques

- L'exécution d'une de ces lignes, on peut rentrer les données en les séparant par des espaces, des tabulations ou des retours chariot ↩.
- Il faut que les données lues correspondent au type attendu de chaque variable, sinon il y a une erreur à l'exécution.

3.5.3 Lecture directe du clavier

Il existe une fonction avec laquelle on peut entrer une valeur sans valider avec la touche entrée. Cette entrée manipule uniquement des caractères. Il faut donc déclarer des variables de type caractère

Exemple 24

```
C:=readkey; {lit une touche au clavier}
 C:=upcase(readkey); {lit une touche au clavier et la convertit en minuscule}
```

Chapitre 4

Structures de contrôle

4.1 Introduction

Dans un langage impératif, on peut définir l'état d'un programme en cours d'exécution par deux choses :

– L'ensemble des variables du programme ;

– L'instruction qui doit être exécutée

L'exécution d'un programme est alors une séquence d'affectations qui font passer d'un état initial à un état final considéré comme résultat. Les structures de contrôle définissent comment les affectations s'enchaînent séquentiellement.

4.2 La structure séquentielle

4.2.1 La notion de séquence

Une structure séquentielle ou séquence est une suite d'instructions rangées dans l'ordre où elles sont écrites. On parle d'une structure séquentielle chaque fois qu'il s'agit d'une séquence d'instructions simples devant être parcourues l'une après l'autre. Il apparaît immédiatement qu'une séquence est une structure algorithmique très élémentaire. Ainsi par exemple, si S1 et S2 représentent deux instructions, leur composition est notée :

```
S1; S2;
```

respectivement :

```
S1;
```

`S2;`

Ce qui signifie que :

- les instructions S1 et S2 sont exécutées une par une,
- chacune d'elles est exécutée exactement une fois,
- l'ordre dans lequel elles sont exécutées est le même que celui dans lequel elles ont été écrites,
- terminer l'instruction S2 implique de finir la structure séquentielle.

Il apparaît immédiatement que dans une structure séquentielle l'ordre des instructions joue un rôle fondamental. Ainsi par exemple, dans la séquence d'instructions.Ainsi par exemple, dans la séquence d'instructions

```
X1 := 2;
X2 := X1+3;
Writeln( X2 );
```

d'abord la valeur 2 est affectée à la variable `X1`, puis, la valeur 5 est affectée à la variable `X2` et le contenu de la variable `X2` est écrit sur le fichier standard de sortie, c'est-à-dire l'écran. Il n'est pas possible de revenir en arrière pour recommencer l'opération. L'exécution d'une instruction déterminée n'est possible que lorsque toutes les instructions qui la précèdent ont été exécutées.

Il est évident que la notion de séquence peut être généralisée pour un nombre quelconque N d'instructions : `S1; S2; S3; ... ; SN;`

La figurereprésente une traduction graphique de la structure séquentielle. Une telle représentation est appelée *Algorigramme* , *Ordinogramme* ou encore *Organigramme*.

Exemple 25 *Calcul d'un produit de 2 nombres*

```
Algorithm Produit;
Variable
    a,b : réel; (*opérandes*)
    p : réel ; (* résultat du produit*)
Début
    Ecrire('Saisir le nombre a ');
    Lire(a);
    Ecrire('Saisir le nombre b ');
    Lire(b);
```

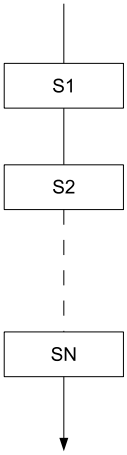


FIGURE 4.1 – Algorigramme de la structure séquentielle.

```
    p:=a * b;
    Ecrire (p);
Fin.
```

4.2.2 Les instructions composées

Une instruction composée est une série de N ($N \geq 2$) instructions qui doivent être exécutées en séquence. Une instruction composée est délimitée par les mots réservés `BEGIN` et `END` qui indiquent respectivement le début et la fin de l'instruction composée.

Voici quelques exemples d'instructions composées :

Exemple 26 `BEGIN`

```
Readln( X, Y );
Writeln( X, Y );
Z := X;
X := Y;
Y := Z;
Writeln( X, Y )
END;
```

Exemple 27 `S1;`
`S2`
`BEGIN S31; S32; S33 END;`
`S4;`
`BEGIN`
`S51;`
`BEGIN S521; S522 END;`
`S53`
`END;`
`S6;`

4.3 Les structures alternatives

Une instruction alternative permet de faire le choix entre une, deux ou plusieurs actions suivant qu'une certaine condition est remplie ou non. Une telle structure algorithmique est encore appelée sélection. On distingue deux types d'instructions alternatives :

- le choix simple (instruction `IF`) et
- la sélection multiple.(instruction `CASE`)

4.3.1 Le choix simple

Syntaxe :

☛En pseudo-code

```
SI  <condition> ALORS
                      <bloc_instructions1>
                SINON
                      <bloc_instructions2>
```

☛en PASCAL

```
IF  <condition> THEN
                     <bloc_instructions1>
                ELSE
                     <bloc_instructions2>
```

Cette instruction exprime les instructions qui sont exécutées suivant que la condition (l'expression booléenne) ¡condition¿ est remplie. Elle est interprétée de la manière suivante :

SEMANTIQUE :

« Si la condition `<condition>` est *vraie*, alors exécuter l'instruction qui suit le mot réservé `THEN`, sinon exécuter l'instruction qui suit le mot réservé `ELSE`." »

Dans cette interprétation, l'exécution des instructions `<bloc_instructions1>`et `<bloc_instructions2>` est mutuellement exclusive ce qui signifie que seule une des deux instructions sera exécutée.

Points importants :

- Surtout pas de point virgule immédiatement avant le ELSE !!!
- Instruction alternative : facultative En effet les deux formes suivantes sont équivalentes.
  - ```
    IF < condition > THEN
                        BEGIN
                            < instruction >;
                        END;
    ```
  - ```
    IF < condition > THEN
                        BEGIN
                            < instruction >;
                        END
                    ELSE ; { rien }
    ```

- Valeur de la condition : booléenne
- Blocs d'instructions :
  - Instructions simples
  - Instructions structurées
  - Instructions composées.

Organigrammes :

- sans SINON
- avec SINON

Exemple 28 *Calcul d'une racine carrée (avec* `SI... ALORS`*)*

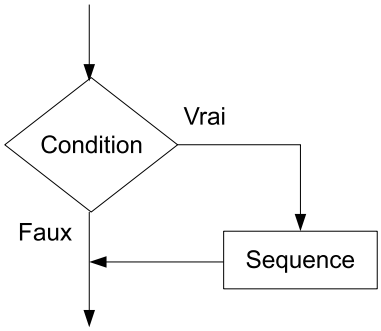


FIGURE 4.2 – Organigramme de la sélection sans alternative

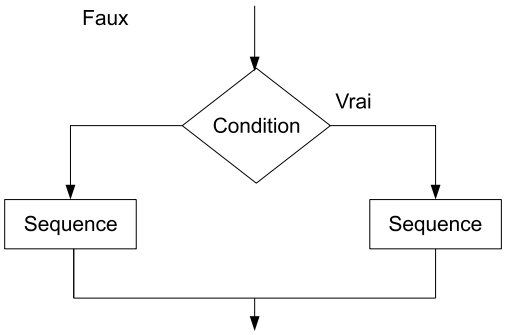


FIGURE 4.3 – Organigramme de la sélection avec alternative

```
Algorithm SI_ALORS;
Variable
    x: réel ; (*opérande*)
    r: réel ;  (*résultat de la racine carrée*)
Début
    Ecrire ('Saisir le nombre x');
    Lire (x);
    Si x > 0  Alors
    Début
        r := racine (x);
        Ecrire (r);
    Fin
Fin.
```

Exemple 29 *Calcul d'une racine carrée (*`SI... ALORS... SINON`*)*

```
Algorithm SI_ALORS_SINON;
Variables :
    x: réel; (*opérande*)
    r: réel ;(*résultat de la racine carrée*)
Début
    Ecrire('Saisir le nombre x');
    Lire (x);
    Si x < 0 Alors
                Ecrire ('x est négatif')
             Sinon
                Début
                    r := racine (x);
                    Ecrire (r);
                Fin

Fin.
```

Ambiguïtés syntaxiques

Il faut bien percevoir que la formulation de l'instruction `IF` n'est pas toujours sans ambiguïté. Ainsi par exemple, l'ambiguïté syntaxique de la construction :

`IF < expr1 > THEN IF < expr2 > THEN < st1 > ELSE < st2 >;`

peut être évitée par le réécriture suivante :

```
IF < expr1 > THEN
                          IF < expr2 > THEN < st1 >
                                                ELSE < st2 >;
```

On dit ici que les instructions `IF` sont imbriquées.

Afin d'éviter des ambiguïtés de ce genre lors de la lecture d'un programme, il est vivement recommandé de bien mettre un `THEN` et un `ELSE` de la même structure alternative au même niveau vertical afin de ne pas les mélanger entre eux. Cette façon de procéder est appelée indentation ou paragraphage. L'identation est très souvent effectuée de manière automatique par un outil qui accompagne le compilateur (indenteur ou paragrapheur).

En général, une telle ambiguïté syntaxique est écartée définitivement soit en utilisant les parenthèses symboliques `BEGIN` et `END`, soit en respectant la règle suivante :

Règle : « "*La partie* `ELSE` *se rapporte toujours au mot réservé* `IF` *précédent le plus proche pour lequel il n'existe pas de partie* `ELSE`. »

Dans une construction de structures alternatives imbriquées il doit y avoir autant de mots `THEN` que de mots `IF`. Ainsi par exemple, dans le texte Pascal :

```
IF N>0 THEN IF A>B THEN Max := A ELSE Max := B;
```

la partie `ELSE` se rapporte au mot réservé `IF` situé à l'intérieur de la séquence ce qui peut être élucidé en écrivant :

```
IF N>0 THEN
                IF A>B THEN Max := A
                            ELSE Max := B;
```

Si ce n'est pas ce qu'on a voulu exprimer, il faut se servir des parenthèses symboliques `BEGIN` et `END` pour forcer des appartenances respectives comme par exemple :

```
IF N>0 THEN
                  BEGIN
                       IF A>B THEN Max := A
                  END
             ELSE Max := B;
```

Le lecteur se rendra bien compte de la signification différente des deux constructions précédentes.

Exemple 30 *Equation du premier degré avec C=0*
Ecrire un programme qui résoud une équation du premier degré Ax+b=0 qui lit les valeurs de A et B entrées par l'utilisateur.

```
Program Premier_Degre;
Var
        A, B : real;
Begin
     write('entrez les coefficients A et B : ');
     readln(A,B);
     if A=0   then {évaluation de la condition}
               if B=0   then
```

```
                        writeln('Indéterminé !')
                    else
                        writeln('Impossible !')
                else
                    writeln('La solution est : ',-B/A:10:3);
End.
```

Exemple 31 ***Maximum de deux nombres***
Ecrire un programme qui calcule le maximum de deux nombres entrés au clavier.

```
Program MAXIMUM_DEUX ;
var  X,Y : real ;
     MAX : real ;
Begin
     writeln('Tapez les deux nombres:')
     read (X, Y) ;
     if  X > Y  then {évaluation de la condition}
                    MAX := X
                else
                    MAX := Y ;
       writeln('Le plus grand nombre est ',MAX);
End.
```

4.3.2 La sélection multiple : le CAS...

Cette méthode est utilisée pour tester une solution parmi N. Par exemple, au cas où un menu est proposé à l'utilisateur (1) pour lire, 2) pour écrire, 3) pour calculer, 4) pour sortir etc...), il est important de tester si l'utilisateur a tapé 1, 2, 3 ou 4. Au lieu d'utiliser plusieurs `IF... THEN... ELSE...` imbriqués, il est préférable de choisir une sélection multiple (`CASE` en Pascal). Ainsi au lieu d'écrire :

```
IF reponse=1 THEN
               Instructions de lecture...
             ELSE
               IF reponse=2 THEN
```

```
            Instructions d'écriture...
      ELSE
            IF reponse=3 THEN
                        instructions de calcul...
```

avec un organigramme représenté par la figure 4.4,

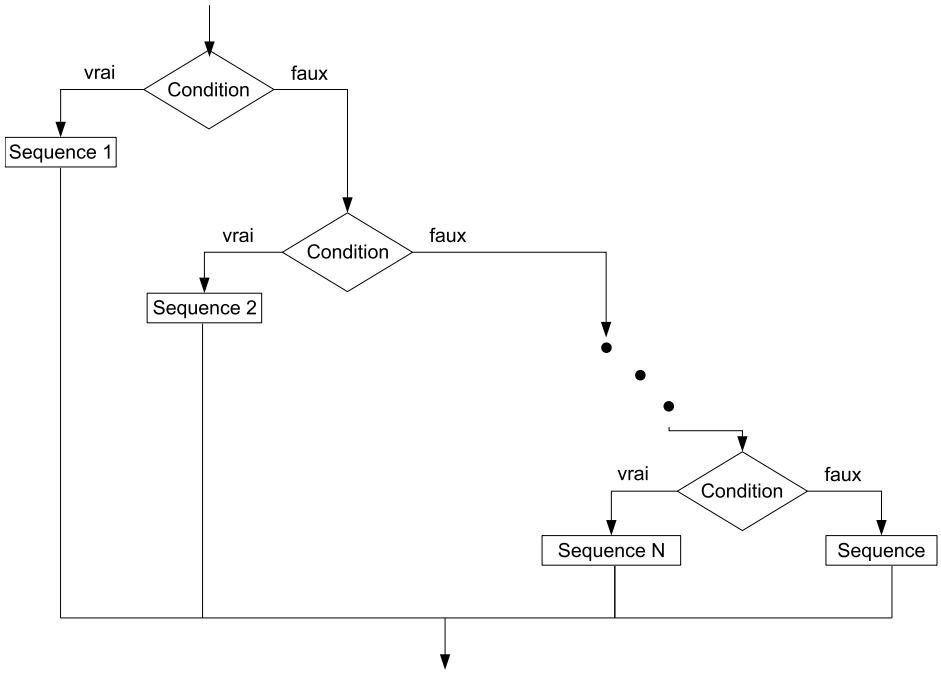


FIGURE 4.4 – Organigramme de la structure avec IF imbriqués

Il serait préférable d'écrire :

```
CASE reponse OF
          1 : Instructions de lecture...
          2 : Instructions d'écriture...
          3 : instructions de calcul...
       End;
```

L'instruction de *sélection multiple* ou instruction `CASE` encore appelée *analyse par cas* est une généralisation supplémentaire de l'instruction `IF`. Elle est fréquemment utilisée

en programmation s'il faut faire le choix entre plus de deux alternatives. Sa formulation est simplifiée par l'instruction `CASE`.

Syntaxe

☛ en pseudo-code

```
CAS variable PARMI
  constante1  : suite d'instructions1
  constante2  : suite d'instructions2
  intervalle1 : suite d'instructions3
  ...
  SINON suite d'instructions par defaut
END;
```

☛En Pascal

```
CASE variable OF
  constante1 : suite d'instructions1
  constante2 : suite d'instructions2
  intervalle1 : suite d'instructions3
  ...
  ELSE suite d'instructions par defaut
END;
```

La séquence précédente est interprétée de la manière suivante :

« Si variable prend la valeur `constante1` , alors exécuter la suite d'`instructions 1`. Si variable prend la valeur `constante2`, alors exécuter la suite d'`instructions2`, si variable est dans l'`intervalle1` alors, exécuter la suite d'`instructions3` etc. , sinon exécuter la suite d'`instructions par défaut` qui suit le mot réservé `ELSE`. »

- Comme dans l'instruction `IF`, l'exécution de chaque branche est mutuellement exclusive.
- La variable variable est appelée sélecteur et doit être d'un type scalaire.
- Les constantes `CASE` doivent être toutes différentes et du même type que le sélecteur. Elles sont interprétées comme des étiquettes.
- Seules les égalités sont possibles au niveau du test (Pas de comparaisons de type `<,>,<=,>=` ou `<>`) On peut néanmoins utiliser des intervalles .

– On peut donner une liste de constantes, ou des intervalles de constantes. **Attention**, chaque valeur possible ne doit être représentée qu'une fois au plus (sinon il y a erreur à la compilation). Par exemple, on ne peut pas faire des intervalles se chevauchant, comme `3..6` et `5..10`, les cas `5` et `6` étant représentés `2` fois.

Exemple 32 *Simuler une calculatrice*

```
Program calculette ;
  var  A, B : real ;
         RESULTAT : real;
         TOUCHE : char;
   Begin
       write('entrez une opération ');
       write('(taper un nombre, un opérateur puis un nombre) : ');
       readln(A,TOUCHE,B);
       case TOUCHE of
           '+' : RESULTAT:= A+B;
           '-' : RESULTAT:= A-B;
           '*' : RESULTAT:= A*B;
           '/' : RESULTAT:= A/B;
          end;
       writeln(A, TOUCHE, B,' = ', RESULTAT);
   end.
```

Exemple 33 *Ecrire un programme qui lit un caractère, puis classe ce caractère comme espace, lettre, digit ou autre.*

```
PROGRAM caractere;
TYPE  nat_t = (Espace, Lettre, Digit, Autre);
VAR  nat : nat_t; { nature }
       c : char;
BEGIN
    write ('Rentrez un caractere :');
    readln(c);
    { analyse de c }
    case c of
```

```
        'a'..'z', 'A'..'Z', '_' : nat := Lettre;
        '0'..'9' : nat := Digit;
        ' ' : nat := Espace;
        else nat := Autre;
    end; { case c }
    { affichage de nat }
    case nat of
        Espace : writeln ('Espace');
        Lettre : writeln ('Lettre');
        Digit : writeln ('Digit');
        Autre : writeln ('Autre');
        else { case nat }
        writeln ('Erreur case nat : ', ord(nat), ' non prevu');
    end; { case nat }
END.
```

4.3.3 Le concept de condition

Une *condition* est une expression booléenne. La valeur d'une condition est du type booléen dont les seules valeurs sont les valeurs de vérité `True` et `False`. Une valeur booléenne est obtenue en appliquant les opérateurs booléens `AND`, `OR` et `NOT` ainsi que les opérateurs relationnels `=`, `<>`, `<`, `<=`, `>`, `>=` à des opérandes booléennes.

On envisage les *conditions de relation simple* et les *conditions de relation composée*. Dans une condition de relation simple deux expressions arithmétiques sont comparées au moyen des opérateurs relationnels. Une condition de relation composée se construit au moyen de conditions de relation simple reliées les unes aux autres par les opérateurs booléens.

Exemple 34 `NOT( A<=B )`
`( A>3 ) OR ( B<5 )`
`( X>3 MOD 4 ) AND ( Sqr( B )<=6 )`

L'opérateur `AND` permet de simplifier l'écriture d'une série d'instructions `IF` imbriquées de la forme suivante :

```
IF < expression 1 > THEN
```

```
                    BEGIN
                       IF < expression 2 > THEN
                                              BEGIN
                                                < statement 1 >;
                                              END
                                           ELSE
                                              BEGIN
                                                < statement 2 >;
                                              END;
                    END
                       ELSE
                          BEGIN
                            < statement 2 >;
                          END;
```

sous la forme :

```
IF < expression 1 > AND < expression 2 > THEN
                                            BEGIN
                                               < statement 1 >;
                                            END
                                         ELSE
                                            BEGIN
                                               < statement 2 >;
                                            END;
```

L'opérateur `OR` permet de simplifier l'écriture d'une série d'instructions `IF` imbriquées de la forme suivante :

```
IF < expression 1 > THEN
                        BEGIN
                            <statement 1 >;
                        END
                    ELSE
                        BEGIN
                            IF < expression 2 > THEN
```

```
                                                    BEGIN
                                                       < statement 1 >;
                                                    END
                                                ELSE
                                                  BEGIN
                                                     < statement 2 >;
                                                  END;
                              END;
```

sous la forme :

```
IF < expression 1 > OR < expression 2 > THEN
                                                BEGIN
                                                  < statement 1 >;
                                                END
                                            ELSE
                                                BEGIN
                                                   < statement 2 >;
                                                END;
```

4.4 Les structures répétitives

4.4.1 Définition

Une structure répétitive permet d'exécuter une séquence d'instructions un certain nombre de fois jusqu'à ce qu'une condition déterminée soit remplie ou non. La répétition se fait par l'intermédiaire de boucles et d'itérations.

Une *boucle* consiste à parcourir 'une partie d'un programme un certain nombre de fois. Une *Itération* est la répétition d'un même traitement plusieurs fois.

La même séquence d'instructions est réitérée plusieurs fois au cours d'une même exécution

On distingue les boucles à bornes définies (`POUR...FAIRE`) et les boucles à bornes non définies (`TANTQUE...FAIRE` et REPETE...JUSQU'A. Toute structure répétitive - est composée de trois éléments :

– d'une initialisation d'un compteur ;
– d'une condition ;
– d'un bloc d'instructions.

Toute modification d'un quelconque de ces trois éléments nécessite un contrôle de cohérence des deux autres.

4.4.2 Boucle à bornes définies (POUR...FAIRE)

Dans le cas d'une boucle à bornes définies, nous connaissons le nombre d'itérations à effectuer, grâce aux valeurs des bornes minimum et maximum fournies dans la définition de la boucle.

Un indice de boucle varie alors de la valeur minimum (initiale) jusqu'à la valeur maximum (finale)

SYNTAXE :

☛En pseudo-code

```
POUR variable:=valeur_initiale A valeur_finale FAIRE
                <séquence d'instruction>
```

☛En PASCAL

```
FOR variable := valeur_initiale TO valeur_finale DO
                        Bloc d'instructions;
```

Remarques

– la variable doit être de type scalaire (entier, énuméré, intervalle ou caractère) elle ne peut pas être réelle
– si `valeur_initiale > valeur_finale` le `FOR` est ignoré

Exemple 35

```
program boucle_for;
var     i:integer;
begin
   for i:=1 to 5 do
         writeln('le carré de ', i, ' est :', sqr(i));
   writeln;
```

```
   writeln('FIN. A la prochaine...');
end.
```

Il est possible d'imbriquer plusieurs boucles FOR :

```
 For X1 := C1 to C2 do
     Begin
         ...
         For X2 := D1 to D2 do
             Begin
                 ...
             End;
         ...
     End;
```

Exemple 36 *Table de multiplication*

```
PROGRAM table_multiplication;
VAR
    i, j : integer;
BEGIN
    for i := 1 to 10 do
        begin
            for j := 1 to 10 do write (i*j : 3);
            writeln;
        end;
END.
```

4.4.3 Boucles à bornes non définies

Lorsque les bornes ne sont pas connues, il existe deux autres types de boucles :

- Boucle `TANT QUE ... FAIRE ...`
- Boucle `REPETER ... JUSQU'A ...`

Syntaxe de la boucle `TANT QUE` :

☛En pseudo-code

```
TANT QUE <condition>  FAIRE
    <séquence d'instructions>
```

☛En PASCAL

```
WHILE expression DO
     Bloc d'instructions;
```

Remarques

- arrêt si expression est fausse
  ⟹pas de boucle si faux au départ
- incrémentation gérée par le programmeur lui-même
  ⟹pas d'augmentation automatique d'une variable (contrairement à la boucle FOR)
- Les variables de l'expression `expression` doivent être initialisées avant le while, pour que au premier passage `expression` puisse être évaluée.

Organigramme

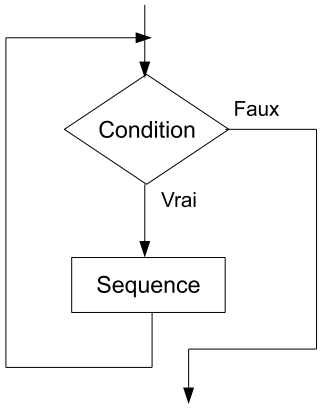


FIGURE 4.5 – Organigramme de la structure TANTQUE... FAIRE

Exemple 37

```
program boucle_while;
    var
        i:integer;
    begin
        i:=1;
        while i <= 5 do
```

```
            begin
                writeln('le carré de ', i, ' est :', sqr(i));
                i:=i+1;  { incrémentation gérée par le programmeur }
            end;
        writeln;
        writeln('FIN. A la prochaine...');
    end.
```

Syntaxe de la boucle `REPETER` :

☛En pseudo-code

```
REPETER
       <séquence d'instructions>
JUSQU'A  <condition>
```

☛En Pascal

```
REPEAT
    Bloc d'instructions
UNTIL expression;
```

Remarques

La boucle s'effectue tant que l'expression est fausse, arrêt quand l'expression est vraie. C'est le contraire de la boucle `WHILE`. Contrairement au `WHILE`, il y a au moins un passage (1 boucle), même si l'expression est vraie ; De même que pour le `WHILE`, c'est le programmeur qui gère l'incrémentation.

Organigramme

Exemple 38

```
program boucle_repeat;
var
    i:integer;
begin
      repeat
          writeln('le carré de ', i, ' est :', sqr(i));
```

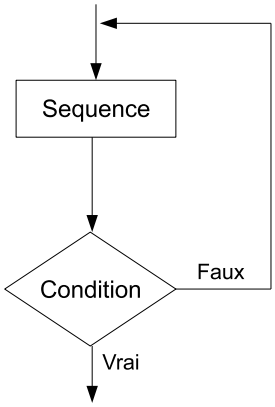


FIGURE 4.6 – Organigramme de la structure REPETER...JUSQUE

```
        i:=i+1;      { incrémentation gérée par le programmeur }
    until i>5;
    writeln;
    writeln('FIN. A la prochaine...');
end.
```

Exemple 39 *exécution répétitive d'un programme*

```
Variables :
    a,b : réel; (*opérandes *)
      p : réel; (*résultat du produit *)
      c : caractère; (*réponse de l'utilisateur *)
Début
    Répéter
        Ecrire ('Saisir le nombre a');
        Lire (a);
        Ecrire ('Saisir le nombre b');
        Lire (b);
        p:=a*b;
        Ecrire (p);
        Ecrire ('encore un calcul ? Non touche N ; Oui autre touche');
        Lire (c);
    Jusqu'à c = 'N';
```

```
Fin.
```

NB : Faire attention aux conditions initiales, aux conditions d'arrêt et à l'incrémentation sinon la boucle risque d'être infinie. Les deux boucles peuvent être choisies indifféremment. Cependant, l'une est le contraire de l'autre, au niveau de la condition d'arrêt :

```
TANTQUE condition1 FAIRE
    <Bloc d'instructions>
```

$\Longleftrightarrow$

```
REPETER
    <Bloc d'instructions>
JUSQU'A non (condition1)
```

- Tant que condition1 est vraie, faire bloc d'instructions...
- Répéter bloc d'instructions, jusqu'à ce que condition1 ne soit plus vraie

Dans ce cas, la condition d'arrêt de la boucle `TANTQUE` est l'opposée de la condition d'arrêt de la boucle `REPETER`.

Exemple 40

```
TANTQUE  (i<>10) FAIRE
    i:= i+1 {on fait varier i jusqu'à 10}
```

est équivalent à :

```
REPETER
   i:=i+1
JUSQU'A (i=10)
```

Il est toujours équivalent d'utiliser une boucle `TANTQUE` ou une boucle `REPETER`. Cependant, il existe une petite différence entre les deux boucles : Dans le cas d'une boucle `REPETER... JUSQU'A`, le bloc d'instructions est effectué au moins une fois, ce qui n'est pas forcément vrai pour une boucle `TANTQUE`. En effet, pour ce dernier type de boucle, si la condition est fausse dès le départ, le bloc d'instructions ne sera pas du tout exécuté.

En revanche, avec une boucle `REPETER ... JUSQU'A`, si la condition est fausse dès le départ, le bloc d'instructions sera quand même exécuté une fois.

Remarque :

Les boucles `REPETER` et `TANTQUE` peuvent être utilisées même si les bornes sont définies. Dans ce cas, il est bien entendu préférable d'utiliser une boucle `POUR` (vue précédemment).

4.4.4 Exemple comparatif

Nous allons à présent traiter le même exemple, avec trois boucles différentes. Il s'agit de reconstruire l'opération de multiplication, en effectuant des sommes successives. Soit à effectuer le produit des entiers naturels a et b (distincts de 0).

Données : a multiplicande, b multiplicateur

Résultat : P produit Méthode : ajouter b fois le multiplicande

Exemple 41 *Forme 1 avec* `POUR`

```
Algorithme Avec_Pour;
var a,b,i,P:entier;
Debut
    Ecrire('Donner a et b');
    Lire(a,b);
    P:=0;
    Pour i:=1 à b Faire
        P:=P+a;
    Ecrire('Le produit est',P);
Fin.
```

Exemple 42 *Forme 2 avec* `TANT QUE`

```
Algorithme Avec_TANTQUE;
var a,b,i,P:entier;
Debut
    Ecrire('Donner a et b');
    Lire(a,b);
    P:=0;i:=1;
    TantQue (i<=b) Faire
```

```
        Debut
          P:=P+a;
          i:=1+1;
        Fin;
    Ecrire('Le produit est',P);
Fin.
```

Exemple 43 *Forme 3 avec* `REPETER`

```
Algorithme Avec_REPETER;
var a,b,i,P:entier;
Debut
    Ecrire('Donner a et b');
    Lire(a,b);
    P:=0;i:=1;
    Repeter
        P:=P+a;
        i:=1+1;
    Jusque (i>b);
    Ecrire('Le produit est',P);
Fin.
```

Nous pouvons modifier les algorithmes précédents en y rajoutant des contrôles de saisie sur les valeurs de a et de b, puisqu'elles doivent être strictement positives. Le controle peut être fait avec la boucle `TANTQUE` ou la boucle `REPETER`. Pour la forme 1 nous avons :

Exemple 44

```
Algorithme Avec_REPETER_Controle_REPETER;
var a,b,i,P:entier;
Debut
    Repeter
        Ecrire('Donner a et b');
        Lire(a,b);
    Jusque (a>0) ET (b>0);
    P:=0;i:=1;
```

```
    Repeter
        P:=P+a;
        i:=1+1;
    Jusque (i>b);
    Ecrire('Le produit est',P);
Fin.
```

Exemple 45

```
Algorithme Avec_REPETER_Controle_TANTQUE;
var a,b,i,P:entier;
Debut
      Ecrire('Donner a et b');
      Lire(a,b);
      TantQue (a<=0) OU (b<=0) Faire
            Debut
                Ecrire('Donner a et b');
                Lire(a,b);
            Fin
      P:=0;i:=1;
      Repeter
         P:=P+a;
         i:=1+1;
      Jusque (i>b);
      Ecrire('Le produit est',P);
Fin.
```

Chapitre 5

Les tableaux et les chaînes de caractères

5.1 Tableaux à 1 dimension

5.1.1 Définition

Un **tableau** est une collection ordonnée de variables (appelées composantes du tableau) ayant toutes le même type. On accède à chacune de ces variables individuellement à l'aide d'un indice.

indice=valeur ordinale, c'est à dire ayant un domaine de valeurs finies et ordonnées. L'indice doit être de type scalaire. Chaque composante du tableau est référencée au moyen d'un indice qui indique la position relative de la composante dans la suite et qui permet donc d'accéder à la composante.

Un tableau T est donc défini par le ***type des indices*** et par le ***type des composantes***. Les tableaux à *une dimension*, c'est-à-dire à un type d'indices, appelés encore en termes mathématiques ***vecteurs***, sont composés d'un ensemble homogène d'éléments de ***même type***.

5.1.2 Déclaration d'un tableau

La déclaration d'un tableau s'effectue en donnant :
- son nom (identificateur de la variable tableau)
- le domaine de variation de l'indice délimité par une borne inférieure correspondant

à l'indice minimal et la borne supérieure correspondant à l'indice maximal.

– le type de ses composantes

☛Syntaxe de définition :

```
Var <Nom_tab> : Tableau[<indice min>..<indice_max>] de <type des composants>;
```

Exemple 46

```
var tab_entier : Tableau[1..10] d'entiers;
```

☛En PASCAL, on a :

```
Var <nom_tab> : ARRAY [<indice min>..<indice max>] OF <type des composants> ;
```

Exemple 47

```
var  tab_entier : ARRAY [1..10] OF integer;
```

La structure `ARRAY` n'est pas une structure "dynamique" mais "statique", c'est-à-dire une structure qui ne change pas de taille au cours de l'exécution du programme. Par conséquent, le nombre d'éléments d'un tableau doit être fixé à priori de manière définitive lors de sa définition.

Soit par exemple l'extrait de programme suivant :

Exemple 48

```
CONST MaxElements = 100;
TYPE Dim = 1..MaxElements;
           Alphabet = 'A'..'Z';
. . .
VAR Boole: ARRAY [ Dim ] OF Boolean;
         Frequency: ARRAY [ Alphabet ] OF Integer
```

Dans cet exemple, le tableau `Boole` est un vecteur de `100` composantes qui ne peuvent prendre que les valeurs `True` et `False`. Le tableau `Frequency` représente un vecteur de 26 composantes, chacune étant de type entier et indicée par une lettre majuscule.

Souvent, on représente la structure d'un tableau moyennant la définition d'un type en écrivant :

```
TYPE <identificateur de type> = ARRAY[<type index>] OF <type composant>;
```

Et ensuite en déclarera les variables de ce type tableau en écrivant :

```
VAR < liste variable >: < identificateur de type > ;
```

Soit pour l'exemple ci-dessous :

Exemple 49

```
CONST MaxElements = 100;
TYPE Dim = 1..MaxElements;
          Alphabet = 'A' . . 'Z';
          TBoole = ARRAY [ Dim ] OF Boolean;
          TFrequency = ARRAY [ Alphabet ] OF Integer;
.......
VAR Boole: TBoole;
         Frequency: TFrequency;
```

Si les variables `I` et `J` prennent respectivement leurs valeurs dans les intervalles `Dim` et `Alphabet`, alors `Boole[ I ]` et `Frequency[ J ]` désignent respectivement les composantes situées à la I-ième place dans le tableau `Boole` et à la J-ième place dans le tableau `Frequency`. Donc, pour accéder à une composante d'un vecteur, il suffit de préciser son indice.

Dans la mémoire centrale, les éléments d'un tableau sont stockés de façon linéaire, dans des zones contiguës. Une variable T de type tableau à une dimension peut être représenté comme suit :

1	2	3	4	5	6	7	8
10	0	1	4	7	2	6	25

FIGURE 5.1 – Tableau à une dimension (vecteur)

Remarques

Quand on ne connaît pas à l'avance le nombre d'éléments exactement, il faut majorer ce nombre, quitte à n'utiliser qu'une partie du tableau. Il est préférable de définir un TYPE tableau réutilisable, plutôt que de déclarer à chaque fois, en `VAR`, le tableau en entier. De même, l'utilisation judicieuse des constantes permet de modifier aisément le programme et limite les sources d'erreurs. Il est interdit de mettre une variable dans l'intervalle de définition du tableau.

5.2 Tableaux à deux dimensions

5.2.1 Définition

Un tableau à deux dimensions, c'est-à-dire à deux types d'indices, est un tableau dont le type des composantes est un type tableau de dimension un, donc, un vecteur. En termes mathématiques, un tableau à deux dimensions est appelé une ***matrice***.

La définition d'un tableau à deux dimensions peut s'effectuer de la manière suivante :

```
TYPE <identificateur> = ARRAY[1..M, 1..N] OF < type-composantes >;
```

Où `< type-composantes >` peut être un type quelconque, sauf le type tableau.

Ainsi par exemple, en termes mathématiques, une matrice réelle Mat d'ordre $N \times M$ est représentée par un tableau de M vecteurs composés chacun de N nombres réels.

Exemple 50
```
CONST MaxLine = 10; {nombre maximal de lignes}
      MaxColumn = 20; {nombre maximal de colonnes}
TYPE
      TLine = 0..MaxLine;
      TColumn = 0..MaxColumn;
      Matrix = ARRAY [ TLine, TColumn ] OF Real;
. . .
VAR Mat: Matrix;
```

Si les variables `L` et `C` prennent respectivement leurs valeurs dans les intervalles `TLine` et `TColumn`, alors `Mat[L,C]` ou bien `Mat[L][C]` désigne la composante de la matrice située à la `L`-ième place dans la `C`-ième colonne, respectivement à la `C`-ième place dans la `L`-ième ligne (voir figure). Par convention, nous allons indiquer souvent par `L` l'indice de parcours des lignes et par `C` l'indice de parcours des colonnes d'une matrice. Un tel tableau sera représenté sous la forme indiquée par la figure 5.2 :

Donc, pour accéder à une composante d'une matrice, il est nécessaire de préciser deux indices, à savoir d'abord l'indice de ligne, puis l'indice de colonne.

Remarques :

– Dimension du tableau ci-dessus : 2 (matrice)

– Nombre de cases disponibles : `MaxLine x MaxColumn`

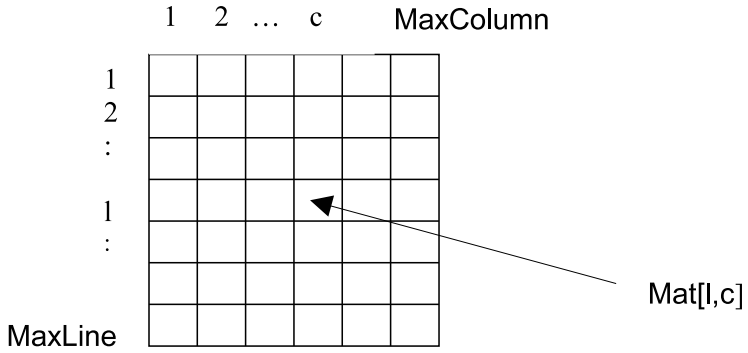


FIGURE 5.2 – Tableau à deux dimensions (matrice)

- le tableau `Mat` est bidimensionnel $\Longrightarrow$ tous les $\texttt{Mat}_{ij}$ sont des variables (i = 1 ... M, j = 1 ... N) de même type
- Accès au contenu d'une case : $\texttt{Mat}_{ij}$

Exemple 51 *Déclarations de tableaux :*

1. *Cas le plus classique*

```
  type LISTE = array [1..100] of real ;
TABLEAU = array [1..10,1..20] of integer ;
CHAINE = array [1..80] of char ;
```

2. *Cas plus complexes*

```
  type POINT = array [1..50,1..50,1..50] of real ;
MATRICE = array [1..M,1..N] of real ;
SYMPTOME = (FIEVRE, DELIRE, NAUSEE) ;
MALADE = array [symptome] of boolean ;
CODE = 0..99 ;
CODAGE = array [1..N_VILLE] of code ;
```

3. *Définition récursive*

```
type CARTE = array [1..M] of array [1..N] of real ;
var  LIEU : carte ;
          --> accès à un élément par   LIEU [I] [J]
```

5.3 Tableaux multidimensionnels

Il est possible de généraliser la définition des tableaux à plusieurs dimensions, c'est-à-dire à plusieurs types d'indices, par simple extension de la notation :

```
TYPE TableauMultidim = ARRAY[<type index>,<type index>,...,<type index>]
                                                OF < type composantes >;
```

où `<type composantes>` peut être un type quelconque, sauf le type tableau. En règle générale, il est peu utile d'utiliser des tableaux à plus de trois dimensions, puisque (1) la clarté diminue considérablement dans ce cas et (2) l'encombrement de l'espace-mémoire devient trop important.

5.4 Manipulations élémentaires de tableaux

Les langages de programmation usuels n'offrent en général qu'un ensemble très réduit de primitives de base définies sur le type tableau. Ainsi est-il presque toujours nécessaire de manipuler les tableaux en décrivant les fonctions que l'on veut réaliser en termes de composantes. Les exemples suivants montrent comment il est possible de réaliser des opérations qui concernent toutes ou une partie des composantes d'un ou de plusieurs tableaux.

5.4.1 Création et affichage d'un tableau

Créer un tableau équivaut à attribuer une valeur à chacune de ses composantes. La création peut se faire soit par une affectation, soit par une saisie des valeurs.

Exemple 52 *Affectation*

```
T[1]:=2; T[2]:=10;...T[20]:=8;
```

Saisie

```
For  i:=1 To 20 Do readln(t[i]);
```

L'Affichage consiste parcourir le tableau à l'aide d'une boucle et à afficher ses différentes composantes

Exemple 53 *Le programme suivant permet de "remplir" un tableau à l'aide d'une boucle Repeat-Until.*

```
Program Mon_tableau;
Const
    Taille_max=10;
Type
    TAB=array[1..Taille_max] of integer;
Var
    Tableau:TAB;
    indice: integer;
Begin
    for indice:=1 to Taille_max do
        Tableau[indice]:=0;
    indice:=1;
    Repeat
        write('entrez le N° ',indice,':');
        readln(Tableau[indice]);
        indice:=indice+1;
    until indice>Taille_max;
End.
```

5.4.2 Maximum et Minimum d'un tableau

La recherche du maximum (minimum) consiste à mettre dans la variable `max` (`min`) le premier élément du tableau `T`, de parcourir ensuite le reste du tableau et de mettre à jour cette variable `max` (`min`) si l'élément en cours d'examen est supérieur (inférieur) au `max` (`min`) courant : pour le maximum on a :

Exemple 54

```
max := T[1] ;
for i :=2 to Taille_max do
        if T[i] > max then
                          max := T[i];
writeln('Le maximum de T:', max) ;
```

et pour le minimum on a :

Exemple 55

```
min := T[1] ;
for i :=2 to Taille_max do
            if T[i] < min then
                              min := T[i];
writeln('Le minimum de T:', min) ;
```

5.4.3 Recherche séquentielle d'un élément dans un tableau

La recherche séquentielle d'un élément `a` dans un vecteur `T` consiste à parcourir ce dernier et de trouver un indice `i` tel que `T[i]=a`.

Exemple 56

```
i := 1;
while (T[i]<>a) and (i<=n) do
i:=i+1;
if (i>n) then
write(a ,'n''appartient pas à T')
else
write(a,' appartient à T');
```

On peut aussi utiliser une variable booléenne trouve qui va contenir le résultat de la recherche :

Exemple 57

```
i := 1; trouve:=false
while  (i<=n)  and (trouve=false) do
    begin
          trouve:=(t[i]=a);
          i:=i+1;
    end;
if trouve then
     write(a,' appartient à T')
        else
     write(a, ' n''appartient à T');
```

5.4.4 Recherche dichotomique d'un élément dans un tableau ordonné

La bonne idée est de comparer l'élément recherché avec l'élément situé au milieu du tableau :

- si l'élément recherché est plus petit, on continue la recherche dans la première moitié du tableau ;
- s'il est plus grand, on continue dans la seconde moitié du tableau ;
- On s'arrête si l'élément recherché a été trouvé ou si on ne peut plus diviser le tableau.

Dans ce qui suit :

- `x` représente l'élément que l'on veut chercher
- `gauche, droite` : délimitent la zone du tableau à l'intérieur de laquelle on veut rechercher `x`

Exemple 58

```
.........................
VARIABLES  gauche,droite: entier;
                        milieu : entier
DEBUT
   (*saisie du tableau T de n éléments et de l'élément x à rechercher*)
   ....................................
```

```
   gauche:=1; droite:=n; trouve :=faux
   TANTQUE (gauche<=droite) ET non Trouve FAIRE
          DEBUT
             milieu:=(gauche +droite ) DIV 2;
             SI (x = t[milieu])  ALORS
                                   trouve :=vrai
                                 SINON
                                   SI (x  < t[milieu]) ALORS
                                                         droite:=milieu-1;
                                                       SINON
                                                         gauche:=milieu+1;
          FIN
   SI trouve ALORS
               ecrire(x,' appartient à T')
             SINON
               ecrire(x, ' n''appartient à T');

FIN.
```

5.4.5 Recherche d'un élément dans une matrice

Nous nous proposons d'écrire un programme qui détermine si l'un au moins des éléments de la matrice `Mat` d'ordre $N \times M$ est égal à a (élément recherché). Voici un extrait de ce programme (la matrice `Mat` a été définie dans le paragraphe 5.2.1) :

Exemple 59 .....

```
trouve := False;
l := 1;
while ( l<=n ) and not trouve do
        begin
            c := 1;
            while ( c<=m ) and not trouve do
                    begin
                        trouve := ( Mat[ l, c ] = a );
```

```
                    c:=c+1;
                end; {-- WHILE, c>m or trouve}
            l:=l+1;
        end; {-- WHILE, l>N or trouve}

if trouve then
            write(a,' appartient à Mat')
          else
            write(a, ' n''appartient à Mat');
```

5.4.6 Initialisation d'une matrice unité

Nous désirons réaliser un programme permettant l'initialisation d'une matrice unité de dimension 10. Il s'agit donc d'une matrice à 10 colonnes et 10 lignes, ne comportant que des 0, sauf sur sa diagonale ou il n'y a que des 1.

Exemple 60

```
Program SOMME_MATRICE ;
const
    L_MAX = 10 ;
    C_MAX = 10 ;
   I, J : Integer;
Type
    TAB = array [1 . . l_max, 1 . . c_max]  of  integer ;
Var
    MAT : tab ;
Begin
    for  I := 1 to  l_max  do
        begin
            for  J := 1  to  c_max  do
                begin
                    if  I = J then
                            MAT [I, J]  := 1
                        else
                            MAT [I, J]  := 0 ;
```

```
                    write (MAT [I, J]) ;
                end ;
            writeln ;
        end ;
    End.
```

5.4.7 Fusion de deux tableaux ordonnés

Soient deux tableaux triés par ordre croissant `T1` et `T2` possédant respectivement `taille1` éléments et `taille2` éléments, on veut fusionner les tableaux `T1` et `T2` en un troisième tableau `T3` de `taille` (= `taille1+taille2`) éléments triés par ordre croissant. Pour ce travail nous allons choisir trois compteurs `i`, `j` et `k` pour décrire respectivement `T1`, `T2` et `T3` où `T3` est le tableau qui recevra le tableau trié résultant de la fusion. Un extrait de l'algorithme sera le suivant :

Exemple 61

```
.................................
Variables i,j,k :entier;
                 T1,T2,T3 : TAB.;
DEBUT
  (*saisie de T1 et de T2*)
  ..........................
  i=1;j=1,k=0;
  TANTQUE ((i<=taille1) ET(j<=taille2)) FAIRE
       SI (T1[i]<T2[j]) ALORS   (* copier les éléments du T1 tant qu'ils*)
                          DEBUT   (*sont inférieurs a l'élément courant de T2*)
                            k:=k+1;
                            T3[k]:=T1[i];
                            i:=i+1;
                          FIN
                        SINON    (* et inversement*)
                          DEBUT
                            k:=k+1;
                            T3[k]:=T2[j];
                            j:=j+1;
```

```
                        FIN;
    SI (i>taille1) ALORS   (*recopier des éléments  de T2, s'il en reste*)
                    TANTQUE (j<=taille2) FAIRE
                            DEBUT
                              k:=k+1;
                              T3[k]:=T2[j];
                              j:=j+1;
                             FIN
                   SINON       (* idem pour T1*)
                     TANTQUE (i<=taille1) FAIRE
                             DEBUT
                               k:=k+1;
                               T3[k]:=T1[i];
                               i:=i+1;
                             FIN;
  Taille:=k;
FIN;
```

5.5 Les chaînes de caractères

5.5.1 Définition

Une chaîne de caractères est une suite de caractères regroupés dans une même variable. Elle correspond à un tableau de 255 caractères au maximum.

`Type string = array[1..255] of char`

On utilise le Type `string`, qui n'est pas un type standard du pascal (certains compilateurs Pascal ne l'acceptent pas). Ce type permet de manipuler des chaînes de longueur variable.

Déclaration :

```
var  S  : string;
     S2 : string[20];
```

Il est possible, comme dans tout tableau, d'accéder à un caractère particulier de la chaîne `S`, en écrivant simplement : `S[i]`. On peut aussi manipuler la chaîne de manière

globale, sans passer élément par élément. Ceci est très utile pour des affectations ou des tests

Exemple 62

```
S := 'Bonjour';
S[4] := 's';
S[6] :='i'
```

$\Longrightarrow$ *A présent,* `S` *vaut* `'Bonsoir'`

```
S := 'ok';
```

$\Longrightarrow$ *A présent,* `S` *vaut* `'ok'` *On constate que la taille de* `S` *est variable (7 caractères au départ, 2 caractères ensuite)*

5.5.2 Opérateurs et fonctions

Il est possible de comparer des chaînes de caractères, on utilise alors les opérateurs : `=, <, >, <=, >=, <>`

Dans ce cas, l'ordre utilisé est l'ordre lexicographique (utilisation du code ASCII)

Exemple 63 *Soit* `b` *un booléen ;* `b` *est-il vrai ou faux ?*

```
b := 'A la vanille' < 'Zut'; { vrai }
b := 'bijou' < 'bidon'; { faux, c'est > car 'j' > 'd' }
b := 'Bonjour' = 'bonjour'; { faux, c'est < car 'B' < 'b' }
b := ' zim boum' > 'attends !'; { faux, c'est < car ' ' < 'a' }
```

Exemple 64

```
var
S: String[4];
begin
S := '';
Write(S) ; { rien n'est affiché : la chaîne est vide }
S := 'toto' ;
S[1] := 'm' ;
Write(S) ; { la chaîne de caractère contient « moto » }
end.
```

☛CONCATÉNATION :

`s:=concat(s1, s2, s3 ...);` (ou parfois `s:= s1 + s2 + s3...`)

Exemple 65

```
s1:='bon';
s2:='jour';
s3 := s1 + s2 ;
```

Nous obtenons alors `s3` *valant* `'bonjour'`

☛LONGUEUR :

`length (str) -> entier`

Exemple 66

```
var
S: String;
begin
Readln (S);
Writeln('"', S,'"');
Writeln('longueur de la chaîne = ', length(S));
end.
```

Exemple 67

```
s1:='salut';
s2:='bonjour';
```

Nous obtenons alors `length(s1)` *valant* `5` *et* `length(s2)` *valant* `7`

☛FONCTION POS

`pos(souschaîne, chaîne)`

$\Longrightarrow$position de la sous chaîne dans la chaîne.

Exemple 68

```
var S: String;
begin
S := ' 123.5';
{ Convertit les espaces en zéros }
while Pos(' ', S) > 0 do
S[Pos(' ', S)] := '0';
end.
```

☛FONCTION COPY :

`copy (source, index, compteur)`

$\Longrightarrow$ `string` avec "`compteur`" caractères à partir de l'index.

Exemple 69

```
s:=copy('bonjour monsieur', 4, 4);
```

Nous obtenons alors `s` *valant* `'jour'`

☛PROCEDURE DELETE :

`delete(chaine, debut, nb_car)`

$\Longrightarrow$ supprime le nombre de caractères spécifié par `nb_car` à partir de la position indiquée par `debut`.

☛PROCEDURE INSERT :

`insert(chaine1, chaine2, position)`

$\Longrightarrow$ insère `chaine1` dans `chaine2` à partir de la position indiquée par position.

Exemple 70

```
s:=insert('madame ', 'au revoir  Fall' ,11)
```

Nous obtenons alors `s` *valant* `'au revoir madame Fall'`

☛FONCTION ORD

`ORD(caractère)`

$\Longrightarrow$ entier (code ASCII).

Exemple 71 `ORD('A')` *vaut* `65` *et* `ORD('a')` *vaut* `97`

☛FONCTION CHR

`CHR(entier)`

$\Longrightarrow$ caractère ayant ce code ASCII.

Exemple 72 `CHR(65)` *vaut* `'A'` *et* `CHR(97)` *vaut* `'a'`

Chapitre 6

Les sous programmes

6.1 La programmation modulaire

6.1.1 Définition

Si vous avez un gros programme à mettre au point, et que certaines parties sont semblables ou d'autres très complexes, alors il faut absolument structurer son programme pour ne pas risquer d'y passer trop de temps, lors de modifications ultérieures, ce qui en facilitera la maintenance.
Conclusion : il faut adopter la stratégie *"diviser pour régner"* et chercher à utiliser au maximum "l'existant".

La programmation modulaire consiste à décomposer le problème initial en sous-problèmes de moindre complexité. Cette décomposition se poursuit jusqu'à l'obtention de sous-problèmes plus compréhensibles et plus simples à programmer. On parle d'analyse descendante (ie, on part du niveau le plus complexe et on aboutit à un niveau le plus simple possible du problème).

Par ailleurs on est souvent amené à effectuer plusieurs fois un même ensemble de traitements dans un programme. Cet ensemble de traitements constitue un sous-programme1 (ou module). Cependant pour éviter cette réécriture, on attribue un nom à ce dernier et à chaque fois que son utilisation est souhaitée, on lui fait appel par l'intermédiaire de ce nom.

Avantages :
- la lisibilité
- la structuration

– possibilité d'utiliser un identificateur significatif

Exemple 73 *au lieu d'écrire :*

```
begin
    bloc1
    bloc2
    ...
    bloc1
    ...
    bloc2
    bloc1
    ...
end.
```

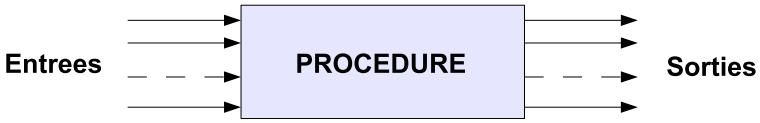
où `bloc1` *et* `bloc 2` *peuvent être des blocs de plusieurs dizaines de lignes de code, on préfèrera écrire :*

```
procedure P1
begin
    bloc1
end;
procedure P2
begin
    bloc2
end;
begin
    P1;
    P2;
    P1;
    ....
    P2;
    P1;
    ...
end.
```

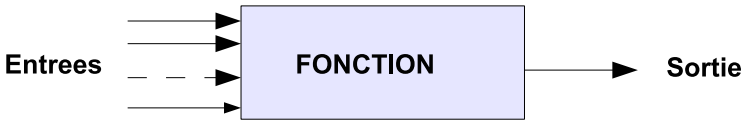
6.1.2 Les différents types de modules

En Pascal, on distingue deux types de modules :

Les **procédures** : Il s'agit de sous-programmes *nommés* qui représentent une ou plusieurs actions, et qui peuvent calculer et ***retourner une ou plusieurs valeurs***. Une procédure `P` doit être définie une seule fois dans un algorithme `A` et peut être appelée plusieurs fois dans `A` ou par une autre procédure de `A`.



Les Fonctions : Il s'agit de sous programmes *nommés* et *typés* qui calculent et ***retournent une et une seule valeur***. Le type de cette valeur est celui de la fonction. De même une fonction `F` doit être définie une seule fois dans l'algorithme `A` et peut être appelée plusieurs fois par `A` et par toutes les autres procédures et fonctions de `A`.



6.2 Les Procédures

6.3 Syntaxe et déclarations

Si l'on se réfère donc au chapitre sur les déclarations, les déclarations s'effectuent dans l'ordre suivant :

```
EN-TETE
 DECLARATIONS
1. CONSTANTES
2. TYPES
3. VARIABLES
```

`4. FONCTIONS / PROCEDURES`

`BLOC D'INSTRUCTIONS EXECUTABLES`

☛Syntaxe de déclaration La définition d'une procédure comprend trois parties :

Partie 1 : *L'entête* :

```
Procedure Nom_proc[(Identif1[, ...]:Type1[;[var] Identif2[, ...]:Type2])];
```

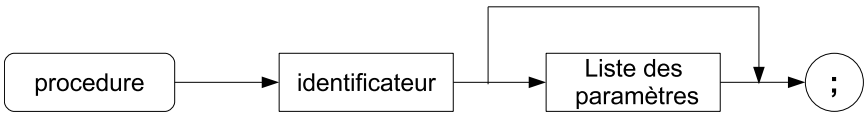


Figure 6.1 – Diagramme de description syntaxique de l'entête d'une procédure

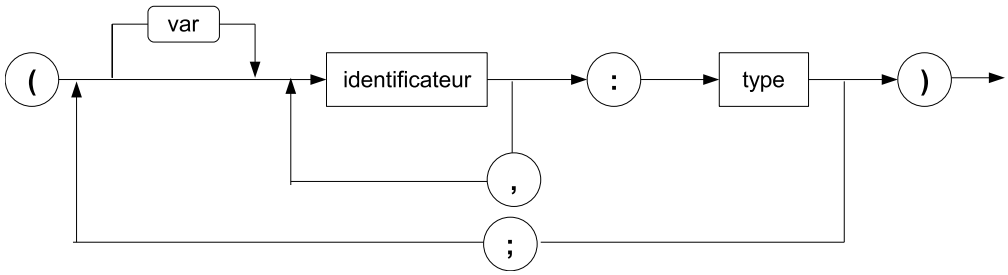


Figure 6.2 – Diagramme de description syntaxique de la liste de paramètres

Partie 2 : *Déclaration des variables locales*

```
Nom_variable_i : type_variable_i;
```

Partie 3 : *Corps de la procédure*

```
Begin
     <instruction1>;
     <instruction2>;
     ...
     <instructionN>
End;
```

➥ Les identificateurs qui suivent le nom de la procédure constituent l'ensemble des *paramètres formels*.

➥ Le mot var (optionnel) n'est utilisé que pour les paramètres de sortie et d'entrée/sortie.

Une fois la procédure déclarée, elle peut être utilisée dans le programme principal par un "appel" à cette procédure, à partir du programme principal ou à partir d'une autre procédure.

☛APPEL D'UNE PROCÉDURE

```
Nom_procedure( expression { , expression } ) ;
```

6.3.1 Exemples

Exemple 74 *Ecrire un programme qui calcule successivement la moyenne de la taille de trois individus ainsi que la moyenne de leur poids et celle de leur âge.*

Version sans emploi de procédure (à rejeter) :

```
Program Moyenne;
    Var  A, B, C, M : real;
    Begin
        writeln ('Entrez la taille des 3 individus : ');
        readln (A, B, C);
        M := (A+B+C) / 3;
        writeln ('La moyenne est : ', M:8:2);
        writeln('Entrez le poids des 3 individus : ');
        readln (A, B, C);
        M := (A+B+C) / 3;
        writeln ('La moyenne est : ', M:8:2);
        writeln('Entrez l''âge des 3 individus : ');
        readln (A, B, C);
        M := (A+B+C) / 3;
        writeln ('La moyenne est : ', M:8:2);
   End;
```

Cette version n'est pas idéale, car nous constatons qu'une même partie du programme est écrite trois fois. Il est préférable de regrouper ces instructions dans une procédure. La nouvelle version, avec emploi d'une procédure, est la suivante :

```
Program  MOYENNE_BIS;
Var    A, B, C, M : real;
Procedure  CALCUL;
Begin
      readln (A, B, C);
      M := (A + B + C) / 3;
      writeln ('La moyenne est ', M:8:2);
End;
Begin  { Début du programme principal }
      writeln ('Entrer les tailles de 3 individus');
      CALCUL;
      writeln ('Entrer les poids de 3 individus');
      CALCUL;
      writeln('Entrez l''âge des 3 individus : ');
     CALCUL;
End.
```

Exemple 75 *Pour calculer la somme de deux matrices A et B, il faut lire d'abord ces deux matrices. Ainsi au lieu d'écrire deux fois le sous-programme de lecture d'une matrice, on écrit plutôt une procédure et on l'utilise pour saisir A et B.*

```
Program  SOMME_MATRICIELLE;
Const nmax=50;
Type MATRICE = ARRAY[1..nmax,1..nmax] of integer;
Var    A, B, S : MATRICE;
                  n : integer;
{Procédure de saisie d'une matrice }
Procedure  SAISIE(Var M : MATRICE; dim : integer);
Var i, j : integer;
Begin
      For i:=1 To dim  Do
          For j:=1 To dim  Do
                Begin
                     write('Donnez l''élément M[',i,j,'] ');
```

```
                    readln(M[i,j]);
                End;
End;
{Procédure de calcul de la somme de deux matrices }
Procedure  SOMME(M1  : MATRICE;M2 : MATRICE; var C : MATRICE , dim : integer);
Var i, j : integer;
Begin
      For i:=1 To dim  Do
          For j:=1 To dim  Do
                 C[i,j]:=M1[i,j]+M2[i,j];
End;

{Procédure d'affichage d'une matrice }
Procedure  AFFICHAGE(M : MATRICE; dim : integer);
Var i, j : integer;
Begin
      For i:=1 To dim  Do
         Begin
             For j:=1 To dim  Do
                     Write(M[i,j],' ');
             writeln;
         End;
End;

Begin  { Début du programme principal }
      write ('Entrer la dimension des deux matrices ');
      readln(n);
      writeln (' Lecture de la première matrice ');
      SAISIE(A,n);
      writeln (' Lecture de la seconde matrice ');
      SAISIE(B,n);
      SOMME(A,B,S,n); {Calcul de la somme de A et B}
      AFFICHAGE(S,n);{affichage du résultat}
      readln;
```

```
End.
```

6.4 Fonctions

6.4.1 Utilité des fonctions

D'une manière générale :

- Les procédures ont pour effet de modifier l'état des variables d'un programme. Elles constituent une partie du programme qui se suffit à elle-même. Une procédure peut ne pas retourner des résultats au (sous-) programme appelant. En effet, il peut arriver qu'une procédure exécute un certain nombre d'instructions et que l'on n'ait pas besoin de la valeur résultat (cas de la fonction CALCUL dans l'exemple précédent).
- Les fonctions permettent de définir le calcul d'une valeur d'un type donné et de rendre un résultat au (sous-) programme. Le type de la valeur retournée est très souvent un type de données simple.

On distingue deux catégories de fonction en Turbo-Pascal :

a) **Les fonctions implicites**.

Parmi les fonctions implicites, encore appelées primitives de base du système de programmation on distingue :

- les fonctions usuelles représentées par les opérateurs et,
- les fonctions standard prédéfinies telles que `Sqr`, `Sqrt`, `Sin`, etc. mises à la disposition de l'utilisateur par le système de programmation utilisé.

b) **Les fonctions explicites**.

les fonctions explicites regroupent toutes les autres fonctions calculables à définir par l'utilisateur lui-même. En générale, ces fonctions sont de la forme :

`F(X1,X2,...,XN)`

où `F` est le nom de la fonction et `X1,X2,...,XN` les arguments

6.4.2 Définition

Une fonction est un bloc d'instructions qui doit retourner une valeur de type simple au point d'appel. En général, le rôle d'une fonction est le même que celui d'une procédure. Il y a cependant quelques différences :

- Arguments de la fonction : paramètres typés

– Un ensemble d'arrivée typé
– Renvoi d'une valeur unique

Type de la fonction :

– scalaire, réel, string
– intervalle
– pointeur

Le type doit être précisé par un identificateur, et non par une déclaration explicite.

On déclare les fonctions au même niveau que les procédures (après VAR, TYPE, CONST)

☛Syntaxe de déclaration :

La définition d'une fonction comprend aussi trois parties :

Partie 1 : *L'entête*

```
Function Nom_fonc[(Ident1[,...]:Type1[;[var] Ident2[,...]:Type2])]:Typfonc;
```

Partie 2 : Déclaration des variables locales

```
Nom_variable_i : type_variable_i;
```

Partie 3 : *Corps de la fonction.*

```
Begin
       <instruction1>;
       <instruction2>;
       ...
       nom_fonction:=<expression> {valeur retournée}
       ...
       <instructionN>
End;
```

➥ L'affectation `nom_fonction:=<expression>` ne peut se faire qu'une seule fois par appel à la fonction.

➥ La principale différence avec une procédure est que la fonction renvoie toujours une valeur (un résultat) comme après son appel, d'où la nécessité de déclarer un type pour cette fonction (Toute fonction « vaut »quelque chose.

➥ L'identificateur de fonction est donc considéré comme une expression, c'est-à-dire comme une valeur (calculée par la fonction elle-même).

Ainsi, le simple fait d'écrire l'identificateur de fonction avec ses paramètres a pour conséquence d'appeler la fonction, d'effectuer le traitement contenu dans cette fonction, puis de placer le résultat dans l'identificateur de fonction.

Exemple 76

```
Ecrire un programme qui calcule la puissance d'un nombre réel.
program  EXPOSANT ;
var
      N : integer;
      Y, T : real;
function  PUISSANCE (X : real; N : integer) : real;
    begin
         Y := 1;
         if  N > 0  then
                      for  I := 1 to  N  do
                           Y := Y * X
                   else
                      for  I := N  to  -1  do
                           Y := Y / X;
         PUISSANCE := Y; { affectation de la valeur à l'identificateur}
    end;   { Fin du code de la fonction }

    { Debut du programme principal}
begin
      readln (T, N);     { appel de la fonction dans le writeln }
      writeln('La valeur de ', T,' puissance ', N,' est : ', PUISSANCE(T,N));
end.     { Fin du programme }
```

6.5 Différence entre procédure et fonction

- ➥ Une procédure peut être considérée comme une instruction composée que l'utilisateur aurait créée lui-même. On peut la considérer comme un **petit programme**.
- ➥ Une fonction quant à elle renvoie **toujours une « valeur »**(comprendre le mot

valeur comme objet de type simple, comme par exemple un entier ou un caractère). Elle nécessite donc un type (entier, caractère, booléen, réel, etc...).

NB : Il est interdit d'utiliser l'identificateur d'une fonction comme nom de variable en dehors du bloc correspondant à sa déclaration.

Exemple 77 *Examinons le programme suivant*

```
program exemple;
var x,y : integer;
function double (z : integer) : integer;
    begin
        double := z*2;
    end;
begin
        Readln(x);
        y := double(x);
        double := 8; {erreur à cette ligne lors de la compilation}
 end.
```

Ce programme ne pourra pas fonctionner, car on lui demande d'affecter la valeur `8` *à la fonction* `double`*. Or, Il est interdit d'utiliser l'identificateur d'une fonction comme nom de variable en dehors du bloc correspondant à sa déclaration, d'où l'ERREUR.*

```
        PROGRAM difference;
VAR a, b, c, d : real;
PROCEDURE Produit (x, y : real; | FUNCTION Produit (x, y : real) : real;
                 var z : real); | VAR res : real;
BEGIN                           | BEGIN
   z := x * y;                  |    res := x * y;
END;                            |    Produit := res;
                                | END;
                        BEGIN
                             write ('a b ? '); readln (a, b);
Produit (a, b, c);              | c := Produit (a, b);
```

```
Produit (a-1, b+1, d);          | d := Produit (a-1, b+1);
                              writeln ('c = ', c, ' d = ', d);
                        END.
```

6.6 Variables globales et variables locales

6.6.1 Déclarations

Dans certains exemples présentés dans ce cours, nous avons effectué toutes les déclarations de variables en tête de programme. Or il est également possible (comme nous l'avons fait dans d'autres circonstances) de déclarer des variables, **au sein d'un bloc fonction ou procédure**. Dans ce cas, ces déclarations se font dans le même ordre : constantes, types, variables (et même procédure(s) et/ou fonction(s)). Lorsque la déclaration est effectuée en en-tête du programme, on parle de **variable globale**. Dans le cas contraire, où la déclaration est faite à l'intérieur même de la procédure ou de la fonction, on parle de **variable locale**.

Exemple 78 *Reprenons le programme qui calcule la puissance d'un nombre réel. Nous désirons de plus calculer la puissance T^N avec des exposants (N) variant de $-N$ à $+N$, N étant le nombre entier entré par l'utilisateur. Par exemple, si on tape 3 pour T et 2 pour N, le programme calculera les valeurs 3^{-2}, 3^{-1}, 3^0, 3^1, et 3^2 Cette fois-ci, nous utiliserons des variables locales, ce qui est bien meilleur*

```
Program  EXPOSANT ;
var   I, N : integer;
         T : real;
Function  PUISSANCE (X : real; N : integer) : real;
var               { utilisation de variables locales }
      Y : real;
      I : integer;
Begin    { Code de la fonction }
     Y := 1;
```

```
     if  N > 0  then
                  for  I := 1 to  N  do
                            Y := Y * X
                else
                  for  I := -1  downto  N  do
                            Y := Y / X;
     PUISSANCE := Y;
end;          { Fin du code de la fonction }
{ Debut du programme principal}
begin
      readln (T, N);
      for I := -N to N do
            writeln (PUISSANCE (T, I));  { appel de la fonction }
end.
```

6.6.2 Portée des variables

- **Variable locale** : portée limitée à la procédure ou à la fonction en question
- **Variable globale** : active dans le bloc principal du programme, mais également dans toutes les procédures et fonctions du programme.

  Chaque variable située dans un bloc de niveau supérieur est active dans toutes les procédures de niveau inférieur.

Dans le cas où un même nom est utilisé pour une variable globale et pour une variable locale, cette dernière à la priorité dans la procédure ou dans la fonction où elle est déclarée (niveau local).

ATTENTION : Dans le cas où un même nom est utilisé pour une variable locale et pour une variable globale, le programme considère ces variables comme **deux variables différentes** (mais ayant le même nom).

Exemple 79

```
program PORTEE;
    var
        A, X : real;
    procedure INTERNE;
```

```
var
    B, X : real;
begin
  B := A / 2;
  writeln (B);
  writeln (A + X);
end;
begin { programme principal }
  readln (A, X);
  writeln (A / 2);
  writeln(A + X);
  INTERNE;
end.
```

Question : que se passe-t-il à l'exécution ?

Réponse : le programme ne fonctionnera pas comme on l'espérait, car `X` est considérée comme une variable locale dans `INTERNE`, et cette variable locale n'a pas été initialisée ! Dans la procédure `INTERNE`, `X` n'aura donc aucune valeur (il ne faut surtout pas la confondre avec le `X` du programme principal, qui est une variable globale).

Pour que le programme fonctionne correctement, il faudra alors ne pas déclarer `X` comme locale dans `INTERNE`.

6.6.3 Locale ou globale ?

Il vaut toujours mieux privilégier les variables locales aux variables globales.

- *Inconvénients d'une utilisation systématique de variables globales* :
  - manque de lisibilité
  - présence de trop nombreuses variables au même niveau
  - ne facilite pas la récursivité
  - risque d'effets de bord si la procédure modifie les variables globales
- *Avantages d'une utilisation de variables locales* :
  - meilleure structuration
  - diminution des erreurs de programmation
  - les variables locales servent de variables intermédiaires (tampon) et sont « oubliées »(effacées de la mémoire) à la sortie de la procédure.

DÉFINITION

```
Procedure NIVEAU_SUP;
VAR X, Y, ...

Procedure NIVEAU_INF;
var X,Z,...
begin
    X:=...
    Y:=...
    Z:=...
end;

begin
end.
```

- La variable `X` déclarée dans `NIVEAU_SUP` est locale à cette procédure.
- Dans la procédure `NIVEAU-INF`, cette variable est occultée par l'autre variable `X`, déclarée encore plus localement.
- La portée de `Y` est celle de toute la procédure `NIVEAU-SUP`, sans restriction (elle est locale à `NIVEAU-SUP`, et donc globale dans `NIVEAU-INF`).
- En revanche, `Z` a une portée limitée à la procédure `NIVEAU-INF` (elle est locale à `NIVEAU-INF`, et ne peut être utilisée dans `NIVEAU-SUP`).

Exemple 80 *Le programme suivant utilise une procédure pour calculer le double d'un nombre entier. On a ici un exemple d'imbrication d'une fonction dans une procédure.*

```
program test;
   var  i,j : integer;  { variables globales }
   procedure INITIALISE;
   begin
        i := 0;
        j := 0;
   end;
   procedure CALCUL;
       function DOUBLE (i : integer) : integer;
```

```
    begin
        double := 2 * i;
    end;
begin
     i := 3;
     j := DOUBLE(i);
end;
Begin
     INITIALISE;
     readln(i,j);
     CALCUL;
     writeln(i,j);
End.
```

Comme il n'y a pas de déclarations locales de `i` *et* `j` *dans les procédures et la fonction mises en jeu, les variables* `i` *et* `j` *globales seront utilisées pour les calculs.*

IMPORTANT : Bien que tolérée, l'utilisation de variables globales est fortement déconseillée (risques de modifications de valeur non souhaitées) ; préférer les passages de paramètres. Si toutefois des variables globales doivent être utilisées, leur donner des noms suffisamment évocateurs pour éviter le risque de confusion.

Les règles à retenir sont les suivantes :

1. une variable est globale à un (ensemble de) bloc dès lors qu'elle est déclarée dans un bloc les emboîtant ; elle est visible dans le bloc où elle est déclarée et dans tous les blocs internes ; sa durée de vie est celle du bloc où elle est déclarée.
2. une variable est locale à un bloc si elle est déclarée dans ce bloc ; elle est visible seulement dans le bloc où elle est déclarée ; sa durée de vie est celle du bloc où elle est déclarée. une déclaration locale masque toujours une déclaration plus globale (si une déclaration locale porte le même nom qu'une déclaration plus globale, c'est le nom local qui l'emporte).
3. une variable déclarée localement n'existe que pendant l'exécution de la procédure, et ne sert que à cette procédure.
4. le programme principal n'a jamais accès à une variable locale de procédure.
5. une procédure n'a jamais accès à une variable locale d'une autre procédure.

6. les règles pour les paramètres sont les mêmes que pour les variables locales.

Exemple 81 *Programme avec procédures non imbriquées*

```
Program VisibiliteDesDeclarations ;
Const Pi = 3.14159 ;
Var Global : Real ;
Procedure Visibilite ( Var X : Real ) ;
Var  Pi : Real ;          { Local , masque la constante }
     T : tTableau ;       { Erreur : tTableau  inconnu car non encore déclaré }
Begin
      Pi := 3.14 ;
      X := Pi ;
      Global := Global - Pi
End ;
Type tTableau = Array  [ 1..5000 ] Of Real ;
Var R : Real ;
Begin { Du programme principal }
     Global := Pi ;
     Visibilite ( R ) ;
     Writeln ( X ) ;   { Erreur : X ( paramètre ) est local }
     Writeln ( R : 8 : 5  , '  ' , Global : 8 : 5 ) ;
End .
Après correction des 2 erreurs, ce programme fournit comme résultat :
3.14000    0.00159
```

Exemple 82 *Programme avec procédures emboîtées*

```
Program ProceduresEmboitees ;
Var  G : Real ;
     A : Integer ;
     Procedure Proc1 ;
         Procedure Proc1Bis ;
         Var A : Real ;
```

```
        Begin              { de Proc1Bis }
          A := 9.81 ;
          Writeln ( A : 5 : 2 , '  ' , G : 5 : 2 )
        End ;               { de Proc1Bis }
    Begin              { de Proc1 }
      Proc2 ;          { Interdit car non encore définie }
      Proc1Bis ;
      Writeln ( A : 5 , '  ' , G : 5 : 2 )
    End ;             { de Proc1 }
    Procedure Proc2 ;
    Begin              { de Proc2 }
      Proc1Bis ;    { Interdit car inaccessible }
      G := 10.5 ;
      Writeln ( A : 5 , '  ' , G : 5 : 2 )
    End ;
Begin    { du programme principal }
    G := 3.14 ;
     A := -5 ;
  Proc1 ;
     Proc2 ;
     Writeln ( A : 5 , '  ' , G : 5 : 2 ) ;
End .
```

Après correction des 2 erreurs, ce programme fournit comme résultat :

```
   9.81 3.14
     -5 3.14
     -5 10.5
     -5 10.5
```

6.7 Paramètres

6.7.1 Définition

Problématique : Exemple

Supposons qu'on ait à résoudre l'équation du second degré en divers points du programme :

- la première fois $Rx^2 + Sx + T = 0$,
- la deuxième fois $Mx^2 + Nx + P = 0$,
- la troisième fois $Ux^2 + Vx + W = 0$.

Comment faire en sorte qu'une ***même procédure*** puisse les traiter toutes les trois, c'est-à-dire ***travailler sur des données différentes*** ?

➥ 1ère possibilité : utiliser des variables globales `A`, `B`, `C` pour exprimer les instructions dans la procédure, et, avant l'appel de la procédure, faire exécuter des instructions telles que :

`A := R, B := S, C := T`, etc.

problème des variables globales, multiplication des affectations, etc ... . Solution à écarter !

➥ 2ème possibilité : définir une procédure de résolution d'une équation du second degré avec une liste d'arguments, et transmettre les données à l'aide de paramètres. La déclarations sera :

```
procedure SECOND_DEGRE(A,B,C:integer);
```

à l'appel, on écrira :

```
SECOND_DEGRE (M, N, P);   { appel avec les valeurs de M, N et P }
  SECOND_DEGRE (R, S, T);    { appel avec les valeurs de R, S et T }
```

Lors de la déclaration de la procédure, donner une liste d'arguments (paramètres) signifie une transmission de l'information entre le programme principal et le sous-programme

Il faut préciser le type des arguments par un identificateur et par une déclaration de type.

Exemple 83 `Procedure OK (x,y : real);` *Déclaration correcte*

préférer : `Procedure OK (tab : tableau);`

à :

```
Procedure OK (tab : array[1..100] of integer);
```

6.7.2 Paramètres formels et paramètres effectifs

Dans la déclaration de la procédure `SECOND_DEGRE` , les paramètres `A`, `B` et `C` sont appelés paramètres formels dans la mesure où ils ne possèdent pas encore de valeurs. Lors de l'utilisation de la procédure dans le programme principal (donc à l'appel) `M`, `N` et `P` sont des paramètres effectifs (ou réels).

☛Lors de l'appel de la procédure, il y a remplacement de chaque paramètre formel pa un paramètre effectif, bien spécifié.

Soit la déclaration de procédure suivante :

Exemple 84 `Procedure SECOND_DEGRE (A, B, C : integer);`

Lors de l'appel de la procédure, si on écrit :

```
SECOND_DEGRE (M, N, P);
```

A l'entrée du bloc de la procédure, `A` *prendra la valeur de* `M`*,* `B` *prendra celle de* `N` *et la variable* `C` *sera affectée de la valeur de* `P`*.*

☛Dans une procédure, le nombre de paramètres formels est exactement égal au nombre de paramètres effectifs. De même à chaque paramètre formel doit correspondre un paramètre effectif de **même type**.

☛Un paramètre formel est toujours une variable locale et ne peut être utilisé en dehors de la procédure (ou de la fonction) où il est défini. En revanche un paramètre effectif peut être une variable globale.

En Pascal, lors de la déclaration d'une procédure, il est possible de choisir entre deux modes de transmission de paramètres :

- passage par valeur et
- passage par adresse ;

6.7.3 Passage de paramètre par valeur

Rappelons d'abord que :

Déclarer une variable de nom `X` $\Longrightarrow$ réserver un emplacement mémoire pour `X`.

Ainsi lors de l'appel d'une procédure un emplacement mémoire est réservé pour chaque paramètre formel. De même un emplacement mémoire est aussi réservé pour chaque paramètre effectif lors de sa déclaration. Cependant lors de l'appel, **les valeurs des paramètres effectifs sont copiées dans les paramètres formels**. Ainsi l'exécution

des instructions de la procédure se fait avec les valeurs des paramètres formels et toute modification sur ces dernières ne peut affecter en aucun cas celles paramètres effectifs. Dans ce cas on parle de passage par valeur. C'est le cas dans la procédure `SAISIE` (paragraphe 6.3.1) du paramètre `dim` et dans la procédure `AFFICHAGE` (paragraphe 6.3.1) des paramètres `M` et `dim`. Notons que dans ce type de passage, les valeurs des paramètres effectifs sont connues avant le début de l'exécution de la procédure et jouent le rôle uniquement d'entrées de la procédure.

Exemple 85 `procedure ID_PROC (X, Y : real; I : integer; TEST: boolean);`

Points importants :

- Les paramètres formels sont des variables locales à la procédure, qui reçoivent comme valeur initiale celles passées lors de l'appel On parle alors d'`AFFECTATION`. Exemple : `ID_PROC (A, B, 5, true);` `X` a alors pour valeur initiale celle de `A`, `Y` celle de `B`. `I` a pour valeur initiale `5`, et `TEST true`
- Le traitement effectué dans la procédure, quel qu'il soit, ne pourra modifier la valeur des paramètres effectifs. Exemple : après exécution de `ID_PROC`, `A` et `B` auront toujours la même valeur qu'auparavant, même s'il y a eu des changements pour ces variables, dans la procédures.
- Le paramètre spécifié lors de l'appel peut être une expression.
  Ainsi, `ID_PROC (3 / 5, 7 div E, trunc (P), true)` ; est un appel correct.

Exemple 86 *Nous cherchons à écrire un programme qui échange les valeurs de deux variables saisies par l'utilisateur.*

```
program  TEST;
var
    A, B : real;
procedure  ECHANGE (X, Y : REAL);
var
    T : real;
begin
     T := X;
     X := Y;
```

```
      Y := T;
      Writeln(X,Y);
 end;
Begin
      readln (A, B);
      ECHANGE (A, B);
      writeln (A, B);
End.
```

Cette solution est mauvaise. En effet, une simulation de son exécution donnera :

`A = 5` `B = 7` *(saisie)*

`X = 7` `Y = 5`

`A = 5` `B = 7` *(*`A` *et* `B` *restent inchangés !)*

Le résultat de l'action accomplie par la procédure n'est pas transmis au programme appelant, donc `A` *et* ***B*** *ne sont pas modifiés.*

La Transmission est unilatérale.

Avantage : grande liberté d'expression dans l'écriture des paramètres effectifs. Cela évite les erreurs et les effets de bord (que nous verrons plus loin).

Tout paramètre est utilisé, pour passer des valeurs dans la procédure, mais n'est jamais modifié.

Si on désire récupérer l'éventuelle modification du paramètre, il faut alors utiliser un autre procédé, décrit dans la section suivante (passage par adresse).

Exemple 87 *Ceci est un programme assez qui ne fait pas grand chose et dont le seul intérêt est d'illustrer le passage de paramètres ainsi que la notion d'effet de bord.*

```
program Effet-de_bord;
var
       i,j : integer;
function double (i : integer) : integer;
   begin
         double := 2 * i;
   end;
function plus_un (j : integer) : integer;
   var
```

```
        i: integer;
    begin
          i := 10;
          plus_un := j + 1;
    end;
 function moins_un (j : integer) : integer;
    begin
          i := 10;
          moins_un := j - 1;
    end;
Begin     {programme principal}
     i := 0; j := 0;      {i=0 ; j=0}
     j := plus_un(i);     {i=0 ; j=1}
     j := double(j);      {i=0 ; j=2}
     j := moins_un(j);    {i=10 ; j=1}
End.
```

La variable `i` *a été modifiée dans la procédure, alors que l'on s'attendait à des modifications sur* `j`*. On appelle cela un effet de bord. Un tel phénomène peut être très* ***dangereux*** *(difficulté à retrouver les erreurs).*

Efet de bord Voici le scénario catastrophe à éviter

- On est dans une procédure `P` et on veut modifier une variable `x` locale à `P`.
- Il existe déjà une variable globale ayant le même nom `x`.
- On oublie de déclarer la variable locale `x` au niveau de `P`.
- A la compilation tout va bien !
- A l'exécution, `P` modifie le `x` global alors que le programmeur ne l'avait pas voulu.
- Conséquence : le programme ne fait pas ce qu'on voulait, le `x` global a l'air de changer de valeur tout seul !

$\Longrightarrow$ Erreur très difficile à détecter ; être très rigoureux et prudent !

6.7.4 Passage de paramètre par adresse

En pascal, la différence principale entre le passage par valeur et le passage par adresse c'est que dans ce dernier, un seul emplacement mémoire est réservé pour le paramètre

formel et le paramètre effectif correspondant. Autrement dit, dans ce cas chaque paramètre formel de la procédure utilise directement l'emplacement mémoire du paramètre effectif correspondant. Par conséquent toute modification du paramètre formel entraîne la même modification du paramètre effectif correspondant. Soulignons que dans ce mode de passage de paramètres, les valeurs des paramètres effectifs peuvent être inconnues au début de l'exécution de la procédure. Cependant un paramètre formel utilisant ce type de passage ne peut être que le résultat de la procédure.

Pour spécifier dans une procédure qu'il s'agit du mode par adresse, il suffit, lors de la déclaration de la procédure, d'ajouter le mot clé `VAR` devant la déclaration du paramètre concerné. C'est le cas par exemple du paramètre `S` dans la procédure `SOMME` de l'exemple du paragraphe 6.3.1.

Exemple 88 `procedure ID_PROC (var X, Y : real; Z : integer);`

☛ **Points importants :**

- Lors de l'appel, des paramètres réels sont substitués aux paramètres formels : **SUBSTITUTION**
- Seule une variable peut être substituée aux paramètres réels, il est impossible de faire l'appel avec une constante ou une expression évaluable !

  Exemple 89 `ID_PROC (U, V, 7);` *est correct.*

  `ID_PROC (4, A - B, 8);` *est tout à fait incorrect.*

- Tout changement sur le paramètre formel variable change aussi le paramètre effectif spécifié lors de l'appel.

Exemple 90
```
program essai;
var
      i : integer;
procedure double (x : integer ; var res : integer);
begin
      res := 2 * x;
end;
Begin
     i := 1;          {i=1}
     double (5,i);    {i=10}
```

```
End;
```

Dans cet exemple, `x` *est un paramètre transmis par valeur (donc non variable), alors que res est un paramètre passé par adresse (donc considéré comme variable).*

☛ **Principe :** s'il y a nécessité de renvoyer les modifications au programme appelant, on emploi des paramètres variables.

Exemple 91 *Reprenons et corrigeons le programme qui échange les valeurs de deux variables saisies par l'utilisateur.*

```
program  TEST_BIS;
var
    A, B : real;
procedure  ECHANGE (var  X, Y : real);
var
    T : real;
begin
     T := X;
     X := Y;
     Y := T;
     writeln(X,Y);
end;
begin
     readln (A, B);
     ECHANGE (A, B);
     writeln (A, B);
end.
```

Une simulation du déroulement du programme donnera :

`A = 5` `B = 7` *(saisie)*
`X = 7` `Y = 5`
`A = 7` `B = 5` *(*`A` *et* `B` *ont été modifiés !)*

Le résultat de l'action de la procédure a été transmis au programme appelant !

6.7.5 Bon réflexes

- ☛ Le seul moyen pour une procédure de communiquer avec l'extérieur, c'est à dire avec le reste du programme, ce sont les variables globales et les paramètres.
- ☛ Il faut toujours éviter soigneusement les effets de bords. Le meilleur moyen est de paramètrer compllétement les procédures, et d'éviter la communication par variables globales.
- ☛ Les variables de travail tels que compteur, somme partielle, etc doivent être locales à la procédure, surtout pas globale.
- ☛ Prendre l'habitude de prendre des noms de variables différents entre le programme principal et les procédures : on détecte plus facilement à la compilation les effets de bords.
- ☛ Chaque fois que l'on appelle une procédure, on vérifie particuliérement le bon ordre des paramètres et la correspondance des types. La compilation est trés pointilleuse sur les types, mais par contre elle ne détecte pas les inversions de paramètres de même type

6.7.6 Cas des fonctions

Pour les fonctions, on peut agir de même. Le principe est strictement analogue à celui des procédures. On distingue donc, là encore, les passages de paramètres par valeur des passages de paramètres par adresse (ou variables).

Exemple 92
```
function LETTRE (c : char) : boolean;
  begin
      if (c in ['A'..'Z']) or (c in ['a'..'z'])  then
                              lettre := true
                                  else
                                 lettre := false;
    end;
function MAJ (var c : char) : boolean;
  begin
    if not LETTRE(c)  then
                      maj : false
                else
                     begin
```

```
                    if c in ['a'..'z'] then
                            begin
                                c := chr (ord(c) - ord('a') + ord('A'));
                                maj := true;
                            end
                                  else
                                            maj := false;
                end;
    end;
```

Ce programme résume ce que nous avons pu voir avec les procédures. La première fonction utilise un passage de paramètre par valeur car nous n'avons pas besoin de modifier ce paramètre. Cette fonction est utilisée pour tester si un caractère est une lettre (minuscule ou majuscule).

La fonction `MAJ` *(qui utilise la fonction précédente) modifie un caractère qui est une lettre minuscule en sa majuscule correspondante.*

Chapitre 7

Comp

Algorigramme : Traduction graphique de l'algorithme. Parfois appelé Ordinogramme ou Organigramme.

On a une liste de un cas : une instruction séparée par des ;. Les listedecas peuvent être de plusieurs types :

- une valeur, par exemple mardi ;
- une liste de valeurs, séparées par des virgules. Pour reprendre l'exemple précédent, on peut choisir samedi,lundi ;
- un intervalle : jeudi..samedi.

Chapitre 8

Fichiers

CODES D'ERREUR DE IORESULT

```
{$I-}
ASSIGN(File, FileName);
RESET(File);
CLOSE(File);
{$I+}
Err:=IOResult;
CASE Err OF
  0   : WRITELN('No error'.);
  2   : WRITELN('File not found');
  3   : WRITELN('Path not found');
  4   : WRITELN('Too many open files');
  5   : WRITELN('File access denied');
  12  : WRITELN('Invalid file access code');
  15  : WRITELN('Invalid drive number');
  16  : WRITELN('Cannot remove current directory');
  17  : WRITELN('Cannot rename across drives');
  18  : WRITELN('No more files');
  100 : WRITELN('Disk read error');
  101 : WRITELN('Disk write error');
  102 : WRITELN('File not assigned');
  103 : WRITELN('File not open');
```

```
  104 : WRITELN('File not open for input');
  105 : WRITELN('File not open for output');
  106 : WRITELN('Invalid numeric format');
  150 : WRITELN('Disk is write-protected);
  152 : WRITELN('Drive not ready');
ELSE WRITELN('Error #, Err);
END;
```

Chapitre 9

La récursivité

« J'ai épousé une veuve qui avait une fille adulte. Mon père, qui nous rendait souvent visite devint amoureux de ma (belle) fille et l'épousa. Dès lors, mon père devint mon gendre et ma (belle) fille devint ma (belle) mère. Quelques mois plus tard, ma femme donna naissance à un fils qui devint le beau-frère de mon père et à ce titre aussi mon oncle par alliance. La femme de mon père - qui est ma (belle) fille - a donné aussi naissance à un fils. Du coup, j'avais un (demi) frère et en même temps un petit-fils. Ma femme est ma grand-mère puisqu'elle est la mère de ma mère (par alliance). Donc je suis le mari de ma femme mais aussi son petit-fils (par alliance). En d'autres termes, je suis mon propre grand-père. »
N. Wirth, " Algorithm + data structures = programs ", 1976.

9.1 Définition

La récursivité permet d'écrire des programmes qui résolvent des problèmes que l'on ne sait pas résoudre soi-même.

Une fonction ou une procédure est récursive dés lors qu'il est fait appel à cette fonction ou à cette procédure dans le corps d'instructions qui la définit. L'intérêt d'un tel concept est qu'il correspond bien à l'idée intuitive qu'on se fait d'un procédé de calcul automatique qui consiste à réappliquer une même procédure sur des objets similaires (quoique éventuellement de taille différente)

En termes informatiques, un objet est dit récursif, s'il se contient lui-même ou s'il est défini à partir de lui-même.

– la procédure ou fonction s'appelle elle-même
– il s'agit donc d'une tâche qui s'exprime simplement en fonction d'elle-même

Le concept de récursivité est spécialement mis en valeur dans les définitions mathématiques Les mathématiciens utilisent plutôt le mot récurrence :

> Si une propriété est vraie pour $K = 0$ et lorsqu'elle est vraie pour $K = N$, N quelconque mais fixe, on peut démontrer qu'elle est vraie pour $K = N + 1$, alors on peut affirmer qu'elle est vraie pour tout K supérieur ou égal à 0.

Des exemples classiques de définitions par récurrence sont les suivants :

– **Définition par récurrence d'un nombre entier naturel**
  (i) 0 est un entier naturel.
  (ii) Si N est un entier naturel, alors $N + 1$ est aussi un entier naturel.
  On sait donc que $N = 0$ est un entier naturel et on peut passer de N à $N + 1$.
– **Définition par récurrence de la fonction factorielle**
  (i) $0! = 1$.
  (ii) Pour N entier naturel $(N > 0), N! = N \times (N - 1)!$.

9.2 Récursivité simple

La fonction puissance $x \vdash x^n$ peut être définie récursivement comme suit :

$$x^n = \begin{cases} 1 & \text{si } n = 0 \\ x \times x^{n-1} & \text{si } n \geq 1 \end{cases}$$

La fonction correspondant s'écrit :

```
Fonction Puiss(x :reel ; n : entier) :reel ;
Debut
   SI n=0 ALORS
               Puiss :=1 ;
            SINON
               Puiss :=x*Puis(x,n-1) ;
Fin
```

9.3 Récursivité multiple

Une récurvité peut contenir plus d'un appel récursif. c'est le cas par exemple dans la définition des combinaisons C_n^p

$$C_n^p = \begin{cases} 1 & \text{si } p = 0 \text{ ou } p = n \\ C_{n-1}^p + C_{n-1}^{p-1} & \text{sinon} \end{cases}$$

L'algorithme correspondant est :

```
Fonction Combi(n :entier ; p : entier) :entier ;
Debut
   SI p=0 ou p=n ALORS
               Combi :=1
            SINON
               Combi :=Combi(n-1,p)+Combi(n-1,p-1) ;
Fin
```

9.4 Récursivité mutuelle

Des définitions sont dites mutuellement récursives si elles dépendent les unes des autres. C'est par exemple le cas pour la définition de la parité :

$$pair(n) = \begin{cases} vrai & \text{si } n = 0 \\ impair(n-1) & \text{sinon} \end{cases}$$

et

$$impair(n) = \begin{cases} faux & \text{si } n = 0 \\ pair(n-1) & \text{sinon} \end{cases}$$

Les algorithmes correspondant s'écrivent :

```
Fonction Pair(n :entier) :booleen ;
Debut
   SI n=0 ALORS
               Pair :=vrai
            SINON
               Pair :=Impair(n-1) ;
Fin
```

et

```
Fonction Impair(n :entier) :booleen ;
Debut
   SI n=0 ALORS
               Impair :=faux
            SINON
               Impair :=Pair(n-1) ;
Fin
```

9.5 Récursivité imbriquée

$$A(m,n) = \begin{cases} n+1 & \text{si } m = 0 \\ A(m-1,1) & \text{si } m > 0 \text{ et } n = 0 \\ A(m-1, A(m,n-1)) & \text{sinon} \end{cases}$$

```
Fonction Acker(m :entier ; n : entier) :entier ;
Debut
   SI m=0 ALORS
               Acker :=n+1 ;
            SINON
               SI n=0 ALORS
                           Acker :=Aker(m-1,1)
                     SINON
                           Acker :=Acker(m-1,Acker(m,n-1))
Fin
```

9.6 Exemples d'algorithmes récursifs

9.6.1 Somme des N premiers entiers

On désire calculer la somme des N premiers entiers naturels, comme par exemple : $1+2+3+4+5+6$.

Une définition de la fonction S sera alors :

$$S(N) = 1 + 2 + 3 + ... + N$$

C'est donc un programme itératif qu'il faudra utiliser :

```
Function SOMME(N:integer):longint;
Var
    i : integer;
    sum : longint;
Begin
    sum:=0;
    for i := 1 to N do
        sum := sum + i;
    SOMME := sum;
end;
```

Mais on peut aussi utiliser un programme récursif ! En effet, nous nous appuierons sur le fait que :

$$\begin{cases} S(N) = S(N-1) + N \\ S(0) = 0 & \text{pour la condition d'arrêt} \end{cases}$$

Le programme sera donc :

```
program ADDITION ;
var N : integer;
function SOMME ( N : integer ) : integer ;
begin
    if N = 0 then
                SOMME := 0
            else
                SOMME := SOMME(N - 1) + N ;
end;
begin
    readln(N);
    writeln(SOMME(N));
end.
```

La fonction `SOMME` est une fonction récursive car la définition de `SOMME` utilise `SOMME` elle-même. Lors de l'exécution du programme, à chaque appel de la fonction `SOMME`, le système va

effectuer un **empilement** (sauvegarde ou mise de côté) des valeurs paramètres et des résultats de calculs. A la sortie de la fonction, ces valeurs seront restituées (on dit qu'elles sont **dépilées**) et réinjectées dans le programme.

Exemple d'execution du programme avec N=4 :

```
début
appel de SOMME avec N = 3                  {début de l'empilement}
    début
    appel de SOMME avec N = 2
        début
        appel de SOMME avec N = 1
            début
            appel de SOMME avec N = 0      {fin de l'empilement}
                début
                SOMME <--- 0
                fin                        {début du dépilement}
            SOMME <--- SOMME(0) + 1
            fin
        SOMME <--- SOMME(1) + 2
        fin
    SOMME <--- SOMME(2) + 3
    fin
SOMME <--- SOMME(3) + 4
fin                                        {fin du dépilement}
```

Résultat : `SOMME(4) = 10`

9.6.2 Puissance d'un entier

Rappelons la formule de la fonction puissance :

$$x^n = x * x * x * \ldots * x$$

C'est itératif . Le programme sera donc :

```
Function PUISSANCE(X : real; N:integer):real;
  Var
      i : integer;
```

```
    puiss : longint;
Begin
    puiss:=1;
    if N > 0  then
                for i := 1 to N do
                      puiss := puiss * X;
              else
                for i:= N  to -1 do
                      puiss := puiss / X;
    PUISSANCE := puiss;
end;
```

Considérons à présent que :

$$x^n = \begin{cases} x * x^{n-1} & \text{si } n > 0 \\ x^{n+1}/x & \text{si} \end{cases}$$

et que la condition d'arrêt est : $x^0 = 1$.
On obtient alors un programme récursif :

```
program PUIS ;
var
    X : real ;
    P : integer ;
function PUISSANCE (X : real ; N : integer ) : real ;
begin
    if N = 0  then
                PUIS := 1
              else
                 if N > 0 then
                           PUIS := PUIS (X , N - 1) * X
                         else PUIS := PUIS (X , N + 1) / X
end;
begin
    write('Nombre :');
    readln(N);
    write('Exposant entier :');
```

```
        readln(P);
        writeln('Le résultat est :', PUISSANCE (X , P) ) ;
   end.
```

9.6.3 Conversion binaire récursive

Ecrire un programme récursif qui fait la conversion en base d'un entier

On démontre que tout entier $N(N \geq 0)$ s'écrit de façon unique sous la forme :

$$N = a_0 + a_1 2^2 + \ldots + a_p 2^p$$

où les a_i sont des entiers égaux à 0 à ou à 1. L'écriture $a_p a_{p-1} \ldots a_0$ (juxtaposition des chiffres $a_p, a_{p-1}, \ldots, a_0$ est appelée écriture en base 2 de N. On remarque que a_0 est le reste de la division euclidienne de N par 2 ; si Q en est le quotient, a_1 est le reste de la division euclidienne de Q par 2 ; etc.

Le programme permettant la conversion est :

```
PROGRAM Recubin;
var N : integer;

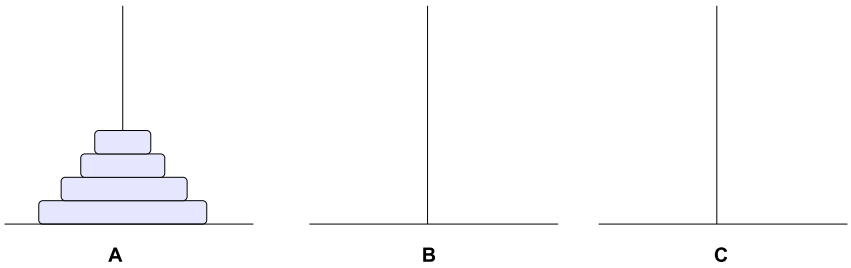
PROCEDURE BIN(M : integer);
BEGIN
     IF  M<2 THEN
               write(M)
             ELSE
               BEGIN
                  BIN(M DIV 2);
                  write( M mod 2);
               END;
END; (* BIN*)

BEGIN
       Write('Introduisez un nombre entier '); readln(N);
       WHILE N<999 DO
               BEGIN
                     BIN(N); writeln;
                     write('Introduire un nombre entier ');
```

```
                readln(N);
            END
END.
```

9.6.4 Tour de Hanoï

Explication du problème On dispose de trois tours numérotées A, B, C. Au départ n disques circulaires de diamètres décroissants, numérotés 1 (le plus petit) jusqu'à n (le plus grand) se trouvent sur la tour A, le plus grand disque se trouve tout en bas et le plus petit au-dessus.



BUT : Déplacer la pile de la tour A à la tour B ; en ne déplaçant qu'un disque à la fois, et en s'assurant que nul disque ne repose sur un disque de plus petite dimension.

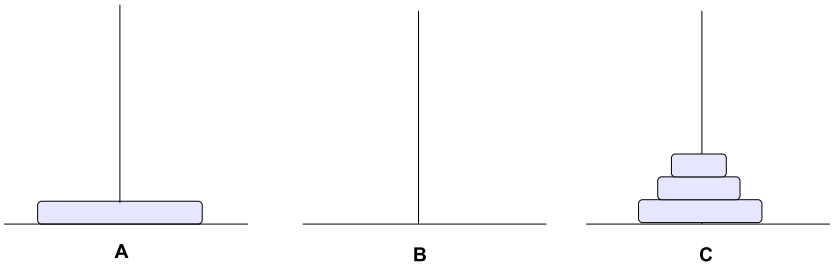
Décomposition de la tache : On va imaginer, afin de ramener le problème à un nombre de disques plus petit, que l'on sait faire passer tous les disques sauf le dernier, d'un pilier à un autre. On a donc diminué de un le nombre de disque à déplacer. C'est le principe de la récursivité : on va définir le problème à partir d'un problème similaire d'ordre inférieur (comme factorielle $N! = N * (N-1)!$ ou puissance $x^n = x * x^{n-1}$).

Soit à déplacer N disques de la tour 1 à la tour 3. Pour ce faire :

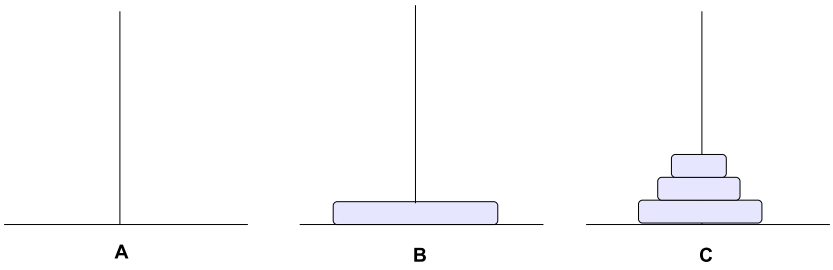
- déplacer $N-1$ disques de la tour 1 à la tour 2 (on suppose qu'on sait le faire)
- déplacer 1 disque de la tour 1 à la tour 3 (dans ce cas, il ne reste plus qu'un seul disque, c'est facile)
- déplacer $N-1$ disques de la tour 2 à la tour 3 (même supposition qu'au début : on sait le faire)

On ramène donc la résolution du problème à l'ordre N à la résolution du même problème à l'ordre $N-1$

On obtient donc :



Ensuite on déplace le disque n de A vers B et on obtient :



Pour résoudre entièrement le problème, il ne reste qu'à déplacer $n-1$ disques mis en réserve de C vers B.

Programme récursif

```
program HANOI ;
var
    NDISQUE : integer ;
procedure TOUR ( N_DISQ , T_ORIG , T_DEST , T_INT : integer) ;
begin
    if N_DIS > 0 then
            begin
                TOUR (N_DIS - 1 , T_ORIG , T_INT , T_DEST) ;
                writeln ('Déplacer le disque de ', T_ORIG,' A', T_DEST) ;
                TOUR (N_DIS - 1 , T_INT , T_DEST , T_ORIG) ;
            end;
end;
begin
    write('l''Entrez le nombre de disque :');
    readln(NDISQUE);
    TOUR (NDISQUE , 1 , 3 , 2 );
```

```
end.
```

9.6.5 Dessin en abyme

On dispose d'un matériel comprenant une feuille de papier et une plume manipulée à l'aide de deux primitives :

- *placer*(x, y) qui positionne la plume aux points de coordonnées (x, y)
- tracer(x, y) qui trace le segment de droite reliant le point de positionnement de la plume au point de coordonnées (x, y). La plume est alors positionnée en (x, y)

On désire alors tracer un dessin selon le motif suivant décrit dans la figure 9.1 :

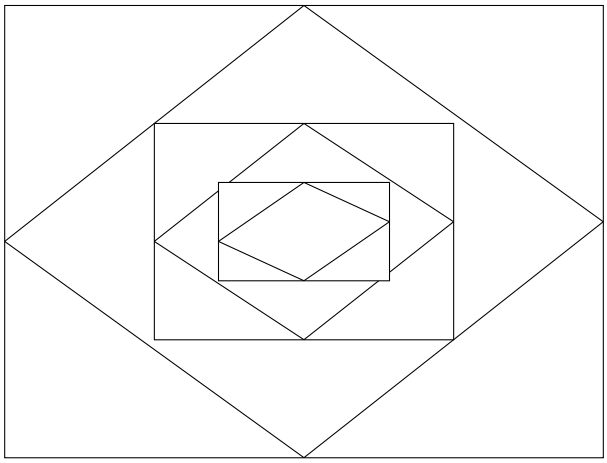


FIGURE 9.1 – Dessin en abyme.

Soit P_n $(n \geq 0)$ un dessin avec n motifs. Si $n \geq 1$ on peut décomposer P_n en P_1 et P_{n-1} ou :

– P_1 représente le motif le plus grand et

– P_{n-1} le motif le plus petit.

Pour minimiser le déplacement de la plume, nous choisirons de tracer le dessin :

– sans lever la plume

– et sans jamais repasser sur un trait déjà tracé.

$\Longrightarrow$ la plume revient à son point de départ après avoir tracé P_n

$\Longrightarrow$ le point de départ et d'arrivée de la plume pour P_{n-1} se trouve sur le tracé de P_1 et c'est le sommet du plus grand carré de P_{n-1}

Les contraintes ci dessous interdisent de dessiner complètement P_1 avant P_{n-1}. On doit :

(1) tracer les segments de P_1 reliant le somme de départ de P_1 au sommet de départ de P_{n-1}

(2) tracer P_{n-1}

(3) achever le tracer de P_1 en reliant le point de départ de P_{n-1} au point de départ de P_1

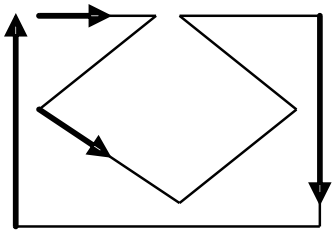


FIGURE 9.2 – Mode de tracé de P_1 sans jamais lever la plume :

Nous obtenons la procédure suivante dans laquelle x et y sont les coordonnées du point de départ du dessin, t la longueur du coté du plus grand carré du dessin

```
PROCEDURE Dessin(x, y : reel; t :reel;  n : entier)
DEBUT
SI n > 0 ALORS
DEBUT
                    Tracer(x+t/2, y) ;
                    Tracer(x+t/4, y-t/4) ;
                    Dessin(x+t/4, y-t/4, t/2, n-1) ;
                    Tracer(x,y-t/2) ;
                    Tracer(x+t/2, y-t) ;
                    Tracer(x+t, y-t/2) ;
                    Tracer(x+t/2, y) ;
                    Tracer(x+t,y) ;
                    Tracer(x+t, y-t) ;
                    Tracer(x, y-t) ;
                    Tracer(x, y);
        FIN
FIN
```

9.7 Les Algorithmes de Tri récursif

9.7.1 Tri avec insertion récursif

Soit à trier un tableau `T` de type `TAB` défini par :

`Type TAB = Tableau[1..nmax] d'entier`

Un tri récursif, $TRI(N)$ des N éléments du tableau T peut être spécifié comme suit :

```
TRI(N) = Si N>1 Alors :
                 TRI(N-1) (* tri des N-1 éléments du tableau*)
                 X:=T[N]
                 Inserer(X,N-1) (* insertion de T[N]
                  dans les N-1 éléments déjà triés*)
```

Les procédures `INSERER` et `TRI` sont définies ci-dessous (le tableau `T` est une variable globale) :

```
PROCEDURE INSERER(X : integer; k : integer);
VAR  i : integer;
BEGIN
   IF  (X >= T[k]) THEN
                    T[k+1]:=X
                  ELSE
                    IF  (X<= T[1]) THEN
                                    BEGIN
                                      FOR i:=k  DOWNTO 1 DO
                                            T[i]:=T[i+1];
                                      T[1]:=X;
                                    END
                                  ELSE
                                    BEGIN
                                       i:=k;
                                       WHILE  (T[i]>X) DO
                                              BEGIN
                                                T[i+1]:=T[i];
                                                i:=i-1;
                                              END;
                                        T[i+1]:=X;
                                   END;
END;

PROCEDURE TRI(N : integer);
```

```
VAR X : integer;
BEGIN
      IF (N>1) THEN
                  BEGIN
                    TRI(N-1);
                    X:=T[N];
                    INSERER(X,N-1);
                  END;
END;
```

9.7.2 Diviser pour régner

Les algorithmes de tri itératifs étudiés jusqu'ici sont tous d'une complexité de l'ordre de $o(n^2)$. Dans chaque cas, un passage du vecteur réduit un problème en n à un problème en $n-1$. Comme pour la dichotomie par apport à la recherche linéaire, il es possible de mieux faire en réduisant un problème en n à deux problèmes en $n/2$. Le terme employé dans ce cas est celui de diviser pour régner. On divise le vecteur en deux moitié, on trie chaque moitié et on remet les deux ensembles. On distingue deux groupes d'algorithme dans cette catégorie : le tri rapide et le tri par fusion.

Le tri rapide Le tri rapide est structurellement récursif et constitue par ailleurs l'un des algorithmes les plus utilisés dans les applications informatiques. Dans le cas d'une distribution aléatoire de valeurs, surtout si n est grand l'algorithme de tri rapide est le meilleur de ceux présentés dans ce cours.

Principe Le principe de base du tri rapide (*quick sort*) est le partitionnement en trois parties du tableau T à ordonner suivant un élément pivot de telle sorte que les éléments du premier soient plus petit que le pivot et les éléments du second plus grands. On appelle récursivement la procédure de tri sur ces deux sous tableaux.

- $T[milieu]$ valeur de comparaison, avec $milieu = partieEntiere(g+d)/2$ et g et d les bornes inférieure et supérieure du tableau traité.
- $T[g], T[g+1], \ldots, T[milieu-1]$ où les $T[k]$ $T[i]$ pour tout k élément de $[g, milieu-1]$
- $T[milieu], T[mileu+1], \ldots, T[d]$ où les $T[k]$ $T[i]$ pour tout k élément de $[milieu+1, d]$ et ensuite du tri rapide des tableaux :
- $T[g], \ldots, T[milieu-1]$ et
- $T[milieu+1], \ldots, T[d]$.

Le partitionnement se fait de la façon suivante : au départ on choisit un pivot et le tableau est ensuite partitionné autour de ce pivot. On se déplace vers la droite du tableau tant qu'on reste inférieur à la valeur du pivot et vers la gauche tant qu'on reste supérieur vers à la valeur

du pivot. On permute les deux éléments mal placés, et on réitère ce processus jusqu'à ce que l'indice de parcours partant de la gauche dépasse celui partant de la droite. Le même processus est appliqué sur les deux tableaux résultant du partitionnement.

Pour choisir ce pivot, l'idéal serait de prendre une valeur médiane, mais sa recherche prendrait trop de temps, alors on peut choisir aléatoirement une valeur. En l'occurrence, j'ai choisi la valeur du milieu.

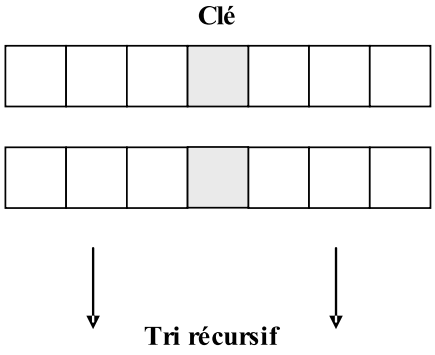


FIGURE 9.3 – Tri rapide

La procédure réalisant le tri est la suivante :

```
PROCEDURE TRI_RAPIDE( var T : Tab; deb,fin : entier) ;
DEBUT
   SI (deb<fin) ALORS
                  DEBUT
                     PARTITIONNER( T ,deb,fin) ;
                     TRI_RAPIDE(T,deb,((deb+fin) DIV 2) - 1);
                     TRI_RAPIDE(T,((deb+fin) DIV 2)+1,fin);
                  FIN
FIN
```

La procédure `PARTITIONNER` est la suivante

```
PROCEDURE PARTITIONNER( var T : Tab; gauche,droite : entier).
VARIABLES  i, j, pivot: entier;
DEBUT
   i:=gauche; j:=droite;
```

```
   pivot:=(gauche+droite) DIV 2;
   REPETER
        TANTQUE (T[i]<T[pivot]) FAIRE
                 i:=i+1;
        TANTQUE (T[j]>T[pivot]) FAIRE
                 j:=j-1;
        SI (i<=j) ALORS
                    DEBUT
                      aux:=T[i];
                      T[i]:=T[j];
                      T[j]:=aux;
                      i:=i+1;
                      j:=j-1;
                    FIN
   JUSQUA (i>j)
FIN
```

Le tri fusion

Le tri fusion consiste à couper le tableau à trier en deux sous tableaux de même taille, puis à appeler récursivement le tri sur les deux sous tableaux ainsi obtenus, qu'on fusionne une fois triés à l'aide d'une fonction de complexité linéaire (fonction de fusion de deux tableaux triés vue dans le chapitre sur les tris).

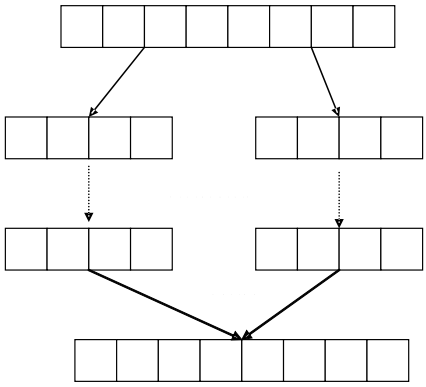


Figure 9.4 – Tri fusion

```
PROCEDURE TRI_FUSION( var T : Tab; deb,fin : entier).
DEBUT
```

```
   SI (deb<fin) ALORS
                  DEBUT
        TRI_FUSION(T,deb,(deb+fin) DIV 2) ;
                     TRI_FUSION(T,((deb+fin) DIV 2)+1,fin) ;
                     FUSIONNER(T,deb,(deb+fin) DIV 2,fin) ;
                  FIN
FIN
```

La procédure FUSION est laissée à titre d'exercice.

Chapitre 10

Algorithmes de tri

10.1 Le problème du tri

- tri (synonyme : classement) : opération qui, en général, a pour but de classer des objets, des documents, des articles d'un fichier etc. dans l'ordre croissant ou décroissant des indicatifs ou des numéros d'ordre attachés à chacun de ces objets.
- Il est nécessaire que ces indicatifs appartiennent à un ensemble ordonné au sens mathématique (par exemple ordre lexicographique). Ce seront par exemple des poids ou des numéros ou des indicatifs alphanumériques. Cet ensemble ordonné est appelé critère de tri, indicatif de tri ou clé de tri.
- Lorsqu'une structure est triée, elle permet un accès rapide pour chercher une information surtout lorsque le nombre d'informations que possède cette structure est très important comme dans les bases de données.
- On distingue traditionnellement
  - le tri interne opérant sur des données présentes en mémoire centrale, et
  - le tri externe opérant sur des données appartenant à des fichiers externes

Nous nous limiterons ici aux problèmes du tri interne car ils sont mieux formalisés, plus facile à étudier et indépendants de l'environnement (ordinateur, système,etc.). Nous nous intéresserons aussi aux méthodes de tri des tableaux. Le même principe reste valable sur n'importe quelle autre structure de données.

10.2 Spécification du tri

Soit T un tableau de n éléments muni d'une relation d'ordre $op \in (<=, >=, <, >)$. Trier ce tableau c'est construire un tableau qui devra satisfaire la spécification suivante :

$\forall i \in [1..n-1]\, t[i]\ op\ t[i+1]$

Cette relation devra être vérifiée par n'importe quel algorithme de tri.

10.3 Méthodes de tri

Il existe plusieurs algorithmes de tri. Dans ce chapitre nous allons nous limiter aux algorithmes de tri suivants

- tri par sélection
- tri par bulles
- tri par insertion (insertion dichotomique et insertion séquentielle)

Les algorithmes de tri récursif seront étudiés dans le chapitre sur la récursivité. Nous considérons des tableaux de type `TAB` défini par

```
Type TAB = Tableau[1..nmax] d'entier
```

10.3.1 Le tri par sélection

Principe

Le tri par sélection est l'algorithme de tri le plus simple. Le principe consiste à trouver l'emplacement de l'élément le plus petit du tableau, c'est-à-dire l'entier m tel que $T_i < T_m$ pour tout i. Une fois cet emplacement trouvé, on échange les éléments T_i et T_m. Puis on recommence ces opérations avec la suite $< T_2, T_3, \ldots, T_n >$ Ainsi, on recherche le plus petit de cette nouvelle suite et on l'échange avec T_2. Et ainsi de suite... jusqu'au moment ou a une suite constituée d'un seul élément T_n

```
PROCEDURE Tri_Selection (var t:TAB, n :entier);
VARIABLES  i,j,aux,imin : entier
DEBUT
      POUR i:=1  A n-1 FAIRE
             DEBUT
                   imin :=i;
                   POUR j:=i+1 A n FAIRE
                            SI t[j]<t[imin] ALORS  imin:=j;
                   aux:=t[i]:
                   t[i]:=t[imin];
                   t[imin]:=aux;
            FIN
FIN
```

Exemple 93 *Appliquer l'algorithme avec le tableau suivant*

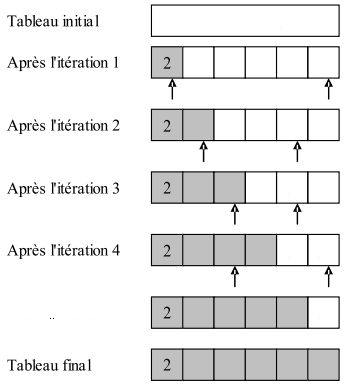


FIGURE 10.1 – Tri par sélection

10.3.2 Tri par insertion

Voici un algorithme de tri basé sur l'insertion dans un vecteur trié. Principe On est à l'étape i.

- Supposons déjà trié `v[1..i-1]`, et non traité `v[i..vn]`.
- On pose `x = v[i]`. La case `i` est disponible.
- On cherche `k` tel que `v[k-1]<= x <v[k]`.
- On insère `x` à la position `k` dans `v[1..i-1]`, ce qui oblige à décaler d'abord `v[k..i-1]` vers `v[k+1..i]` ; le trou en i est bouché.
- Maintenant `v[1..i]` est trié et `v[i+1..vn]` est non traité.

10.4 Tri par comptage

Cette méthode consiste à construire un vecteur d'indices ind, où l'on calcule la position que devrait avoir chaque élément pour que le vecteur soit trié.

Résultat sur le tri des lettres du mot `'ETABLES'`.

i	1	2	3	4	5	6	7
v	E	T	A	B	L	E	S
ind	3	7	1	2	5	4	6
w	A	B	E	E	L	S	T

```
PROCEDURE tri_comptage ( v : vec_t; vn : integer; var w : vec_t);
VAR i, k : integer;
     ind : array [1..VMax] of integer;
BEGIN
    { init }
    for i := 1 to vn do ind[i] := 1;
    { construit ind }
    for i := 1 to vn-1 do
        for k := i+1 to vn do
                if v[k] < v[i]  then
                                   ind[i] := ind[i]+1
                                else
                                   ind[k] := ind[k]+1;
    { construit w }
    for i := 1 to vn do w[ind[i]] := v[i];
END;
```

10.4.1 Tri avec insertion récursif

Soit à trier un tableau T de type TAB défini par :

```
Type TAB =Tableau[1..nmax] d'entier
```

Un tri récursif, TRI(N) des N éléments du tableau T peut être spécifié comme suit :

$$TRI(N) = \begin{cases} \text{Si } N > 1 \text{ Alors } TRI(N-1) \\ X := T[N] \\ Inserer(X, N-1) \end{cases}$$

```
Procedure INSERER(X : entier ; k : entier) ;
VAR i : entier ;
Debut
   SI X >= T[k] ALORS
                  T[k+1] :=X
                  SINON
                     SI X<= T[1] ALORS
                           Début


   Fin
```

10.4.2 Tri par fusion

10.4.3 Tri rapide

10.5 Conclusion

Chapitre 11

Structures de données élémentaires

De manière générale, résoudre un problème à l'aide d'un ordinateur revient à déterminer un algorithme opérant sur une *structure de données* bien choisie. En d'autres termes, il faut faut structurer le codage de l'information de façon adéquate. Nous allons dans ce chapitre en étudier quelques exemples élémentaires, notamment les listes, les piles, les files et les arbres tous représentant la notion d'ensemble.

En informatique il existe plusieurs manières de représenter la notion mathématique d'ensemble. Ce chapitre introduit quelques structures utilisées de façon très intensive en programmation. L'objectif principal est de gérer un ensemble fini d'éléments dont le nombre n'est pas fixé à priori. Les éléments de cet ensemble peuvent être des entiers ou des réels, des chaînes de caractères ou même des objets informatiques plus complexes. Par ailleurs, il est possible d'appliquer un certain nombre d'opération sur cet ensemble indépendamment de la nature des élément le constituant. La série d'opération potentiellement supportée est la suivante :

- $vide(E)$ permet de tester si l'ensemble E est vide ou pas
- $recherche(E, x)$ vérifie l'appartenance de l'élement x à l'ensemble E
- $insertion(E, x)$ ajoute x à l'ensemble E
- $suppression(E, x)$ supprime l'élément x de l'ensemble E

11.1 Rappels Les pointeurs

11.1.1 Notion de pointeur

On appelle *Lvalue* (*left value*) tout objet pouvant être placé à gauche d'un opérateur d'affectation. Une *Lvalue* est caractérisée par :

- son adresse, c'est-à-dire l'adresse-mémoire à partir de laquelle l'objet est stocké ;

– sa valeur, c'est-à-dire ce qui est stocké à cette adresse.

```
int i, j;
i = 3;
j = i;
```

Deux variables différentes ont des adresses différentes. L'affectation `i = j;` n'opère que sur les valeurs des variables

– Pointeur = adresse sur un objet de type donné.
– Sert principalement pour la mémoire dynamique
– description d'un pointeur
  `type * identificateur`
– l'opérateur d"adressage `&` permet d'accéder à l'adresse d'une variable. Toutefois `&i` n'est pas une *Lvalue* mais une constante : on ne peut pas faire figurer `&i` à gauche d'un opérateur d'affectation. Pour pouvoir manipuler des adresses, on doit donc recourir un nouveau type d'objets, les pointeurs.
– Par exemple l'expression
  `char * p`
  définit la variable p comme un pointeur vers un caractère et
  `*p`
  désigne le caractère contenu à l'adresse `p`
– " Déclaration
  Exemple 94 `char x, * p;`
  `. . .`
  `p = &x ;`
  `*p = w;`
  `. . .`
  `typedef char * type_p_char`
  `typedef enum{rouge, noir} * type_p_couleur;`
  `personne * p_perso;`
  `typedef personne * type_p_personne;`
  `type_p_perso p_jean`

• Une définition de pointeur alloue de l'espace suffisant pour contenir une adresse
• On peut pointer sur n'importe quel type
  – prédéfini
  – structure, union, enum
  – tableau, chaîne

  - pointeur
  - fonction
- Le type `void *` est un type générique de pointeur, il ne spécifie pas la nature de l'objet pointé
  `int f(void *, void *)`
- Taille d'un pointeur
  elle est égale à `sizeof(void *)` qui est souvent la taille de `int`

11.1.2 Pointeurs et arguments

Les pointeurs permettent de réaliser une transmission par adresse des arguments aux fonctions.

Les tableaux sont transmis de cette façon

Permet à une fonction de travailler directement sur les arguments à l'appel

Exemple 95 . . .

```
void echange(int px, int py){
    int temp = px:
    px = py;
    py = temp;
}
. . .
int x, y
. . .
/* échange des valeurs de x et y */
echange(&x,&y);
. . .
```

11.1.3 Pointeurs et tableaux

Un tableau t est assimilé à l'adresse `&t[0]` de son premier élément une variable tableau est un pointeur constant sur le type de base

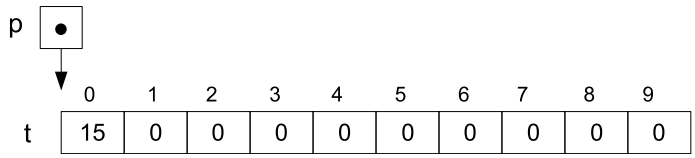
Exemple 96

```
main(){
    short t[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
    short *p;
    printf("t = %p, &t[0] = %p\n",t, &t[0]);
```

```
    p = t
    *p = 15;
    printf("p = %p, &t[0] = %p\n",p, &t[0]);
}
```

Résultats

```
t  = 001CCE68, &t[0] = 001CCE68
p = 001CCE68, t[0] = 15
```



11.1.4 Arithmétique des pointeurs

La valeur d'un pointeur étant un entier, on peut lui appliquer un certain nombre d'opérateurs arithmétiques classiques. Les seules opérations arithmétiques valides sur les pointeurs sont :

- l'addition d'un entier à un pointeur. Le résultat est un pointeur de même type que le pointeur de départ ;
- la soustraction d'un entier à un pointeur. Le résultat est un pointeur de même type que le pointeur de départ ;
- la différence de deux pointeurs pointant tous deux vers des objets de même type. Le résultat est un entier.

Notons que la somme de deux pointeurs n'est pas autorisée.

L'arithmétique des pointeurs est définie pour être compatible avec les tableaux : c'est une arithmétique relative à la taille du type de base

`TRUC * p ;  int i ; TRUC  t[100] ;`

Ces déclarations introduisent `p` comme variable pointeur et `t` comme constante pointeur. Ainsi les expresion `p = t` et `p++` sont légitimes, mais `t = p` et `t++` sont illégales.

- Pointeur +/- entier
  `p+i` est de type `TRUC *`
  pointe `i` objets de type `TRUC` après celui pointé par `p` (sa valeur entière est `(size_t)p + i*sizeof(TRU`
- Compatibilité avec les tableaux
  `t[i] est évalué comme *(t+i)`

(en particulier on peut écrire `p[i]` pour `*(p+i)`) Si `p` pointe sur `t[k]`, `p+i` pointe sur `t[k+i]`

Les opérations `++` et `--` s'appliquent aux pointeurs (pas aux tableaux qui sont des pointeurs constants)

- `++p` pointe sur l'objet suivant de type `TRUC`
- `++p` a pour valeur entière `(size_t)p+sizeof(TRUC)`

- Différence de pointeurs (de même type)
  nombre d'objets compris entre leurs adresses
  `(p+i)-p` vaut `i` et est du type entier signé `ptrdiff_t` de `<stddef.h>`
- En particulier, `&t[i+1]-&t[i]` vaut `1`
- Si `p` et `q` sont deux pointeurs sur des objets de type type, l'expression `p - q` désigne un entier dont la valeur est égale à `(p - q)/sizeof(type)` .

Les opérateurs de comparaison sont également applicables aux pointeurs, à condition de comparer des pointeurs qui pointent vers des objets de même type.

11.1.5 Copie de tableau

```
. . .
double s[ TAILLE],  t[TAILLE]
. . .
void copie1( double s[ ], double t[ ] ){
     int i;
     for( i = 0 ;  i < TAILLE  ; i++)
              t[i] = s[i];
}
void  copie2(double s[ ], double t[ ]){
      int i;
      for ( i = 0 ; i <TAILLE ; i++)
              *(t+i) = *(s+i)
}
void copie3(double *s, double *t){
     int i;
     for ( i = 0; i < TAILLE ; i++)
            *t++ = *s++
}
. . .
```

Dans les fonctions, `s` et `t` sont des variables locales de type pointeur ; donc `s++` et `t++` sont des expressions légales. Elles ne sont pas légales sur les variables globales `s` et `t` qui sont de type tableau

☛REMARQUES

- En paramètre de fonction TYPE * t est équivalent à TYPE t[]
- Comparaisons possibles (==, !=) entre pointeurs
  - `p != q, p == 0`
  - `(p)` a la valeur de verité de(`p!=NULL`)
  - (`!p`) a la valeur de vérité de (`p == NULL`
- `*p++ = i ;` a le même effet que `* p = i , p = p+1;`
- opérations interdites : `-` (moins unaire), `*, /, %`, opération de bit à bit

11.1.6 Allocation dynamique

- Avant de manipuler un pointeur, et notamment de lui appliquer l'opérateur d'indirection `*`, il faut l'initialiser. Sinon, par défaut, la valeur du pointeur est égale à une constante symbolique notée `NULL` définie dans `stdio.h`. En général, cette constante vaut `0`. Le test `p == NULL` permet de savoir si le pointeur `p` pointe vers un objet.
- On peut initialiser un pointeur `p` par une affectation sur `p`. Par exemple, on peut affecter à `p` l'adresse d'une autre variable. Il est également possible d'affecter directement une valeur à `*p`. Mais pour cela, il faut d'abord réserver à `*p` un espace-mémoire de taille adéquate. L'adresse de cet espace-mémoire sera la valeur de `p`. Cette opération consistant à réserver un espace-mémoire pour stocker l'objet pointé s'appelle allocation dynamique. Cette utilisation de la mémoire dynamique se fait en C par les fonctions de la bibliothèque `<stdlib.h>` :
- `void malloc(size_t nombre_octets)`
  - retourne un pointeur de type `void *` pointant vers un objet de taille `nombre_octets` octets. Pour initialiser des pointeurs vers des objets qui ne sont pas de type `void`, il faut convertir le type de la sortie de la fonction malloc à l'aide d'un cast. L'argument `nombre_octets` est souvent donné à l'aide de la fonction `sizeof()` qui renvoie le nombre d'octets utilisés pour stocker un objet.
  - NULL en cas d'échec
  - zone non initialisée
- **Exemple 97** *Ainsi pour initialiser un pointeur p vers un entier, on écrit :*

```
#include<stdlib.h>
int * p;
p = (int *) malloc(sizeof(int));
```

on aurait pu également écrire Ainsi pour initialiser un pointeur p vers un entier, on écrit :
`p = (int*)malloc(4);`
puisqu'un objet de taille int est stocké sur 4 octets.

- `void * calloc(size_t nb, nombre_octets)`
  - retourne un pointeur sur une zone pour un tableau de `nb` éléments de `nombre_octets` chacun. Même rôle que `malloc`, mais elle initialise en plus l'objet pointé `*p` à zéro
  - `NULL` en cas d'échec
  - zone initialisée à `0`
- **Exemple 98** *Ainsi, si* `p` *est de type* `int*`*, l'instruction*
  `p = (int*)calloc(N,sizeof(int));`
  est strictement équivalente à

```
p = (int*)malloc(N * sizeof(int));
for (i = 0; i < N; i++)
*(p + i) = 0;
```

  L'emploi de `calloc` *est simplement plus rapide.*
- `void free(void * p)` lorsque l'on n'a plus besoin de l'espace-mémoire alloué dynamiquement (c'est-à-dire quand on n'utilise plus le pointeur p), il faut libérer cette place en mémoire.
  - libère la zone d'adresse `p` (allouée par `Xalloc`)
  - aucun effet si `p = NULL`
- `void * realloc(void * p,size_t n)`
  - modifie la taille de l'objet `p` pointé par `p` (alloué par `Xalloc`)
  - retourne un pointeur sur une zone de `nombre_octets` octets ;
  - `NULL` en cas d'échec
- NB : A toute instruction de type `malloc` ou `calloc` doit être associée une instruction de type `free`.

11.1.7 Pointeurs et tableaux

11.1.7.1 Pointeurs et tableaux à une dimension

Tout tableau en C est en fait un pointeur constant. Dans la déclaration
`int tab[10];`
`tab` est un pointeur constant (non modifiable) dont la valeur est l'adresse du premier élément du tableau. Autrement dit, `tab` a pour valeur `&tab[0]`. On peut donc utiliser un pointeur initialisé à `tab` pour parcourir les éléments du tableau.

Exemple 99 `#define N 5`

```
int tab[5] = {1, 2, 6, 0, 7};
main()
{
  int i;
  int *p;
  p = tab;
  for (i = 0; i < N; i++)
    {
      printf(" %d \n",*p);
      p++;
    }
}
```

- On accède à l'élément d'indice `i` du tableau tab grâce à l'opérateur d'indexation `[]`, par l'expression `tab[i]`. Cet opérateur d'indexation peut en fait s'appliquer à tout objet `p` de type pointeur. Il est lié à l'opérateur d'indirection `*` par la formule `p[i] = *(p + i)`
- Pointeurs et tableaux se manipulent donc exactement de même manière. Par exemple, le programme précédent peut aussi s'écrire

  Exemple 100 `#define N 5`

  ```
  int tab[5] = {1, 2, 6, 0, 7};
  main()
  {  int i;
      int *p;
     p = tab;
     for (i = 0; i < N; i++)
     printf(" %d \n", p[i]);
  }
  ```

Toutefois, la manipulation de tableaux, et non de pointeurs, possède certains inconvénients dûs au fait qu'un tableau est un pointeur constant. Ainsi

- on ne peut pas créer de tableaux dont la taille est une variable du programme,
- on ne peut pas créer de tableaux bidimensionnels dont les lignes n'ont pas toutes le même nombre d'éléments.
- Ces opérations deviennent possibles dès que l'on manipule des pointeurs alloués dynamiquement.

Exemple 101 *Ainsi, pour créer un tableau d'entiers à* `n` *éléments où* `n` *est une variable du programme, on écrit*

```
#include <stdlib.h>
main()
{
  int n;
  int *tab;
  ...
  tab = (int*)malloc(n * sizeof(int));
  ...
  free(tab);
}
```

Si on veut en plus que tous les éléments du tableau `tab` soient initialisés à zéro, on remplace l'allocation dynamique avec `malloc` par

`tab = (int*)calloc(n, sizeof(int));`

Les éléments de `tab` sont manipulés avec l'opérateur d'indexation `[]`, exactement comme pour les tableaux.

Les deux différences principales entre un tableau et un pointeur sont

- un pointeur doit toujours être initialisé, soit par une allocation dynamique, soit par affectation d'une expression adresse, par exemple `p = &i ;`
- un tableau n'est pas une *Lvalue* ; il ne peut donc pas figurer à gauche d'un opérateur d'affectation. En particulier, un tableau ne supporte pas l'arithmétique (on ne peut pas écrire `tab++;`).
-

11.1.7.2 Pointeurs et tableaux à plusieurs dimensions

- Un tableau à deux dimensions est, par définition, un tableau de tableaux. Il s'agit donc en fait d'un pointeur vers un pointeur. Considérons le tableau à deux dimensions défini par :

  `int tab[M][N];`

  `tab` est un pointeur, qui pointe vers un objet lui-même de type pointeur d'entier. tab a une valeur constante égale à l'adresse du premier élément du tableau, `&tab[0][0]`. De même `tab[i]`, pour `i` entre `0` et `M-1`, est un pointeur constant vers un objet de type entier, qui est le premier élément de la ligne d'indice `i`. `tab[i]` a donc une valeur constante qui est égale à `&tab[i][0]`.

- Exactement comme pour les tableaux à une dimension, les pointeurs de pointeurs ont de nombreux avantages sur les tableaux multi-dimensionnés.
- On déclare un pointeur qui pointe sur un objet de type `type *` (deux dimensions) de la même manière qu'un pointeur, c'est-à-dire

  ```
  type **nom-du-pointeur;
  ```

- De même un pointeur qui pointe sur un objet de type `type **` (équivalent à un tableau à 3 dimensions) se déclare par

  ```
  type ***nom-du-pointeur;
  ```

- **Exemple 102** *Par exemple, pour créer avec un pointeur de pointeur une matrice à* `k` *lignes et* `n` *colonnes à coefficients entiers, on écrit :*

  ```
  main()
  {
    int k, n;
    int **tab;
    tab = (int**)malloc(k * sizeof(int*));
    for (i = 0; i < k; i++)
      tab[i] = (int*)malloc(n * sizeof(int));
        ....
    for (i = 0; i < k; i++)
      free(tab[i]);
    free(tab);
  }
  ```

  La première allocation dynamique réserve pour l'objet pointé par `tab` *l'espace-mémoire correspondant à* `k` *pointeurs sur des entiers. Ces* `k` *pointeurs correspondent aux lignes de la matrice. Les allocations dynamiques suivantes réservent pour chaque pointeur* `tab[i]` *l'espace-mémoire nécessaire pour stocker* `n` *entiers.*

  Si on désire en plus que tous les éléments du tableau soient initialisés à zéro, il suffit de remplacer l'allocation dynamique dans la boucle `for` *par*

  ```
  tab[i] = (int*)calloc(n, sizeof(int));
  ```

- Contrairement aux tableaux à deux dimensions, on peut choisir des tailles différentes pour chacune des lignes `tab[i]`. Par exemple, si l'on veut que `tab[i]` contienne exactement `i+1` éléments, on écrit

  ```
  for (i = 0; i < k; i++)
    tab[i] = (int*)malloc((i + 1) * sizeof(int));
  ```

11.1.8 Pointeurs et chaînes de caractères

- chaîne de caractères : tableau à une dimension d'objets de type `char`, se terminant par le caractère nul `'\0'`. On peut donc manipuler toute chaîne de caractères à l'aide d'un pointeur sur un objet de type `char`. On peut faire subir à une chaîne définie par

  `char *chaine;`

- des affectations comme

  `chaine = "ceci est une chaine";`

- et toute opération valide sur les pointeurs, comme l'instruction

  `chaine++;`.

- Remarques

  Différence entre les deux déclarations suivantes :

  `char s[] = "Bonjour";`

  et

  `char *t =  "Bonjour";`

  – `s` est (l'adresse d') un tableau de caractères
  – `t` est un l'adresse d'un (tableau de) caractère(s)
  – `s[i]`, `*(s+i)`, `t[i]`, `*(t+i)` sont valides
  – `s` n'est pas une variable mais `*s`, `s[i]`, `*(s+i)` sont des variables ;
  – `t` est une variable mais `*t`, `t[i]` et `*(t+i)` n'en sont pas

  Exemple 103 *Ainsi, le programme suivant imprime le nombre de caractères d'une chaîne (sans compter le caractère nul).*

```
#include <stdio.h>
main()
{
  int i;
  char *chaine;

  chaine = "chaine de caracteres";
  for (i = 0; *chaine != '\0'; i++)
    chaine++;
  printf("nombre de caracteres = %d\n",i);
}
```

- La fonction donnant la longueur d'une chaîne de caractères, définie dans la librairie standard `string.h`, procède de manière identique. Il s'agit de la fonction `strlen` dont la syntaxe est

  `strlen(chaine);`

où `chaine` est un pointeur sur un objet de type `char`. Cette fonction renvoie un entier dont la valeur est égale à la longueur de la chaîne passée en argument (moins le caractère `'\0'`).

- **Exemple 104** *L'utilisation de pointeurs de caractère et non de tableaux permet par exemple de créer une chaîne correspondant à la concaténation de deux chaînes de caractères :*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main()
{
  int i;
  char *chaine1, *chaine2, *res, *p;
  chaine1 = "chaine ";
  chaine2 = "de caracteres";
  res = (char*)malloc((strlen(chaine1) + strlen(chaine2)) *
        sizeof(char));
  p = res;
  for (i = 0; i < strlen(chaine1); i++)
    *p++ = chaine1[i];
  for (i = 0; i < strlen(chaine2); i++)
    *p++ = chaine2[i];
  printf("%s\n",res);
}
```

On remarquera l'utilisation d'un pointeur intermédiaire `p` qui est indispensable dès que l'on fait des opérations de type incrémentation. En effet, si on avait incrémenté directement la valeur de `res`, on aurait évidemment « perdu »la référence sur le premier caractère de la chaîne. Par exemple,

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main()
{
  int i;
  char *chaine1, *chaine2, *res;
```

```
  chaine1 = "chaine ";
  chaine2 = "de caracteres";
  res = (char*)malloc((strlen(chaine1) + strlen(chaine2)) * sizeof(char));
  for (i = 0; i < strlen(chaine1); i++)
    *res++ = chaine1[i];
  for (i = 0; i < strlen(chaine2); i++)
    *res++ = chaine2[i];
  printf("\nnombre de caracteres de res = %d\n",strlen(res));
}
```

imprime la valeur 0, puisque res a été modifié au cours du programme et pointe maintenant sur le caractère nul.

- Copie de chaines :
  - `strcpy(char * s, const char * t)` de `<string.h>` effectue la copie de `t` vers `s`
  - Trois fonctions equivalentes

```
void copie1(char s[], char t[]){
     int i = 0 ;
     while (s[i] = s[i])
           i++;
}
void copie2(char s[], char t[]){
     int i = 0 ;
     while (*(s+i) = *(t+i))
           i++;
}
void copie3(char *s, char *t){
     while (*s++ = *t++)
           ;
  }
```

  - Le marqueur de fin de chaîne est important pour le test :
    `while (*s++ = *t++)` est la forme réduite de `((*s++ =*t++) !='\0')`
- Chaînes de caractères et pointeurs
  - `char chaine[50] = "Bonjour"`

    définit et initialise le tableau `chaine` ; l'utilisateur doit prévoir une taille suffisante (`8`, ici)

- `char chaine[] = "Bonjour"`
  idem sauf allocation du nombre minimum de caractères nécessaire par le compilateur (8, ici)
- `chaine` est un pointeur constant (comme tout tableau en C) de type `char *`
- Le contenu du tableau chaine est modifiable mais le pointeur chaine est constant
- `char * x = "Bonjour"`
  définit et initialise le pointeur sur la constante
- la valeur du pointeur `x` est modifiable, mais la chaîne sur laquelle il pointe est constante
- **Exemple 105** `char chaine[50] = "Bonjour" ;`

```
char *x = "Bonjour" ;
...
/* possible*/
if  (*chaine == 'B') .....chaine[3] = 't';
if(*x=='B').....x++;

/* illegal*/
chaine++;.....*(x+3) = 't';
.....
```

11.1.9 Pointeurs et structures

11.1.9.1 Pointeur sur une structure

- Contrairement aux tableaux, les objets de type structure en C sont des *Lvalues*. Ils possèdent une adresse, correspondant à l'adresse du premier élément du premier membre de la structure. On peut donc manipuler des pointeurs sur des structures.
  Exemple 106 *Ainsi, le programme suivant crée, à l'aide d'un pointeur, un tableau d'objets de type structure.*

```
#include <stdlib.h>
#include <stdio.h>
struct eleve
   {
     char nom[20];
     int date;
   };
typedef struct eleve *classe;
main()
```

```
{
  int n, i;
  classe tab;
  printf("nombre d'eleves de la classe = ");
  scanf("%d",&n);
  tab = (classe)malloc(n * sizeof(struct eleve));
  for (i =0 ; i < n; i++)
       {
         printf("\n saisie de l'eleve numero %d\n",i);
         printf("nom de l'eleve = ");
         scanf("%s",&tab[i].nom);
         printf("\n date de naissance JJMMAA = ");
         scanf("%d",&tab[i].date);
       }
  printf("\n Entrez un numero  ");
  scanf("%d",&i);
  printf("\n Eleve numero %d:",i);
  printf("\n nom = %s",tab[i].nom);
  printf("\n date de naissance = %d\n",tab[i].date);
  free(tab);
}
```

- Si `p` est un pointeur sur une structure, on peut accéder à un membre de la structure pointé par l'expression

  `(*p).membre`

- L'usage de parenthèses est ici indispensable car l'opérateur d'indirection `*` à une priorité plus élevée que l'opérateur de membre de structure. Cette notation peut être simplifiée grâce à l'opérateur pointeur de membre de structure, noté `->`. L'expression précédente est strictement équivalente à

  `p->membre`

  Ainsi, dans le programme précédent, on peut remplacer `tab[i].nom` et `tab[i].date` respectivement par `(tab + i)->nom` et `(tab + i)->date`.

11.2 Listes chaînées

11.2.1 Definition

Definition 107 *Une liste est une représentation de données par un ensemble L de valeurs qui possède les propriétés suivantes :*

- *l'existence d'un premier élément L_1, que nous appellerons* ***tête*** *de L (TETE(L))*
- *l'existence d'un dernier élément L_n appelé* ***queue*** *de liste*
- *l'existence pour tout L_i de L, à l'exception de la queue d'un élément L_{i+1} que nous appellerons* **successeur** *de L_i*

Les fonctions d'accès à une liste sont les suivantes :

- $Premier(L) = L_1$
- $Dernier(L) = L_n$
- $Successeur(L_i) = L_{i+1}$
- $Predeceseur(L_i) = L_{i-1}$
- $Successeur(L_n) = nil$
- $Predeceseur(L_1) = nil$

De plus, les opérations suivantes sont admises sur les listes

- Lecture d'un élément : il s'agit d'accéder à un élément de rang quelconque et d'en déduire toutes les informations le concernant
- Ecriture d'un élément : il s'agit de modifier l'information contenue dans un élément
- Suppression d'un élément : Pour que la structure reste cohérente, il faut mettre à jour la fonction Successeur à chaque fois qu'on ôte un élément
  - Si on veut supprimer L_i pour $1 \leq i \leq n$
    $Successeur(L_{i-1}) = L_{i+1}$
  - Si celui-ci est le premier de la liste (L_1) alors :
    $Premier(L) = Successeur(L_1)$
  - Si celui- ci est le dernier élément de la liste alors :
    $Dernier(L) = L_{n-1}$ et par conséquent $Successeur(L_{n-1}) = nul$
- Adjonction d'un nouvel élément L_s -
  - ajout au en milieu entre L_i et L_{i+1} , on fait dans l'ordre :
    $Successeur(L_s) = L_{i+1}$
    $Sucesseur(L_i) = L_s$
  - ajout en fin de liste :
    $Successeur(L_n) = Ls$
    $Successeur(L_s) = Nil$

– ajout au début :
$Successeur(Ls) = L1$
$Premier(L) = (Ls)$

Chaque élément de la liste est contenu dans une *cellule* ou *maillon*. La cellule contient deux champs deux champs principaux :

– un champ appelé *clé* contenant une information sur l'élément en question
– un champ *suivant* (*pointeur*) contenant l'adresse de la cellulle suivante de la liste chaînée

Si le champ suivant d'un élément vaut NIL, cet élément n'a pas de sucesseur et est donc le dernier élément ou la **queue** de liste. Le premier élément est appelé la **tête** de la liste. Une liste L est manipulée via un pointeur vers son premier élément que l'on notera $Tete(L)$. Si $Tete(L)$ vaut NIL alors la liste L est vide.

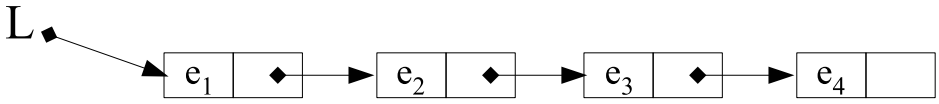


FIGURE 11.1 – Exemple de liste chaînée.

Une liste est une structure données similaire à un tableau en ce sens qu'il s'agit d'un arrangement linéaire d'éléments, mais elle en diffère par le fait que qu'on accède aux éléménts non pas à l'aide d'un indice, mais en parcourant tous les prédecesseurs, et que sa taille n'est pas fixée initialement, et peut(théorie) devenir aussi grande que l'on veut à l'éxecution. On remarquera toutefois que les listes ne présentent pas que des avantages sur les tableaux : l'accès au k-ieme élément d'une liste nécessite un parcours le long de k nœuds (ou $k-1$ appels à la fonction *Queue* tandis que que lire t[k] ne nécessite qu'une seule opération et s'exécute en un temps constant.

Une liste peut prendre plusieurs formes :

– **Liste doublement chaînée** : en plus du champ *suivant*, chaque élément contient un champ *precedant* qui est un pointeur précédent dans la liste. Si le champ *precedent* d'un élément vaut *nil*, cet élément n'a pas de prédecesseur et est donc le premier élément ou la tête de liste. Une liste qui n'est pas doublement chaînée est dite **simplement chaînée**

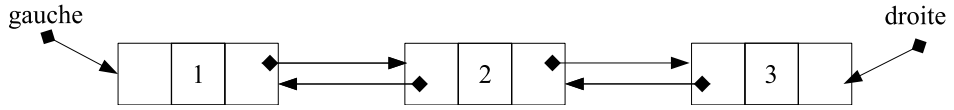


FIGURE 11.2 – représentation graphique de la liste doublement chaînée.

- **Triée ou non triée** : suivant que l'ordre linéaire des éléments dans la liste correspond ou non à l'ordre linéaire des clés de ces éléments
- **circulaire** : si le champ précécesseur de la tête de la liste pointe sur la queue, et si le champ successeur de la queue pointe sur la tête. La liste est alors vue comme un anneau.

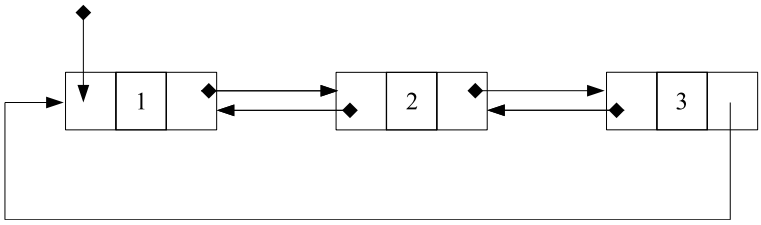


FIGURE 11.3 – Représentation graphique de la liste circulaire.

Le langage Pascal permet de représenter des une liste chaînée à l'aide de pointeurs : les cellules sont des enregistrements (record) dont un des champs contient l'adresse de la cellule suivante. L'adresse dce la première cellule est contenue dans une variable de tête de liste. Les déclarations correspondantes sont les suivantes :

```
type
   Liste = ^Cellule ;
   Cellule =record
               cont : Element ;
               suiv :liste ;
            end ;
var a : Liste ;
```

En C, on peut définir le type Liste des listes d'éléments de type int comme suit, à l'aide de pointeurs :

```
struct cellule{
int cont
struct cellule * suiv
}
typedef struct cellule * Liste
```

On parle de liste chaînée pour faire référence à ce type d'implémentaion basé sur les pointeurs. Cette notion correspond intuitivement à la représentation de la figure 11.13 où chaque flèche représente un pointeur. Nous venons de définir un type `Liste` qui est formellement un

pointeur de `struct cellule`. Il est important de bien comprendre que lorsque les cellules qui constituent une liste sont bien chaînées entre elles, il suffit d'aoir un pointeur vers la première cellule pour être ensuite capable d'accéder à tous les éléments. La liste vide est simplement codée par par un pointeur vers la constante `NULL`. De plus, le dernier pointeur sera lui aussi affecté à cette valeur particulière afin de bien indiquer la fin d'une liste liste

11.2.2 Algorithmes de manipulation de listes chaînées

11.2.2.1 Création d'une liste vide

☛En pseudo-code

```
Procedure viderListe(var  L: Liste);
Debut
    L:=nil
Fin
```

```
void viderListe(Liste *L)
{
    *L=NULL;
}
```

11.2.2.2 Test de la liste vide

☛En pseudo-code

```
Fonction estVide( L: Liste):booleen;
Debut
    estVide:=(L=nil);
Fin
```

☛En langage C

```
int estVide(Liste L)
{
    return (L==NULL);
}
```

11.2.2.3 Tête de liste

```
int tete(Liste L)
{
    if estVide(L)
        {
            printf("erreur liste vide\n")
            exit(-1);
        }
    else
    reurn(L->cont);
}
```

11.2.2.4 Ajout en tête

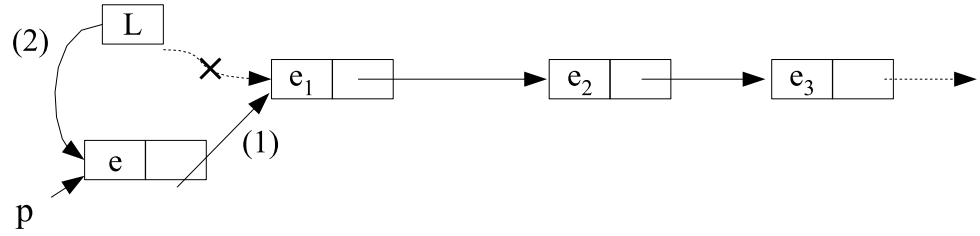


FIGURE 11.4 – Ajout en tête de liste.

La procédure ajoutTete insère l'élément e en tête de liste. Ce choix permet de limiter le nombre d'opération. Notez que la liste est passé par adresse.

☛EN PSEUDO-CODE

```
Procedure  ajoutTete(e : Element; var  L: Liste);
var  nouv:Liste;
Debut
    Allouer(nouv)
    nouv^.cont:=e;
    nouv^.suiv:=L;
    L:=nouv;
Fin
```

☛EN LANGAGE C

```
void  ajoutTete(Element e , Liste *L)
{
  Liste nouv;
  nouv=(Liste) malloc(sizeof(struct cellule));
  nouv->cont:=e;
  nouv->suiv:=*L;
 *L:=nouv;
}
```

11.2.2.5 Ajout en tête (version fonction)

La fonction cons ajoute un élément en tête de liste comme la procédure ajouTete définie plus haut.

☛En pseudo-code

```
Fonction cons(e: Element; L:liste):Liste;
var nouv : Liste;
Debut
    Allouer(nouv);
    nouv^.cont:=e
    nouv^.suiv:=L
    cons:=nouv;
Fin
```

☛En langage C

```
Liste cons(Element e, Liste L)
{
   Liste nouv;
   nouv=(Liste) malloc(sizeof(struct cellule));
   nouv->cont=e;
   nouv->suiv=L
   cons=nouv;
}
```

11.2.2.6 Ajout en queue

La procédure ajoutQueue insère l'élément e en queue de liste, après la queue actuelle. La variable `q` est modifiée itérativement par `q :=q^.suiv` (`q->suiv` en C) de façon à parcourir

tous les éléments jusqu'à ce que l'on arrive sur **le dernier élément** de la liste (`q^.suiv=nil`) ou `q->suiv==NULL` en C

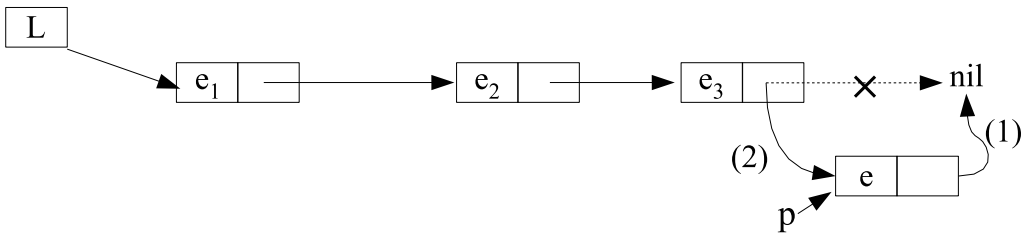


FIGURE 11.5 – Ajout en queue.

☛EN PSEUDO-CODE

```
Procedure  ajoutQueue(e : Element; var L: Liste);
var  nouv:Liste;
Debut
    Allouer(p)
    nouv^.cont:=e;
    SI  estVide(L)  ALORS
                        L:=nouv;
                    SINON
                        Debut
                            q:=L;
                            TANTQUE q^suiv<>nil FAIRE
                                    q:=q^.suiv;}

                            q^.suiv:=nouv;

                       Fin
Fin;
```

☛EN LANGAGE C

```
void  ajoutQueue(Element e,  Liste *L)
{
    Liste nouv;
    nouv=(Liste) malloc(sizeof(struct cellule));
```

```
    nouv->cont:=e;
    if  estVide(L)
           *L:=nouv;
    else
       {
          q:=*L;
          while (q->suiv!=NULL)
                 q:=q->suiv;
          q->suiv=nouv;
       }
}
```

11.2.2.7 Ajout au milieu

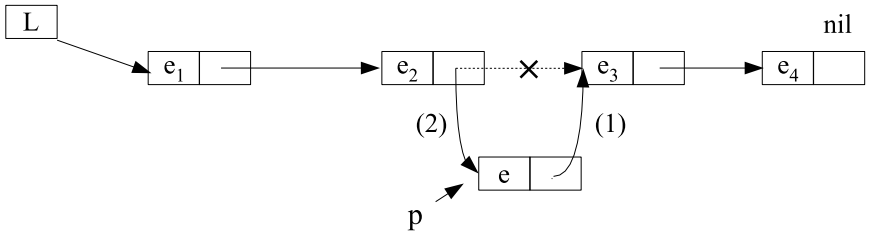


FIGURE 11.6 – Ajout au milieu.

Ici, nous ajoutons l'élément à sa bonne place dans une liste déjà triée par ordre croissant.

☛EN PSEUDO-CODE

```
Procedure  ajoutMilieu(e : Element; var L: Liste)
var  p,courant:Liste;
Debut
    Allouer(p)
    pˆ.cont:=e;
    SI  estVide(L)  ALORS
                        L:=p;
                    SINON
                        Debut
                            q:=L;
                            TANTQUE q<>nil ET qˆ.cont < e FAIRE
                                    Debut
```

```
                                courant:=q
                                q:=qˆ.suiv;
                            Fin
                        pˆsuiv:=q;
                        courantˆ.suiv:=p;

                    Fin

Fin;
```

☛En langage C

```
void  ajoutMilieu(Element e , Liste *L)
{
    Liste nouv,courant,q;
    nouv=(Liste) malloc(sizeof(struct cellule));
    nouv->cont:=e;
    if  estVide(L)
          *L:=nouv;
    else
       {
          q:=*L;
          while (q!=NULL) && (q->cont < e)
                   {
                     courant:=q;
                     q:=qˆ.suiv;
                   }
          nouv->suiv:=q;
          courant->suiv:=nouv;
       }

};
```

11.2.2.8 Concaténation de deux listes

Maintenant, nous allons donner une fonction `concatL` qui consiste à mettre deux listes bout à bout pour en construire une troisième dont la longueur est égale à la somme des longueur des deux autres. Les deux listes ne sont pas modifiées

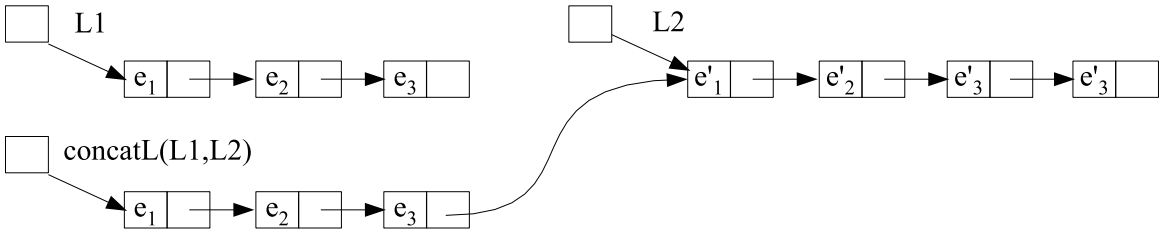


FIGURE 11.7 – Concaténation de deux listes par `concatL`.

☛EN PSEUDO-CODE

```
Fonction concatL(L1: Liste; L2:Liste):Liste;
Debut
    SI estVide(L1) ALORS
                    concatL:=L2
                SINON
                    concatL:=cons(L1^.cont,concatL(L1^.suiv,L2));
Fin
```

☛EN LANGAGE C

```
Liste concatL(Liste L1, Liste L2):;
{
    if estVide(L1)
           concatL:=L2
    else
           return(cons(L1->cont,concatL(L1->suiv,L2)))
}
```

Exercice 11.2.1 *Donner uen procédure* `concatL2` *qui fait la même chose que la fonction* `cons` *mais cette fois ci en transformant la première liste pour donner le résultat*

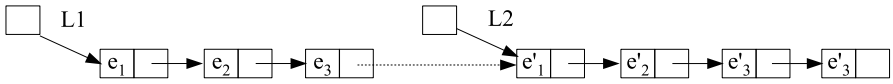


FIGURE 11.8 – Concaténation de deux listes par `concatL2`.

11.2.2.9 Insertion d'un élémént à la k-ieme place

☛En pseudo-code

```
Fonction ajoutek(e:Element ; k :entier; L: Liste): Liste;
Debut
    SI k=0 ALORS
                ajoutek:=cons(e,L)
           SINON
                ajoutek:=cons(L^.cont,ajoutek(e,k-1,L^.suiv)};
Fin
```

☛En langage C

```
Liste ajoutek(Element e, int k, Liste L);
{
    if (k==0)
         return(cons(e,L))
    else
         return(cons(L->cont,ajoutek(e,k-1,L->suiv)));
}
```

Exercice 11.2.2 *Donnez une version itérative de la fonction ajoutek*

11.2.2.10 Recherche d'un élément

La fonction `existeIter` effectue un parcours itératif de la liste pour rechercher l'élément e dans la liste `L`. La variable `q` est modifiée itérativement par `q :=q^.suiv` de façon à parcourir tous les éléments jusqu'à ce que l'on arrive **à la fin** de la liste (`q^.suiv=nil`)

☛En pseudo-code

```
Fonction existeIter(e:Element; L: Liste):booleen;
var trouve : booleen;
Debut
    trouve:=faux;
    TANTQUE (L<>nil) ET (NON trouve) FAIRE
        Debut
            trouve:=(L^.cont=e);
```

```
            L:=L^.suiv
        Fin
    existeIter:=trouve;
Fin
```

☛En langage C

```
int existeIter(Element e, Liste L)
{
    int  trouve;
    trouve:=0;
    while (L!=NULL) && (!trouve)
        {
            trouve=(L->cont==e);
            L:=L->suiv
        }
    return(trouve);
}
```

La fonction `existeRec` constitue une version récursive plus simple de la recherche.

☛En pseudo-code

```
Fonction existeRec(e:Element; L: Liste):booleen;
Debut
    SI L=nil  ALORS
                existeRec:=faux;
              SINON
                SI L^.cont=e; ALORS
                                existeRec:=true
                              SINON
                                existeRec:=existeRec(e,L^.suiv)
Fin
```

☛En langage C

```
int existeRec(Element e, Liste L )
{
    if (L==NULL)
```

```
          return(0);
    else
       if (L->cont==e)
          return(1)
       else
          return(existeRec(e,L->suiv);
}
```

NB : La récursivité est beaucoup adpatée aux listes puisque le type des listes vérifie l'équation suivante

$$Liste = \{Liste_vide\} \biguplus element \times Liste \tag{11.1}$$

où $\biguplus$ est l'union disjointe et $\times$ le produit cartésien

La fonction `existeRec` peut être réécrite en `existeRec2` comme suit :

☛En pseudo-code

```
Fonction existeRec2(e:Element; L: Liste):booleen;
Debut
    SI L=nil  ALORS
                existeRec:=faux;
              SINON
                existeRec2:=(L^.cont=e)||existeRec2(e,L^.suiv)
Fin
```

☛En langage C

```
int existeRec2(Element e, Liste L )
{
    if  (L==NULL)
                return (0);
              else
                return((L->cont==e)||existeRec2(e,L->suiv));
}
```

11.2.2.11 Impression des éléments d'une liste

version itérative

☛En pseudo-code

```
Procedure afficheListe(L : Liste);
var p : Liste;
Debut
    TANTQUE (p<>=nil) FAIRE
        Debut
            Ecrire( p^.cont);
            p:=p^.suiv;
        Fin
Fin
```

☛En langage C

```
void afficheListe(Liste L)
{
    Liste p=L;
    while(p!=NULL)
        {
            printf("%d", p->cont);
            p=p->suiv;
            printf("\n");

        }
}
```

version récursive

☛En pseudo-code

```
Procedure afficheListeRec(L : Liste L);
Debut
    Si (estVide(L)) Alors
                        AllerALigne
                    Sinon
                        Debut
                            Ecrire(L^.cont);
                            afficheListeRec(L^.suiv);
                        Fin
Fin
```

☛En langage C

```
void afficheListeRec(Liste L)
{
    if (!estVide(L))
        printf("\n");
    else
        {
            printf("%d",L->cont);
            afficheListeRec(L->suiv);
        }

}
```

11.2.2.12 Nombre d'élements d'une liste

La version récursive est la suivante :

☛En pseudo-code

```
Fonction nbElementsRec( L: Liste):entier;
Debut
    SI L=nil  ALORS
                nbElementsRec:=0;
              SINON
                nbElementsRec:=1+ nbElementsRec(L^.suiv)
Fin
```

☛En langage C

```
int nbElementsRec(Liste L);
{
    if  (L==NULL)
          return(0);
    else
          return(1+ nbElementsRec(L->suiv));
}
```

Ceci est plus simple que la version itérative suivante :

☛En pseudo-code

```
Fonction nbElementsRIter(L: Liste):integer;
var nb : integer;
Debut
    nb:=0;
    TANTQUE (L<>nil)  FAIRE
        Debut
            nb:=nb+1L:=L^.suiv;Fin
            nbElementsRIter:=nb;
Fin
```

☛En langage C

```
int nbElementsRIter(Liste L);
{
    int nb;
    nb=0;
    while (L!=NULL)
        {
            nb++;
            L=L->suiv
        }
    return(nb);
Fin
```

11.2.2.13 Suppression d'un élément

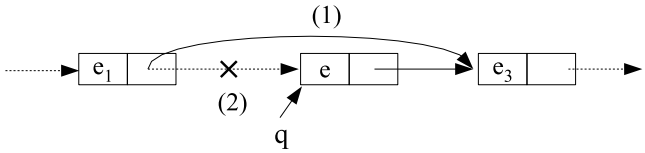


FIGURE 11.9 – Suppresion de la cellule contenant e (en milieu).

Pour supprimer la cellule contenant `e`, il faut modifier la valeur du champ `suiv` contenu dans le prédecesseur de `e` : le sucesseur du prédecesseur de `e` devient le successeur de `e` ($Sucesseur(Predecesseur(e) \leftarrow Sucesseur(e)$). Le traitement est différent si l'élement à supprimer est le premier élément de la liste.

☛En pseudo-code

```
Fonction suppressionRec(e:Element;var L: Liste);
Var q : Liste;
Debut
   SI non estVide(L) ALORS
                       SI (L^cont=e) ALORS
                                       Debut
                                         q:=L;
                                         L:=L^.suiv;
                                         Liberer(q);
                                      Fin
                                     SINON
                                       supprimerRec(e,L^.suiv);
Fin;
```

☛En langage C

```
void suppressionRec(Element e,Liste L);
{
 Liste q;
 if non (!estVide(L))
             if (L^cont==e)
                    {
                      q=L;
                      L=L->suiv;
                      free(q);
                    }
             else
                supprimerRec(e,&L);
};
```

Une version itérative possible est la suivante :

☛En pseudo-code

```
Procedure suppressionIter(e:Element;var L: Liste);
var succ,cour : Liste;
Debut
```

```
SI L<>nil ALORS
            SI L^.cont=e ALORS
                          Debut
                            q:=L;
                            L:=L^.suiv;
                            Liberer(q);
                          Fin
                         SINON
                           Debut
                             cour:=L;
                             succ:=L^.suiv;
                             TANTQUE succ<>nil FAIRE
                                 SI succ^.cont<>e ALORS
                                                  Debut
                                                   cour:=succ;
                                                   succ:=succ^.suiv;
                                                  Fin;
                                                 SINON
                                                   Debut
                                                     cour^.suiv:=succ^.suiv;
                                                     Liberer(succ);
                                                     succ:=nil;
                                                    Fin;
                           Fin
Fin
```

☛En langage C

```
void suppressionIter(Element e, Liste L)
{
    Liste succ,cour,q ;
    if  (!estVide(L))
          if (L->cont==e)
                {  q=L;
                   L=L->suiv;
                   free(q);
                }
```

```
          else
             {
               cour=L;
               succ=L->suiv;
               while (succ!=NULL)
                   {
                       if (succ->cont!=e)
                           {
                              cour=succ;
                              succ=succ->suiv;
                           }
                       else
                         {
                           cour->suiv=succ->suiv;
                           free(succ);
                           succ=nil;
                         }
                   }
}
```

Exercice 11.2.3 *Ecrire une procédure qui supprime un élément e dans une liste L triée*

Exercice 11.2.4 *Ecrire une procédure qui construit la liste des nombres premiers inférieurs ou égaux à un entier n donné en utilisant la méthode clasique du crible d'ératosthène.*

11.2.2.14 Image miroir d'une liste

La procédure de calcul de l'image miroir d'une liste L consiste à construire une liste dans laquelle les éléments de L sont rencontrés dans l'ordre inverse de ceux de L.

☛En pseudo-code

```
Fonction miroirRec(L: Liste):Liste;
Debut
    SI estVide(L) ALORS
                    miroirRec:=nil
                  SINON
                    miroirRec:=concatL(miroirRec(L^.suiv),cons(L^.cont,nil));
Fin
```

☛En langage C

```
Liste miroirRec(Liste L );
{
    if estVide(L)
          return(NULL)
    else
          return(concatL(miroirRec(L->suiv),cons(L->cont,nil)));
}
```

Cette fonction inverse une liste en $\Theta(n^2)$ opérations si n est la longueur de la liste car on fait de l'orde de n appels récursifs à `miroirRec` qui exécute à chaque fois une concaténation de liste. La fonction suivante permet d'inverser une liste en seulement $\Theta(n)$ opérations

☛En pseudo-code

```
Fonction miroirRec2(L: Liste;Acc : Liste):Liste;
Debut
    SI estVide(L) ALORS
                    miroirRec2:=Acc
                  SINON
                    miroirRec2:=miroirRec2(L^.suiv,cons(L.cont,Acc));
Fin
```

☛En langage C

```
Liste miroirRec2(Liste L ; Liste Acc  );
{
    if estVide(L)
         return(Acc);
    else
         return((L^.suiv,cons(L.cont,Acc)));
}
```

Une version itérative pour l'image miroir est la suivante :

```
Procedure miroirIt(var L : Liste) ;
var p,q : Liste ;
Debut
```

```
   p=nil ;
   TANTQUE (L<>nil) FAIRE
            Debut
               q :=L^.suiv ;
               L^.suiv :=p ;
               p :=L
               L :=q;
            fin
   L :=p;
Fin
```

Exercice 11.2.5 *Ecrire une fonction récursive qui enlève toutes les occurences d'un élément dans une liste*

11.2.2.15 Suppression de la totalité d'une liste

☛En pseudo-code

```
Procedure libereListe(var L: Liste);
Debut
    SI L<>nil  ALORS
        Debut
            libereListe(L^.suiv);
            liberer(L);
            L:=nil);
        Fin
Fin
```

☛En langage C

```
void libereListe(Liste *L );
{
    SI *L<>NULL
        {
            libereListe((*L)->suiv);
            free(*L);
            *L=NULL;
```

```
        }
}
```

11.2.2.16 Suppression de toutes les occurences d'un élément dans une liste

```
Liste supprimeTOUTES(Element e, Liste L)
{
    if (L==NULL)
        return NULL;
    else
        if(tete(L)==e)
            return(supprimeTOUTES(e,L->suiv))
        else
            cons(tete(L),supprimeTOUTES(e,L->suiv));
}
```

La fonction supprimeTOUTES a l'avantage d'être simple à écrire, mais elle ne gère pas bien l'espace mémoire puisque la fonction cons crée beaucoup de cellules qui ne sont pas ensuire libérées. La fonction suivante montre comment libérer les cellules qui ne servent plus afin de ne pas saturer la mémoire de données inutiles

```
void supprimeTOUTES(Element e, Liste L)
{
    if (L==NULL)
        return NULL;
    else
        if(tete(L)==e)
            return(supprimeTOUTES(e,L->suiv))
        else
            cons(tete(L),supprimeTOUTES(e,L->suiv));
}

void supprimeTOUTES2(Element e, Liste *L)
{
    Liste P;
    if(*L!=NULL)
        {
```

```
            if ((*L)->cont==e)
                {
                    P=*L;
                    *L=(*L)->suiv;
                    free(P);
                    supprimeTOUTES2(e,L);
                }
            else
                supprimeTOUTES2(e,&((*L)->suiv));
        }

}
```

11.3 Structures auxilaires : Piles, files d'attentes

Ces structures de données que l'on qualifie aussi de structures intermédiaires car elles s'utilisent souvent en conjonction avec d'autres, sont d'une grande importance dans de nombreux domaines tels que :

- systèmes d'exploitation,
- évaluation d'expressions (arithmétiques, logiques, etc.),
- bases de données (interrogation, mise-à-jour)

Ces structures se caractérisent par un comportement particulier en ce qui concerne l'insertion et l'extraction d'un élément. Ainsi, pour une pile, on insère et on extrait les éléments à la même extrémité de la suite linéaire. Pour la file d'attente, l'insertion se fait à un bout et l'extraction à l'autre.

11.3.1 Piles

11.3.1.1 Eléments de base

Definition 108 *Une pile est une structure de données mettant en oeuvre le principe « dernier entré, premier sorti »(LIFO : Last-In, First-Out en anglais).*

Le *Traitement des structures emboîtées* contitue un exemple d'utilisation des piles (par exemple, analyse d'un programme : boucles imbriquées, procédures locales ; parcours d'arbres, etc.) pour lesquelles il faut que les sous-structures soient traitées avant la structure qui les contient. Si on traite une structure de niveau n et que l'on rencontre une sous-structure S,

on interrompt le traitement au niveau n en plaçant les informations indispensables sur la pile qui nous permettront de reprendre le traitement de n après avoir terminé celui de S. Ainsi à tout moment la pile contient, dans l'ordre inverse, les informations relatives aux niveaux momentanément interrompus.

On peut imaginer une pile comme une boite dans laquelle on place des objets et de laquelle on les retire dans un ordre inverse dans lequel on les a mis : les objets sont les uns au dessus des autres et on ne peut accéder qu'a l'objet situé au *sommet* de la pile.

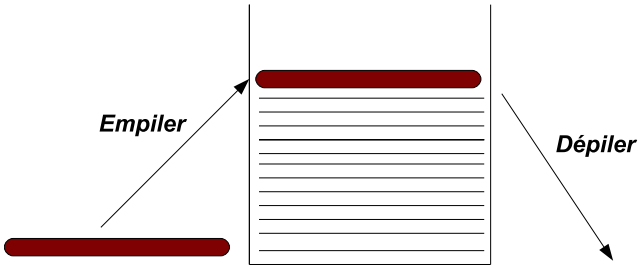


FIGURE 11.10 – Représentation d'une Pile

Description en terme de type abstrait :

- ✔ le rôle d'une pile est de permettre la mise en attente d'informations dans le but de les récupérer plus tard. La première information à être récupérée est celle qui a été mise en attente en dernier (la plus récente).
- ✔ la déclaration d'une pile doit spécifier le type de base des éléments qu'elle pourra contenir.
- ✔ une pile supporte les opérations suivantes :
  - $vide(P)$ est égal à vrai si la pile est vide et faux sinon
  - $empiler(e, P)$ son résultat est la pile obtenue à partir de la pile P en insérant l'élement e au sommet la pile.
  - $sommet(P)$ cette fonction permet de récupérer l'élemnt figurant au sommet d'une pile non vide
  - $depiler(e, P)$ son résultat est la pile obtenue à partir de P en supprimant l'élément qui se situe en son sommet

Les piles sont tout à fait isomorphes aux listes, si ce n'est qu'on a traditionnellement tendance à les imaginer verticales, et qu'on donne aux opérations d'insertion, de lecture de la tête et de suppression d'un élément les noms EMPILER, SOMMET, DÉPILER. Leur codage par des listes chaînées s'effectue de manière immédiate.

11.3.1.2 Pile : représentation par tableau

Il est facile d'implanter une pile au moyen d 'un tableau qui contient les éléments et un indice qui indiquera la position du sommet de la pile. La seule difficulté dans ce cas est la gestion des débordements de interviennent quand on tente d'éffectuer l'opération DEPILER sur une pile vide et l'opération EMPILER sur un tableau codant la pile qui est déjà pleine.

```
const tailleMax = 200;
type TPile = Enregistrement
                taille:integer;
                contenu:Tableau[1..tailleMax] of Element;
            Fin;
var P: TPile;

Procedure ViderPile (var P: TPile);
debut
    P.taille:=0;
Fin;

Function estVideP (var P: TPile): boolean;
debut
    estVidePide := P.taille= 0;
Fin;

Function depiler (var P: TPile): Element;
debut
    Si estVide(P) Alors
                    Erreur('Pile Vide')
                Sinon
                    Debut
                        depiler := P.contenu[P.taille];
                        P.taille:=P.taille-1;
                    Fin
Fin;

Procedure empiler (e: Element; var P: TPile)};
debut
```

```
    Si (P.taille=tailleMax) Alors
                                Erreur('Pile Pleine')
                            Sinon
                                Debut
                                    P.taille := P.taille  +1;
                                    P.contenu[P.Taille] := e;
                                Fin;
Fin;

Procedure depiler (var P:TPile);
debut
    P.taille := P.taille -1;
Fin;
```

Exercice 11.3.1 *Donner les primitives de gestion d'une pile à l'aide d'une liste chaînée.*

11.3.2 Files

11.3.2.1 Eléments de base

Definition 109 *Une file est une structure de données mettant en oeuvre le principe « premier entré, premier sorti »(FIFO : First-In, First-Out en anglais).*

Dans une file d'attente l'ordre d'arrivée des éléments est préservé. Les éléments sont systématiquement ajoutés en queue et supprimés en tête.La valeur d'une file est par convention l'élément de tête.
Exemples d'utilisation

- mise en attente des programmes pour exécution
- accès à un dispositif périphérique (p. ex. imprimante)

Description en terme de type abstrait :

✔ le rôle d'une file d'attente est de permettre la mise en attente d'informations dans le but de les récupérer plus tard. La première information à être récupérée est celle qui a été mise en attente en premier (la plus ancienne).

✔ la déclaration d'une file d'attente doit spécifier le type de base des éléments qu'elle pourra contenir.

✔ Comme pour les piles on définit les opérations :

- $vide(F)$ est égal à vrai si la file est vide et faux sinon
- $emfiler(e, P)$ son résultat est la file obtenue à partir de la file F en ajoutant l'élement e en tête de file.

- $valeur(P)$ cette fonction permet de récupérer l'élement figurant tête de file.
- $defiler(e, P)$ son résultat est la file obtenue à partir de F en supprimant l'élément tête de file.

De la même façon que pour la pile, une file peut être impmémenté en utilisant un tableau ou une liste chaînée. Seulement dans ce cas, il est nécesaire de connaitre le début et la fin de la file qui seront respectivement repérés par les variables *debut* et fin. On note que :

- la file d'attente est vide si et seulement si $debut = fin$.
- lorsqu'un élément arrive en queue de file on incrémente la variable fin
- lorsque l'élément tête de file est traitée, la variable *debut* est incrémentée si la file n'est pas vide

11.3.2.2 File d'attente : représentation par liste chaînée gardée

Dans la suite on illustre l'implémentation d'une file grâce à une liste chaînée gardée (liste dans laquelle on connait à la fois l'adresse du premier et du dernier élément). Dans une liste chaînée gardée, on utilise une cellule (appelée garde) placée au début et n'ayant pas d'information relevante dans le champ `cont` ; l'adresse de la vraie premiere cellule se trouve vans le champ `suiv` de cette cellule.
Ceci permet e ne pas faire de traitement spécial dans le cas où la liste est vide ou du premier élément, car la liste vide contient au moins la garde

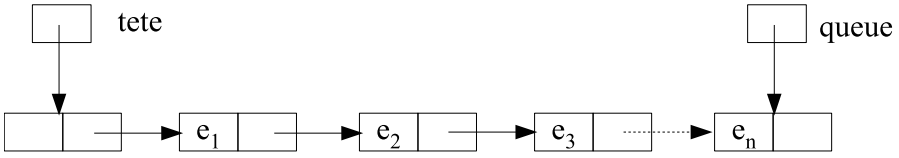


FIGURE 11.11 – file d'attente implémentée par une liste chaînée gardée.

```
type Liste = ^Cellule;
               Cellule = Enregistrement
                           cont : Element;
                           suiv:Liste;
                         fin;

TFile = Enregistrement
            tete : Liste;
            queue : Liste;
```

```
        fin;

Procedure ViderFile (var F: TFile);
Debut
    Allouer(F.tete);
    F.queue:=F.tete;
fin;

Fonction Successeur (var L: Liste):Liste;
Debut
    Successeur:= L^.suiv;
fin;

Fonction estVideFile (var F: TFile):boolean;
Debut
    estVideFile:=F.tete:=F.queue;
fin;

Fonction valeurF (var F: TFile):Element;
var q : Liste;
Debut
    q:=Successeur(F.tete);
    valeurF:=q^.cont;
fin;

Procedure enfiler (e: Element; var F: TFile);
var q : Liste;
Debut
    Allouer(q);
    q^.cont;=e
    q^.suiv;=nil
    F.queue^.suiv;=q
    F.queue:=q;
fin;

Procedure defiler (var F: TFile);
```

```
var q : Liste;
Debut
    SI NON estVideFile(F) ALORS
                              Debut
                                  q:=F.tete
                                  F.tete:=Sucesseur(F.tete);
                                  Liberer(q);
                              Fin
fin;
```

11.3.2.3 File d'attente : représentation par tableau

```
const tailleMax = 200;
type  Element = ...;
      FileDAttente = Enregistrement
                          contenu : Tableau[1..Max] of Element;
                          taille  : 0..tailleMax;
                       fin; { FileDAttente }
var F: FileDAttente;

{initialisation}

F.taille := 0;


function estVide (F: FileDAttente): booleen;
debut
    estVide := F.NbElements=0;
fin;

procedure defiler(var F: FileDAttente; var Valeur: Element);
var i: entier;
debur
    Si  estVide(F) Alors
                       Erreur('file vide')
                   Sinon
                     debut
```

```
                        Valeur := F.contenu[1];
                        Pour i := 2 A F.taille Faire
                                F.contenu[i-1] := F.contenu[i];
                        F.taille  := F.taille -1;
                    fin;
fin;

procedure enfiler(var F: FileDAttente; Valeur: Element);
debut
  Si  F.taille= Max Alors
                      Erreur('file pleine')
                Sinon
                      debut
                         F.taille := F.taille+1;
                         F.contenu[taille] := Valeur;
                      fin;
fin;
```

11.3.2.4 File circulaire

Comme nous pouvons le remarquer, l'organisation la plus simple d'une file d'attente à l'intérieur d'un tableau n'est pas très efficace. C'est pourquoi une solution de file circulaire est proposée dans ce paragraphe.

```
const tailleMax = 200;
type  Element = ...;
      FileCirculaire = Enregistrement
                            contenu : tableau[0..tailleMax] of Element;
                            IndexIn, IndexOut: 0..tailleMax;
                            Plein : booleen;
                       fin;
var FC : FileCirculaire;

FC.IndexIn := 0;
FC.IndexOut := 0;
FC.Plein := false;
```

```
function estVide(FC: FileCirculaire): booleen;
debut
      estVide := (FC.IndexIn = FC.IndexOut) ET (NON FC.Plein);
fin;

procedure enfiler( var FC : FileCirculaire; Valeur : Objet);
debut
    Si FC.Plein Alors
                   Erreur('buffer plein')
               Sinon
                   Debut
                      FC.contenu[FC.IndexIn] := Valeur;
                      FC.IndexIn:=(FC.IndexIn+1) mod (tailleMax+1);
                      FC.Plein := (FC.IndexIn=FC.IndexOut);
                   Fin;
fin;

procedure defiler( var FC : FileCirculaire; var Valeur : Objet);
debut
   Si estVide(FC) Alors
                     Erreur('file vide')
                Sinon
                  Debut
                     Valeur := FC.contenu[FC.IndexOut];
                     FC.IndexOut := (FC.IndexOut+1) mod (tailleMax+1);
                     FC.Plein := false;
                  fin;
fin;
```

11.4 Application :evaluation des expressions arithmétiques postfixées

Dans cette section, nous illustrons par un exemple concret l'utilisation des piles par un programme d'évaluation d'expressions arithmétiques fournies en notation *préfixée* et ne contenant que des entiers naturels et les opérateurs d'addition, de soustraction et de multiplication

(noté `op`. Sur certaines machines à calculer , les calculs s'effectuent en mettant l'opérateur après les nombres sur lesquels on effectue l'opération. On a alors une notation dite *postfixée*. Dans certains langages de programmation, les expressions sont écrites de façon *préfixée*, c'est à dire c'est-à-dire que l'opérateur précède cette fois les deux opérandes. La notation est *infixée* quand l'opérateur est entre les deux opérandes.

Une expression préfixée est ou bien un nombre entier naturel ou bien de l'une des deux formes : $+ \quad e_1 \quad e_2 \qquad - \quad e_1 \quad e_2 \qquad * \quad e_1 \quad e_2$

Ainsi donc `1+2` sera noté "`+ 1 2` "et `3 x (4 + 5)` sera écrit "`x 3 + 4`". On notera que la notation préfixée supprime toute ambiguité concernaNt les priorités entre opérateurs et que par conséquent les paranthèses sont inutiles. On vérifie facilement que les suites de symboles suivant sont des expressions préfixées

```
32
* 4 5
+ 12 8
+ * 2 3 + 12 8
```

Leur évaluation donne respectivement `32`, `20`, `20`, `26`

Afin d'évaluer un expression, nous allons stocker les différents symboles qui la composent dans une pile. l'algorithme d'évaluation est récursif ; si le sommet de la pile contient un entier alors on le retire de la pile et le résultat de l'évaluation est cet entier. Si le sommet de la pile est un opérateur, on le dépile, on évalue une première fois la partie restante (ce qui aura pour effet d'évaluer et de retirer de la pile tout le premier terme), on refait de même pour évaleur et dépiler le second terme et enfin on renvoie le résultat.

prenons par exemple, l'expression $(2 + 3) \times 4 - 4 \times 2$, notée `- * + 2 3 4 * 4 2`.

Si on empile ces symboles, on a le premier signe de soustraction au sommet de la pile. On le retire et on évalue récursivement la pile deux fois de suite. Si le programme fonctionne corresctement, il va une première fois opérer sur la pile `* + 2 3 4 * 4 2`, en extraire la première sous expression bien formée (`* + 2 3 4`) et renvoyer son évaluation, i.e. 20. On va ensuite de nouveau évaluer et dépiler une expression complète, ici le reste de la pile, i.e. `* 4 2` . Le résultat 8 est renvoyé. Le programme calcule donc finalement $20 - 8 = 12$

Le programme suivant implémente cet algorithme. La pile est constituée de cellule contenant un entier `typ` indiquant si l'on a affaire à un opérateur ou à un entier, ainis qu'un entier `val` et un caractère `op` qui ne seront pas bien entendu utilisé simultanément.

```
#include<stdio.h>
#include<stdlib.h>
```

```
#define typVal 1
#define typOp 2

struct pile
    { int typ;
      int val;
      char op;
      struct pile * suiv;
    }

typedef struct pile *PILE;

void empileVal(int v, PILE *P)
{
    PILE nouv;
    nouv=(PILE)malloc(sizeof(struct pile));
    nouv->typ=typVal;
    nouv->val=v;
    nouv->suiv=*P;
    *P=nouv;
}

void empileOP(char oper, PILE *P)
{
    PILE nouv;
    nouv=(PILE)malloc(sizeof(struct pile));
    nouv->typ=typOp;
    nouv->op=oper;
    nouv->suiv=*P;
    *P=nouv;
}

void depile(PILE *P)
{
    PILE temp;
```

```
    temp=*P;
    *P=(*P)->suiv;
    free(temp);
}

int typSommet(PILE P)
{
return P->typ
}

int valSommet(PILE *P)
{
    int v=(*P)->val;
    depile(P);
    return v;
}

char opSommet((PILE *P)
{
    char oper=(*P)->op;
    depile(P);
    return oper;
}

void printPile(PILE P)
{
    if (P!NULL)
        { if (typSommet(P)==typVal)
                printf("%d ",P->val);
          else
                printf("%c ",P->op);
          printPile(P->suiv);
    }
}

void empilesymbole(char *st,PILE *P)
```

```
{
    if (st[0]=='+'||(st[0]=='-')||(st[0]=='*'))
        empileOP(st[0],P);
    else
        empileVal(atoi(st),P);
}

void evaluePile(PILE *P)
{
    if (typSommet(*P)==typVal)
        return (valSommet(P));
    else
        if (typSommet(*P)==typOp)
                switch(opSommet(P))
                    { case '+' : return evaluePile(P)+evaluePile(P);
                      case '-' : return evaluePile(P)-evaluePile(P);
                      case '*' : return evaluePile(P)*evaluePile(P);
                    }
}

void printfInfixe(PILE *P)
{
    char oper;
    if (typSommet(*P)==typVal)
        printf("%d", valSommet(P));
    else
        if (typSommet(*P)==typOp)
            {
                oper=opSommet(P);
                printf("(");
                printfInfixe(P);
                printf("%c",oper)
                printfInfixe(P);
                printf(")");
            }
}
```

```
int main (int argc, char * args[])
{
    int i;
    PILE P=NULL;
    for(i=argc-1;i>0,i--)
        empileSymbole(args[i],&P);
    printf("Etat pile :");
    printPile(P);
    printf("\n Expression : ");
    printfInfixe(&P);
    for(i=argc-1;i>0,i--)
       empileSymbole(args[i],&P);
    printf("\n Evaluation :%d\n", evaluePile(&P));
    exit(0);
}
```

exemple d'execution du programme

```
evalu  - * 2 3 4 * 4 2
Etat Pile : - * 2 3 4 * 4 2
Expression ((2+3)*4)-(4*2))
Evaluation 12
```

11.5 Anneaux et listes chaînées bidirectionnelles

11.5.1 Anneaux

Un anneau est une liste chaînée circulaire. Il est conçu en remplaçant, dans le dernier élément, le pointeur « `Suivant` »à `nil` par un pointeur « `Suivant` »qui pointe vers le premier élément de la liste. Il est alors plus simple d'avoir un pointeur « `Dernier` »en lieu et place du pointeur « `Début` ».

L'intérêt d'une telle structure est de pouvoir atteindre tous les éléments de la chaîne depuis n'importe quel élément. Nous pouvons alors considérer la chaîne comme n'ayant ni début ni fin. Si l'on désire toutefois pouvoir reconnaître le premier élément, il suffit de tester si le pointeur vers cet élément est égal au pointeur `Dernier^.Suivant`.

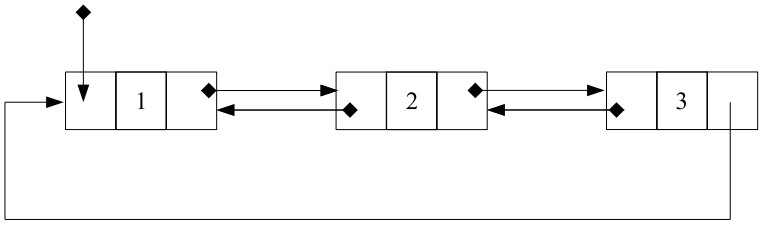


FIGURE 11.12 – Représentation graphique de la liste circulaire.

11.5.2 Manipulations sur les anneaux

Les algorithmes d'insertion, de suppression et de parcours d'éléments sont très similaires à ceux que nous avons vus pour les chaînes linéaires. Il faut juste faire attention, lors d'insertion ou de suppression en fin, de mettre correctement à jour le pointeur `dernier`. De plus, lors d'insertion après un élément donné, il faut tester si l'insertion s'est faite en fin d'anneau pour ajuster le pointeur Dernier en conséquence.

La plupart des algorithmes d'insertion doivent traiter le cas particulier de l'anneau vide. Cela peut se faire à l'aide des instructions suivantes :

```
Si Dernier = nil Alors
    Debut (*anneau vide*)
        Allouer(Dernier);
        Dernier^.cont:=...;
        (* l'él. est son propre succ.*)
        Dernier^.suiv := Dernier;
    fin;
```

Pour le parcours d'un anneau il faut prendre en compte le fait qu'il n'y a plus de pointeur à `nil` pour indiquer la fin de la chaîne. Cela donne un algorithme de parcours tel que :

```
procedure Parcours(Dernier: Liste);
var Courant, premier: Liste;
begin
    { Parcours }
    if Dernier = nil then { anneau vide }
                     else
                        begin
                            Premier := Dernier^.Suivant;
                            Courant:= Premier;
```

```
                repeat
                    { traitement quelconque à appliquer à chaque élément }
                    Traite(Courant);
                    Courant := Courant^.Suivant;
                until Courant = Premier;
            end;
end; { Parcours }
```

11.5.3 Listes chaînées bidirectionnelles

Si l'on désire souvent pouvoir atteindre des éléments d'une chaîne qui se trouvent quelques positions avant un élément donné, on peut alors adjoindre, à chaque élément, un pointeur supplémentaire vers son prédécesseur. On obtient alors une chaîne bidirectionnelle :

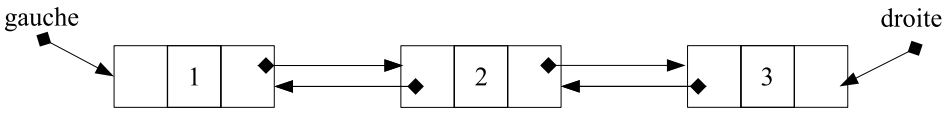


FIGURE 11.13 – représentation graphique de la liste doublement chaînée.

La déclaration contient un pointeur supplémentaire chargé d'indiquer l'emplacement du prédécesseur. Il faut donc que, lors de l'insertion ou de la suppression d'un élément, l'on tienne à jour les deux pointeurs des éléments impliqués dans l'opération.

11.5.3.1 Manipulations sur les chaînes bidirectionnelles

Exemple d'insertion en début

```
type ListeDouble = ^Cellule;
Cellule = record
    cont: Element;
Precedent,Suivant : ListeDouble;
end;

procedure InsertionDebut(var LD: ListeDouble; e: Element);
var Nouv: ListeDouble;
begin { InsertionDebut }
    new(Nouv);
    with Nouv^ do
```

```
        begin
            Info := e;
            Suivant := LD;
            Precedent := nil;
        end; { with }
       if Debut <> nil then
            LD^.Precedent := Nouv;
       LD := Nouv;
end; { InsertionDebut }
```

Exemple d'insertion avant un élément donné

```
procedure InsertionAvant(var LD: ListeDouble; Courant: ListeDouble;e:Element);
var Nouv: ListeDouble;
begin { InsertionAvant }
{ création d'un nouvel élément }
    new(Nouv);
    with Nouv^ do
        begin
            Info := Information;
            Suivant := Courant;
            Precedent := Courant^.Precedent;
        end; { with }
    { modification des liens des éléments existants }
    Courant^.Precedent := Nouv;
    if LD = Courant then
                        LD := Nouv
                   else
                        Nouv^.Precedent^.Suivant := Nouv;
end; { InsertionAvant }
```

Exemple de suppression d'un élément donné :

```
procedure SuppressionMilieu(var LD: Liste; var Courant: Liste);
begin { SuppressionMilieu }
    { mise-à-jour du pointeur Suivant du
    prédécesseur et du pointeur Precedent
    du successeur }
```

```
    if Courant = LD then begin
LD := LD^.Suivant;
```

Chapitre 12

Les arbres

12.1 Terminologie

Nous allons étudier ici une structure de données plus élaborée que toutes les autres vues jusqu'à présent : la structure *d'arbre*.

Definition 110 *De façon informelle un arbre est un ensemble de* nœuds *connectés entre eux de manière bien spécifique :*

- *chaque nœud pointe vers un ensemble éventuellement vide d'autres nœuds, qui en sont les* **fils** *(les* **enfants***)*
- *tous les nœuds sauf un ont un* **père** *et un seul*
- *le noeud père s'appelle la* **racine**.

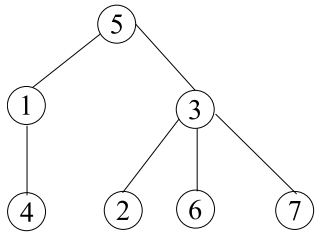


FIGURE 12.1 – Exemple d'arbre.

Lz figure 12.1 constitue un exemple d'arbre. Par convention, la racine est toujours dirigée vers le haut et les branches vers le bas. Notre arbre est constitué de 7 noeuds. Elle a pour racine le noeuds 5 qui a 2 fils 1 et 3. Le noeud 1 a un fils (4) et le noeud 3 en trois(2,6 et 7).

Definition 111 *Une* **feuille** *est un noeud sans fils, comme 4, 2, 6, et 7. Un* **noeud interne** *est un noeud qui n'est pas feuille-ici 1,3 et 5.*

Definition 112 *Quand il existe un chemin d'un noeud n_1 vers un noeud n_2 on dit que n_1 est l'ancêtre de n_2 ou que n_2 est un descendant de n_1. 5 admet les septs noeuds de l'arbre comme descendants et lui même comme ancêtre. Les ancêtres de 6 sont 6, 3 et 5*

Definition 113 *Un* **sous-arbre** *d'un arbre A est un arbre constitué de tous les descendants d'un noeud quelconque de A qui est est la racine. Par exemple les sous ensembles* $\{3, 2, 6, 7\}$ *et* $\{2\}$ *forment des sous arbres de l'arbre de la figure 12.1*

Definition 114 *La* **profondeur** *d'un noeud est la longueur du chemin qui le lie à la racine : 5 est de profondeur nulle, 1 et 3 de profondeur un, 4, 2, 6 et 7 de profondeur deux.*

Definition 115 *La* **hauteur** *d'un arbre est la profondeur maximale de ses noeuds (donc de ses feuilles) : ici 2.*

Definition 116 *Le* **degré** *d'un noeud est égal au nombre de ses fils. : 5 est de degré deux, 1 de degré trois, etc. Le degré d'un arbre correspond au degré le plus élevé de ses noeuds. Une chaîne (liste linéaire) est un arbre de degré 1.*

Definition 117 *L'***arité** *d'un arbre est le degré maximal de ses noeuds, soit trois sur notre exemple.*

Definition 118 *on parle d'***arbre numéroté** *quand l'ensemble de fils de chaque noeuds est ordonné(parfois numéroté par des entiers strictements positifs et distincts) Lorsqu'un arbre est numéroté par des entiers tous compris dans l'intervalle* $[1, k]$*, pour* $k \geq 1$*, on parle d'***arbre k-aire***. On notera qu'un arbre k-aire est d'arité inférieure ou égale à k.*

Definition 119 *Quand k vaut deux on parle d'***arbre binaire***. le fils le plus à gauche est numéroté 1 et appelé* **fils gauche** *tandis que celui de droite est numérité 2 et appelé* **fils droit***. Dans l'arbre binaire de la figure 12.2, la racine 5 adbmet 3 comme fils gauche et 7 comme fils droit. 7 a un fils droit : 8, mais pas de fils gauche*

Definition 120 *Un arbre ordonné est un arbre où la position respective des sous-arbres reflète une relation d'ordre.*

Definition 121 *Un arbre est dit équilibré si, à chaque niveau, la profondeur des différents sous-arbres ne varie pas de plus d'un niveau.*

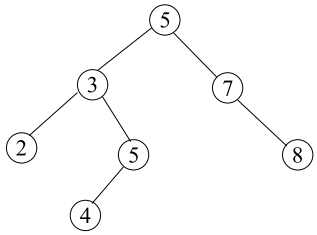


FIGURE 12.2 – Exemple d'arbre binaire.

Definition 122

Les arbres binaires se décrivent plus aisément de manière récursive. Un arbre binaire A est une structure définie sur un ensemble fini de noeuds et qui :

- ne contient aucun noeud, ou
- est formé de trois ensembles disjoints de noeuds : une **racine**, un arbre binaire appelé son **sous-arbre gauche** et un arbre binaire appelé son **sous-arbre droit**.

Definition 123 *Dans un **arbre binaire complet** chaque noeud est soit une feuille, soit de degré deux-aucun noeud n'est donc de degré un.*

Comme pour les listes, on peut implémenter lesz arbres avec des enregsitrements et des pointeurs. Tout noeud sera représenté par un enregistrement contenant une valeur et des pointeurs vers ses sous arbres.

Pour les arbres binaires, les sous arbres seront représentés par les champs `sg`, et `sd` et il sera plus simple de supposer qu'une feuille est un noeud dont les sous-arbre gauche et droit ont une valeur vide.

```
type    Arbre = ^Noeud ;
   Noeud =record
               racine : Element ;
               sg :Arbre ;
               sd :Arbre ;
            end ;
```

Pour représenter les arbres quelconques en peut enchainer les fils dans une liste.

```
type
   Arbre = ^Noeud ;
   ListeArbre= ^Cellule ;
   Cellule =Enregistrement
```

```
            cont : Arbre ;
            suiv : ListeArbre ;
        Fin ;
Noeud =Enregsitrement
            cont : Element ;
            fils :ListeArbre ;

        Fin ;
```

Dans la suite nous traiterons le plus souvent des arbres binaires, donc nous choisirons la première représentattion.

Un arbre étant une structure de données récursive, il est naturel d'utiliser des procédures récursives. comme pour les liste, la structure récursive de programmes nanipulant les arbres découle découle de la définition des arbres puisque le type des arbres binaires vérifie l'équation :

$$Arbre = \{Arbre_vide\} \biguplus Arbre \times Element \times Arbre\} \tag{12.1}$$

12.2 Opérations élémentaires sur les arbres

12.2.0.2 Test de l'arbre vide

```
Fonction Vide( A : Arbre) :booleen ;
Debut
      Vide :=(A=nil) ;
Fin
```

12.2.0.3 Racine d'un arbre

```
Fonction RAC( A : Arbre) :Element ;
Debut
      RAC :=A^.cont ;
Fin
```

12.2.0.4 Sous-arbre gauche

```
Fonction SAG( A : Arbre) :Arbre ;
Debut
```

```
      SAG :=A^.sg ;
Fin
```

12.2.0.5 Sous-arbre droit

```
Fonction SAD( A : Arbre) :Arbre ;
Debut
      SAD :=A^.sd ;
Fin
```

12.2.0.6 Construction d'un nouvel arbre

```
Fonction Construit(r : Element ; g : Arbre ; d : Arbre) :Arbre ;
var a :Arbre ;
Debut
      Allouer(a) ;
      a^.racine :=r ;
      a^.sg :=g ;
      a^.sd :=d ;
      Construit :=a ;
Fin
```

12.2.0.7 Nombre de nœuds d'un arbre

```
Fonction nbNoeuds( A : Arbre) :entier ;
Debut
   SI vide(A) ALORS
               nbNoeuds :=0 ;
            SINON
               nbNoeuds :=1+ nbNoeuds(SAG(A))+nbNoeuds(SAD(A))
Fin
```

12.2.0.8 Hauteur d'un arbre

```
Fonction hauteur( A : Arbre) :entier ;
Debut
   SI vide(A) ALORS
               hauteur :=0 ;
```

```
            SINON
               hauteur :=1+ max(hauteur((SAG(A)),hauteur(SAD(A)))
Fin
```

12.2.0.9 Recherche d'un élément

```
Fonction existe(v : Element ; A : Arbre) :boolean ;
Debut
   SI vide(A) ALORS
               existe :=faux ;
            SINON
               existe :=(v=RAC(A))||existe(v,SAG(A))||existe(v,SAD(A)))
Fin
```

12.3 Parcours d'abres

Les arbres sont beaucoup utilisés en programmation surtout pour l'analyse syntaxique des programmes. En effet, tout compilateur transforme un fichier source écrit dans un langage de programmation, qui est généralement une suite de caractète, en un arbre de syntaxe abstrait qui exprime le sens réel du programme. Ainsi, toute expresion arithmétique ou logique est transformée en un arbre. Par exemple, l'expression :

$$(a + b) \times c \tag{12.2}$$

sera représentée par l'arbre de la figure 12.3

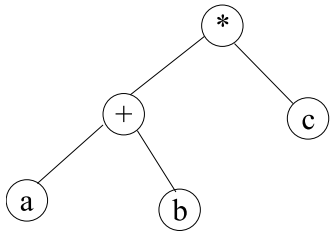


FIGURE 12.3 – Exemple d'arbre binaire.

On n'étudiera pas le processus de transformation d'une expression en arbre mais plutôt

l'opération inverse qui consiste à réécrire un arbre d'expression suivant une forme plus lisible. Suivant l'ordre de parcours de l'arbre, l'expression finale ne sera pas la même. Par exemple pour une expression arithmétique,les notations suivantes s'imposent suivant les opérateurs :

- *préfixée* : l'opérateur est écrit avant ses arguments. Par exemple exepression 12.2 s'écrit, en notation préfixée

$$* + a\, b\, c \tag{12.3}$$

- *postfixée* : l'opérateur est écrit après ses arguments. Par exemple expression 12.2 s'écrit en notation postfixée :

$$a\, b\, + 2\, * \tag{12.4}$$

- *infixée* : c'est la notation habituelle. L'opérateur est entre les opérandes.

A partir de l'arbre 12.3, on peut retrouver ces expressions en appliquant l'un des parcours récursifs suivant :

12.3.0.10 *Parcours préfixé*

on traverse chaque arbre en commençant par la racine, puis son son sous-arbre gauche, et esnuite son sous arbre droit (**RGD**).

```
Procedure PREFIXE( A : Arbre) ;
Debut
   SI Non vide(A) ALORS
                     Debut
                        Traiter(RAC(A)) ;
                        PREFIXE(SAG(A)
                        PREFIXE(SAD(A)
                     Fin
Fin
```

12.3.0.11 *Parcours postfixé*

on parcoure le sous arbre de gauche, ensuite celui de droite et enfin la racine.(**GDR**)

```
Procedure POSTFIXE( A : Arbre) ;
Debut
   SI Non vide(A) ALORS
                     Debut
                        POSTFIXE(SAG(A)
                        POSTFIXE(SAD(A)
```

```
                    Traiter(RAC(A)) ;
                Fin
Fin
```

12.3.0.12 *Parcours infixé*

on inspecte d'abord le sous-arbre de gauche, puis la racine et enfin le sous arbre droit(**GRD**).

```
Procedure INFIXE( A : Arbre) ;
Debut
   SI Non vide(A) ALORS
                Debut
                    INFIXE(SAG(A)
                    Traiter(RAC(A)) ;
                    INFIXE(SAD(A)
                Fin
Fin
```

Tous ces parcours nécessitent une visite et une seule de chaque nœud, soit donc n appels récursifs pour un arbre de n nœuds.

Il existe d'autres types de parcours d'arbres, tels que le parcours en largeur d'abord, qui consiste à inspecter tous les nœuds de même profondeur avant d'aller voir ceux de profondeur plus élevée. Ils ne seront pas abordés dans ce cours.

12.4 Arbres binaires de recherche

12.4.1 Définition

Dans cette partie nous allons aborder la notion d'arbre binaire de recherche. Celle notion est très importante quand il s'agit de structurer l'information en mémoire pour pouvoir la retrouver rapidement.

Definition 124 *Un arbre **binaire de recherche** est un arbre binaire qui a les propriétés suivantes :*

- *tous les nœuds appartiennent à un ensemble totalement ordonné*
- *tous les noeuds du sous arbre gauche d'un noeud on une valeur inférieure (ou égale) à la sienne et tous les noeuds du sous-arbre droit ont une valeur supérieure (ou égale) à la*

valeur du noeud lui même valeur

$$RAC(SAG(A)) \leq RAC(A) \leq RAC(SAD(A)) \tag{12.5}$$

L'arbre présenté dans la figure 12.4 est un arbre binaire de recherche. on notera que tous sous arbre d'un arbre binaire de recherche est aussi un arbre binaire de recherche. Une autre propriété des arbres binaires de recherche est qu'un parcours infixé en imprime les valeurs des noeuds dans l'ordre croissant. cette dernière propriété est utilisée pour mettre en place des algorithmes de tri.

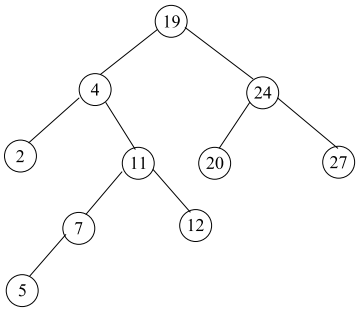


FIGURE 12.4 – Exemple d'arbre binaire de recherche.

12.4.2 Opérations sur les arbres binaires de recherche

12.4.2.1 recherche d'un élément

La recherche d'une valeur dans un arbre binaire de recherche est semblable à celle de la recherche dichotomique : on compare la valeur à chercher avec celle de la racine courante et si on ne l'a pas trouvée, on répète l'opération dans le sous arbre gauche ou droit selon le cas.

```
Fonction existeR(v :Element ; A : Arbre) :booleen ;
Debut
   SI A<>nil ALORS
               SI v=RAC(A) ALORS
                              existeR :=vrai
                           SINON
                              SI v<RAC(A) ALORS
                                             existeR :=existeR(v,SAG(A)) ;
                                          SINON
                                             existeR :=existeR(v,SAD(A)) ;
            SINON
                existeR :=faux
Fin
```

12.4.2.2 Insertion

```
Fonction inserer(v :Element ; var A : Arbre) ;
Debut
   SI A=nil ALORS
               A :=construit(v,nil,nil)
            SINON
               SI v<=RAC(A) ALORS
                              inserer(v,A^.sg) ;
                           SINON
                              inserer(v,A^.sd) ;
Fin
```

Le nombre d'opérations de la recherche ou de l'insertion dépend de la hauteur de l'arbre. Si l'arbre est bien équilibré, pour un arbre de recherche contenant n *noeuds*, on effectuera $\Theta(\log n)$ opérations pour chacune des procédures.

12.4.2.3 Suppression

La suppression d'un noeud pose généralement plus de problèmes que l'insertion. Pour un arbre binaire de recherche, il faut distinguer trois cas :

– suppression d'une feuille.

– suppression d'un noeud n'ayant qu'un seul descendant.

– supression d'un noeud ayant deux descendants.

La suppression d'une feuille ne pose pas de problème ; il suffit de mettre à `nil` le pointeur vers l'élément à détruire et de récupérer la place occupée par l'élément détruit.

La suppression d'un noeud n'ayant qu'un seul descendant est à peine plus compliquée. Il suffit de faire pointer l'ancêtre directement vers le descendant

La suppression d'un noeud ayant deux descendants est plus délicate. Il faut d'abord trouver l'élément le plus à droite du sous-arbre gauche (le plus grand élément inférieur à celui que l'on veut détruire) ; ce sera une feuille ou un noeud à un seul descendant gauche. On transfère alors le contenu de la feuille (ou du noeud à un seul descendant gauche) dans le noeud que l'on voulait détruire et l'on détruit la feuille (ou le noeud à un seul descendant gauche) à la place.

```
procedure Supprime(var A: Arbre);
var A1,A2: Arbre;
Debut
  Si (SAG(A)=nil) ET SAD(A)=nil Alors
          Liberer(P) { une feuille }
       sinon
          Si SAG(A)=nil Alors
                debut
                  A1 := A ; { 1 seul descendant }
                  A := SAD(A)
                  Liberer(A1);
                fin
              Sinon
                 Si (SAD(A)=nil) alors
                        debut
                           A1 := A; { 1 seul descendant }
                           A := SAD(A);
                           Liberer(A1);
                        fin
                    sinon
                      debut { 2 descendants }
                         A1 := SAD(A);
                         { dans le sous-arbre des éléments
                           plus petits, y a-t-il des éléments
                           plus grands que la racine locale?
```

```
                        Dans la négative, échanger puis
                        supprimer la racine locale }
                    Si SAD(A)= nil alors
                        debut
                          A^.racine := A1^.racine;
                          SupprimeArbre(A^.sg);
                        fin
                      sinon
                        debut
                          { recherche du plus grand élément
                            du sous-arbre gauche }
                          Tantque SAD(A) <> nil faire
                                debut
                                  A2 := A1;
                                  A1 := SAD(A);
                                fin; { tantque }
                          { à la fin de la boucle, A1 pointe
                            vers le plus grand des éléments
                            du sous-arbre et A2 vers son
                            ancêtre immédiat }
                          A^.racine := A1^.racine;
                          Supprime(SAD(T2));
                        fin;
                fin; { 2 descendants }
end; { Supprime }
```

12.4.2.4 Minimum d'un ABR

On suppose que A est non vide

```
Fonction minABR(A : Arbre) :Element ;
Debut
   TANTQUE SAG(A)<>nil FAIRE
            A :=SAG(A) ;
   minABR :=RAC(A) ;
Fin
```

12.4.2.5 Maximum d'un ABR

On suppose que A est non vide

```
Fonction maxABR(A : Arbre) :Element ;
Debut
   TANTQUE SAD(A)<>nil FAIRE
            A :=SAD(A) ;
   minABR :=RAC(A) ;
Fin
```

12.5 Arbres équilibré

Un arbre équilibré est un arbre dont les feuilles sont situées à peu près toujours à la même profondeur, et qui est donc le plus « templi »possible. Il existe pluisieurs types d'AVL.

- les ***arbres AVL*** des initials de leurs fondateurs G. M. Adel'son-Vel'skii et E. M. Landis. C'est le premeir type d'AVL. Ce sont des arbres binaires vérifiant la propriété suivante : *la différence entre les huateurs de sous-arbres gauche et droit de tout noeud ne peut exéder 1.*
- les ***arbres rouge et noir***, qui sont des arbres binaires de recherche dont les noeuds sont en plus coloriés(en rouge et noir) et tels que toute feuille et tout fils d'un noeud soit rouge ou noir, et tels que tous les chemins reliant un noeud donné contiennent le même nombre de noeuds noirs.
- les ***arbres 2-3***, les ***arbres 2-3-4***, les ***B-arbres***, qui sont des arbres de recherche dont l'arité est supérieure à deux.
- etc.

12.6 Arbres rouge et noir

Chapitre 13

Exercices

13.0.1 Eléments de base

Exercice 125 *Quel est le résultat de la séquence d'affectations*

```
X := X + Y ; Y := X - Y ; Y := X - Y
```

13.0.2 Les structures conditionnelles et itératives

Exercice 126 *Ecrire un programme qui lit au clavier les coefficients a, b et c de l'équation du second degré $ax^2 + bx + c = 0$ et affiche à l'écran, si elle(s) existe(nt), la ou les solutions réelles ou complexes de cette équation. Si l'équation n'a pas de solution, le programme le signalera en affichant un message à l'écran. Veillez à envisager tous les cas possibles.*

Exercice 127 *Facture EDF On veut établir la facture d'un abonné EDF, connaissant le dernier et l'avant-dernier relevé du compteur. Les règles de tarification sont les suivantes : Consommation inférieure à 100 kwh : prix du kwh = 0.5 eur Consommation comprise entre 100 kwh et 500 kwh : prix du kwh = 0.4 eur Consommation supérieure à 500 kwh : prix du kwh = 0.35 eur Coût de l'abonnement 250 eur Ecrire l'algorithme qui donne le montant de la facture à acquitter toutes taxes comprises*

Exercice 128 *Année bissextile Une année bissextile est une année divisible par 4, à l'exception des années centenaires, mais y compris les années divisibles par 400. Ecrivez l'algorithme qui lit une ann'ee et calcule si elle est bissextile Exemple : L'année 2000 est bissextile.*

Exercice 129 *Date correcte Ecrivez l'algorithme qui lit une date (jour, mois, année) et calcule si la date est correcte. Exemple : La date 29 2 1999 n'est pas correcte, car 1999 n'est pas une ann'ee bissextile*

Exercice 130 *Date du lendemain Ecrivez l'algorithme qui lit la date d'aujourd'hui (jour, mois, ann'ee) et calcule la date du lendemain.*

Exercice 131 *Transformation en heures Soient x et y, deux nombres représentant deux temps exprimés en secondes. Ecrire un algorithme qui transforme ces deux nombres en heures, minutes et secondes, qui calcule et affiche leur somme à partir de la transformation effectuée. On vérifiera que la somme obtenue est égale à la somme de x et y en secondes.*

Exercice 132 *Ecrire un programme qui calcule la moyenne d'une série de nombres entiers positifs ou nuls lus au clavier et l'affiche à l'écran. Le programme s'arrête dès qu'on introduit un nombre négatif. On pourra supposer qu'il y a au moins un nombre positif dans la série.*

Exercice 133 *Ecrire un programme qui lit des nombres entiers au clavier tant que ceux-ci sont en ordre croissant. Le programme affichera le nombre de valeurs en ordre croissant lues.*

Exercice 134 *On doit déplacer un troupeau à l'aide d'une bétaillère ne pouvant accepter plus de 1000 Kg de charge. On pèse successivement chaque bovin avant embarquement en introduisant son poids comme donnée du programme. Un poids négatif marquera la fin du travail. On demande de déterminer :*

1. *le nombre de trajets nécéssaires*
2. *le poids et le nbre de bovins transportés à chaque trajet.*
3. *le poids total du troupeau et le nombre de têtes*

Exercice 135 *Triangle de Pythagore*

Ecrire un programme Pascal qui permet d'afficher le triangle de Pythagore, triangle dans lequel l'insertion d'une ligne correspond au résultat de la multiplication de l'indice de la ligne avec les indices des colonnes.

Exemple : (pour n = 5)

```
1
2       4
3       6       9
4       8       12      16
5       10      15      20      25
```

Exercice 136 *Somme*

Ecrire un programme Pascal qui calcule la somme suivante (une seule boucle est suffisante) :

$$s = \sum_{i=0}^{i=n} (-1)^i x^{2i}/(2i)!$$

Exercice 137 *PPCM*

Le ppcm (pour plus petit commun multiplicateur) de deux entiers a et b est le plus entier m tel que m est divisible par a et b. Par exemple $ppcm(4, 3) = 12$.

1. *Déterminer PPCM (90 ; 84), PPCM(90 ; 18) et PPCM (15 ; 14) en écrivant en ordre croissant les premiers multiples des deux nombres.*
2. *Donner l'algorithme de cacul du PPCM de deux entiers a et b strictement positifs*

Exercice 138 *Calcul d'exponentiel d'un réel x*

Une valeur réelle x étant lue en donnée, on désire calculer une valeur approchée de e^x. On utilisera pour cela le développement en série de e^x :

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \ldots + \frac{x^n}{n!} + \ldots$$

On dira que $1 + x/1! + x^2/2! + x^3/3! + x^4/4! + x^5/5! + ... + x^n/n!$ approxime e^x avec une précision ϵ, si $ABS(x^{n+1}/(n+1)!) \leq$ (ici $ABS(v)$ représente la valeur absolue de v). Donner un algorithme qui permet de calculer e^x avec une précision ϵ.

Exercice 139

1. *Enumérer les différences entre une boucle* `TantQue` *et une boucle* `Repeter`.
2. *Illustrer ces différences en écrivant deux versions d'un algorithme (l'une avec* `TantQue`*, et l'autre avec* `Repeter`*) qui saisit deux entiers* `a` *et* `b` *strictement positifs et donne en sortie les quotient et reste entiers de la division de* `a` *par* `b` *sans utiliser les opérateurs* `DIV` *et* `MOD`.

Exercice 140 *Les nombres d'Armstrong*

On appelle nombre d'Armstrong, un nombre autre que 0 ou 1 qui a la propriété d'être égal à la somme des cubes de ses chiffres. Trouvez tous les nombres d'Armstrong inférieurs à 10000. Les mathématiciens ont montré qu'il n'en existait que 4, compris entre 2 et 1000.

Ecrire un programme Pascal qui permet de trouver tous les nombres d'Armstrong (compris entre 100 et 999.

Exemple :

$$153 = 1^3 + 5^3 + 3^3$$

Exercice 141 *Un entier N est dit parfait s'il est égal à la somme de ces diviseurs sauf lui même.*

Exemple : 6 *est un nombre parfait car* $6 = 1 + 2 + 3$.

1. *donner la spécification du problème*
2. *donner l'algorithme et l'organigramme qui représente la solution du problème*
3. *le coder en turbo pascal*

Exercice 142 *Ecrire un programme qui calcule la moyenne d'une série de nombres entiers positifs ou nuls lus au clavier et l'affiche à l'écran. Le programme s'arrête dès qu'on introduit un nombre négatif. On pourra supposer qu'il y a au moins un nombre positif dans la série.*

Exercice 143 *Ecrire un programme qui lit des nombres au clavier et les affiche tant que le dernier nombre lu est la somme des deux précédents.*

Exercice 144 *Montrez, à l'aide de tables de vérité, les équivalences suivantes :*

1. $p \vee (p \wedge q) \equiv p$
2. $\neg(p \vee q) \equiv \neg(p) \wedge \neg(q)$
3. $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Exercice 145 *Ecrire un algorithme qui compte le nombre de mots dans une phrase. La phrase se termine par une ponctuation ('.', '!' ou '?').*

Exercice 146 *Ecrire un programme qui trace à l'écran un triangle rempli de 'o' limité par des '*' et de hauteur un nombre inférieur à 30 entré au clavier. Par exemple pour n=8*

```
* * * * * * * *
* o o o o o *
* o o o *
* o o *
* o *
* *
*
```

Exercice 147 *La méthode de Newton pour calculer une approximation de la racine carré d'un nombre peut être décrite par :*

- *si y est une approximation de $\sqrt{x}$ alors $(y + x/y)/2$ en est une meilleure*
- *calculer les approximation successives jusqu'à ce que le résultat soit assez bon. On suppose que y est une assez bonne estimatio de $\sqrt{x}$ si $|x - y^2| < \epsilon$*

Etablir l'algorithme complet et la codification en Pascal correspondante

Exercice 148 *On définit une suite double :*

$u_0 = 1 \quad u_{n+1} = (u_n + v_n)/2$

$v_0 = 2 \quad v_{n+1} = \sqrt{u_{n+1} v_n}$

On admet que les suites (u_n) et (v_n) sont adjacentes de limite $\sqrt{27}/\pi$. Ecrire un programme qui lit un entier n et affiche l'approximation du nombre π obtenue à partir de v_n.

Exercice 149 *Le programme pascal suivant comporte des erreurs de syntaxe, trouvez les.*

```
programne repeter;

 var
  race, race2: char;
 somme, moyenne: réel;
  n: integer;

begin

 somme := 0;
 readln(race);
 race2 := race;
 n := 0;
 repeat
  somme := 0;
  while race = race2 then
   begin
    readline(poids);
    somme = somme + poids
    n := n + 1;
    race2 := race;
    readln(race)
   end;
  writeln(race2 ; n, somme / n : 8 : 2);
  n := 0;
 race2 := race;
 until race := 'X'
end

end.
```

Exercice 150 *Ecrire un programme qui lit des nombres au clavier et les affiche tant que le dernier nombre lu est la somme des deux précédents.*

Exercice 151 *Ecrire un algorithme qui permet l'affichage des 9 tables de multiplication, avec une ligne blanche entre chaque table.*

Exercice 152 *Le nombre e peut être défini comme la limite d'une série :*

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \ldots + \frac{1}{n!} + \ldots \tag{13.1}$$

Il est connu que l'approximation :

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \ldots + \frac{1}{n!} + \ldots \tag{13.2}$$

diffère de e d'au plus deux fois le terme suivant dans la série, c'est-à-dire d'au plus $2\frac{1}{(n+1)!}$ *Ecrire un programme calculant e avec une approximation dont l'erreur est inferieure a une constante* ϵ *donnée*

Exercice 153 *Une manière originale pour calculer le carré d'un nombre entier positif N est d'effectuer la somme des N premiers nombres impairs. Exemples : 4 = 1 + 3 +5 + 7 = 16 5 = 1 + 3 + 5 + 7 + 9 = 25 Ecrire un algorithme permettant de calculer le carré d'un nombre entier positif saisi par l'utilisateur en utilisant la méthode ci-dessus.*

Exercice 154 *Ecrire un programme qui lit trois variables N (un entier positif), a (un reel positif) et x (un reel dans* $]-1,1[$*) et qui calcule R avec N*

$$R = 1 + \sum_{k=1}^{N} a(a-1)\ldots(a-k+1)(-1)^k \frac{x^k}{k!}$$

Vous supposerez que la fonction factorielle et l'exponentiation n'existent pas dans le langage de programmation utilise.

13.0.2.1 Tableaux

Exercice 155 *Renversement d'un tableau Écrire une fonction qui renverse un tableau (la première valeur devient la dernière, la deuxième l'avant derni ère, etc). Par exemple, si le tableau est trié en ordre croissant au début de la fonction il doit être trié en ordre décroissant par l'algorithme.*

Exercice 156 *Ecrire un programme qui saisit 10 caractères au clavier, puis ensuite, calcule le nombre de majuscules, et le nombre de lettres dans ce tableau. Un caractère n'est pas forcément une lettre : ce peut être un chiffre, un signe de ponctuation, un caractère spécial. Une majuscule est un caract'ere compris entre 'A' et 'Z' parce que l'ordre des caractères (utilisé notamment par les opérateurs ¡, ¿, ¡= et ¿=) assure que toutes les lettres majuscules sont successives. De même une minuscule est un caractère compris entre 'a' et 'z'. Pour cet exercice, pour simplifier le problème, on ne comptera pas les caractères avec des accents comme des lettres.*

Exercice 157 *Etant donnée une séquence de relevés de températures de longueur n représentée dans un tableau T défini sur l'intervalle* $[1..Lmax](0 < n \leq Lmax)$. *On supposera dans les questions ci-dessous que le tableau considéré contient déjà la séquence, sa saisie dans le tableau n'est donc pas à faire.*

- *Ecrire un algorithme qui calcule les quantités suivantes : étendue (différence entre les valeurs maximum et minimum), valeur moyenne, déviation moyenne (moyenne des valeurs absolues des différences entre la moyenne et chaque relevé). On essaiera de minimiser le nombre de parcours effectués.*
- *Ecrire un algorithme qui détermine le plus petit relevé supérieur ou égal à une température donnée*

Exercice 158 *On considère une sequence d'entiers s de longueur L representee dans un tableau T d'entiers defini sur l'intervalle* $[1 \ldots Lmax], 0 \leq L \leq Lmax$. *On supposera dans les questions ci-dessous que le tableau considere contient deja la sequence, sa saisie dans le tableau n'est donc pas a faire. On veut ecrire un algorithme qui remplace dans T la suite s par la suite* s' *de longueur* L' *(avec* $L' \leq L$*), deduite de s en supprimant tous les elements redondants. Un element de s est redondant s'il est egal a un autre element de s. L'algorithme ne doit pas utiliser de tableau intermediaire pour creer* s'. *L'ordre des elements reste celui de la sequence de depart.*

- *Etudier tout d'abord le probleme en supposant que T peut ne pas etre trie en ordre croissant.*
  Exemple : si $s = [15, 4, 19, 4, 8, 11, 11, 3, 4, 19]$ *etn* $= 10$ *alors* $s' = [15, 4, 19, 8, 11, 3]$ *et* $L = 6$
- *Modifier l'analyse du problème dans l'hypothèse où T est trié en ordre croissant.*
  Exemple : si $s = [3, 4, 4, 8, 11, 11, 15, 19, 19]$ *et* $n = 10$, *alors* $s' = [3, 4, 8, 11, 15, 19]etn = 6$.

Exercice 159 *Etant donné un tableau d'entiers T défini sur* $[1 \ldots n]$ *avecn* > 0 *supposé initialisé, écrire un algorithme qui réalise le décalage circulaire à droite de T selon la spécification suivante : soient A et B les valeurs initiale et finale du tableau T. Le décalage est caractérisé par l'assertion :* $B[1] = A[n]$ *et* $\forall kin[2 \ldots n]$, $B[k] = A[k-1]$.

1. *Ecrire une première solution dans laquelle le problème est sérialisé en deux étapes : on crée tout d'abord un tableau intermédiaire Z, puis on recopie Z dans T.*
2. *Ecrire une deuxième solution sans recourir à un tableau intermédiaire.*

Exemple
avant décalage circulaire à droite (avec $n = 6$*)* $T = [2, 3, 29, 54, -3, 12]$
après décalage circulaire à droite (avec $n = 6$*)* $T = [12, 2, 3, 29, 54, -3]$

13.0.2.2 Fonctions

Exercice 160 *Ecrire une procéduer $triParite(T, n)$ qui trie les n premières cases d'un tableau de MAX entiers suivant le critère suivant : Tous les entiers pairs doivent être au début du tableau et les entiers impairs à la fin. le nombre d'opération seffectuées sera proportionnel à la taille du tableau. Tester votre procédure dans un programme*

Exercice 161 *Ecrire un programme qui permette, après avoir saisi 3 entiers, de les afficher dans l'ordre croissant. Ecrire une fonction MAX qui permette de retourner le plus grand de deux entiers passés en paramètres. Ecrire une fonction MIN qui permette de retourner le plus petit de deux entiers passés en paramètres. Ecrire une fonction GRAND qui permette de retourner le plus grand de trois entiers passés en paramètres, en utilisant la fonction MAX. Ecrire une fonction PETIT qui permette de retourner le plus petit de trois entiers passés en paramètres, en utilisant la fonction MIN. Ecrire une fonction MOYEN qui permette de retourner l'élément du milieu entre trois entiers passés en paramètres, en utilisant les fonctions précédemment étudiées.*

Exercice 162 *Ecrire une fonction qui retourne une approximation du sinus d'un nombre passé en paramètre en utilisant le développement de la fonction sinus suivant.*

$$sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots \tag{13.3}$$

On arretera d'effectuer la somme si le dernier terme ajouté est plus petit qu'une constante ϵ fixée dans le programme ou si on a dejà somme un nombre de termes égal a une constante MAX deéinie dans le programme.

Exercice 163 *Ecrire une fonction qui trie en ordre croissant trois variables entières passée en paramètre*

13.0.3 Récursivité

Exercice 164 *On rappelle que les coeffients binomiaux sont défifinis par :*

$C_n^n = C_n^0$ *et* $C_n^p = C_{n-1}^p + C_{n-1}^{p-1}$ *si* $0 < p < n$ *(*n *et* p *entiers).*

Ecrire une fonction récursive $c(n, p)$ *qui retourne* C_n^p*. Tester votre fonction avec un programme qui demande deux valeurs* n *;* p *à l'utilisateur et affiche la valeur calculée par* $c(n; p)$*. En utilisant la fonction précédente, écrire un programme qui affiche le triangle de Pascal qui est le triangle de nombres dont la ligne* n *contient* $C_n^0, C_n^1, \ldots, C_n^n$*. Le programme demande une valeur* N *à l'utilisateur et affiche le triangle de Pascal de la ligne 1 (qui se réduit à 1) à la ligne* N*. Par exemple pour* $N = 3$*, le résultat est*

```
1
1 1
1 2 1
1 3 3 1
```

Le developpement de $(X + Y)^n$ *est* $X^n + C_n^1 X^{n-1}Y + \ldots + C_n^p X^{n-p}Y^p + \ldots + Y^n$

Ecrire un programme qui demande une valeur n *à l'utilisateur et affiche le developpement de* $(X + Y)^n$ *(on écrira* $X * *nY * *m$ *pour désigner* X^nY^m*). Par exemple pour* $n = 3$*, le programme doit afficher* `1X**3 + 3X**2Y**1 + 3X**1Y**2 + 1Y**3`

Exercice 165 *Pour calculer le plus grand commun diviseur de entiers on utilise l'algorithme d'Euclide qui se déduit du théorème suivant :*

$$\forall (a, b) \in \mathbb{N}, a \geq b, pgcd(a, b) = \begin{cases} a & Si\ b = 0 \\ pgcd(b, a \bmod b) & Sinon \end{cases} \tag{13.4}$$

Exercice 166 *Afin de calculer efficacement* a^b *avec* b *positif, l'idée des algorithmes dits d'*exponentiation *binaire est de décomposer* b *en base* 2

$$b_i = \sum_{i=0}^{k} b_i \times 2^i \quad b_i \in \{O, 1\} \tag{13.5}$$

On peut ensuite remarquer que :

$$a^b = a^{\sum_{i=0}^{k} b_i \times 2^i} = \prod_{i=0}^{k} \left(a^{(2^i)}\right)^{b_i} \tag{13.6}$$

Afin de calculer a^b*, il suffit donc d'être capable de calculer récursivement les* $(a^{(2^i)}$ *qui peuvent s'obtenir récursivement par élevation successive au carré. En déduire une fonction permettant de calculer* a^b

Exercice 167 *Implémentation de la recherche dichotomique On suppose le tableau t trié dans l'ordre croissant. Codez en Pascal l'algorithme de recherche dichotomique*

1. *dans sa version itérative,*
2. *dans sa version récursive.*

13.0.4 Enregistrements

Exercice 168 ***Structures***

Définir une structure fraction permettant de coder les nombres de $\mathbb{Q}$ *(l'ensemble des nombres rationnels) représentés par des fractions.* Dans la suite du sujet, on utilisera obligatoirement cette structure pour représenter les nombres rationnels.

Saisie et affichage

Écrire une fonction de saisie `saisie_fraction` *qui renvoie un nombre rationnel saisie au clavier et une fonction d'affichage* `affiche_fraction` *qui affiche un nombre rationnel (sous forme de fraction). Par exemple, la fonction* `affiche_fraction` *devra afficher à l'écran "*`1 / 3`*" si on passe en paramètre la fraction* $\frac{1}{3}$.

Simplification

Écrire une fonction `simplifie_fraction` *qui renvoie une fraction irréductible égale à la fraction passée en paramètre.*

Attention il faut penser à gérer les cas où le numérateur et/ou le dénominateur est négatif. Par convention, un nombre rationnel négatif sera représenté par un numérateur négatif et un dénominateur positif et un nombre rationnel positif, par un numérateur et un dénominateur positif.

Opérations algébriques

Écrire les fonctions d'addition et de multiplication de deux nombres rationnels. Ces deux fonctions devront renvoyer des fractions irréductibles.

Comparaisons

Écrire les fonctions de tests d'égalité, d'infériorité et de supériorité entre deux nombres rationnels. Écrire une fonction `compar_fraction` *de comparaison de deux nombres rationnels qui renvoie un entier inférieur, égal, ou supérieur à zéro si le premier argument est respectivement inférieur, égal ou supérieur au second.*

Intervalle

Écrire une fonction `app` *qui teste si* $x \in [a; b]$*, avec* a*,* b *et* x*, trois nombres rationnels.*

Nombre de Neper *Écrire une fonction neper qui calcule une valeur approchée de e sous forme de nombre rationnel. On rappelle que :*

$$e = \lim_{n\to\infty} \sum_{k=0}^{n} \frac{1}{k!}$$

Exercice 169 *Sur un fichier concernant N étudiants et étudiantes sont reprises, ligne par ligne, les notes obtenues par chacun(e) pour les cinq questions d'un examen écrit. Les questions sont notées sur 4 points. La toute première ligne du fichier contient le nombre N. Le fichier se présente comme suit :*

```
18
3 4 1 1 3
4 4 4 0 2
4 2 3 1 0
    ........
```

La note globale d'un(e) étudiant(e) est évidemment égale à la somme des cinq notes le (la) concernant.

On demande d'écrire un algorithme qui lit le nombre N et les notes partielles et

- *pour chaque étudiant(e), calcule la note globale et la transforme en une note sur 100*
- *calcule la moyenne générale pour l'ensemble des N étudiant(e)s et l'écrit.*
- *calcule la moyenne par question.*

Exercice 170 *On dispose d'un fichier contenant une suite de caractères* `'A'`, `'C'`, `'G'` *et* `'T'` *correspondant à un fragment d'ADN.* `ACGAGCATTACGATAGTAGATCGATTAGAGATTAAGCGCATAGAG*` *Le fichier se termine par le caractère* `'*` *(sentinelle). On lira ce texte caractère après caractère sans se préoccuper des éventuelles fins de ligne. Ecrire un algorithme qui :*

1. *détermine le nombre de codons* `'GAG'` *apparaissant dans ce génôme*
2. *réécrive le fragment sur un fichier en remplaçant le second codon trouvé par* `'CAT'`

Exercice 171 *On considère une séquence éventuellement vide de caractères représentée dans un fichier. La séquence est une suite alternée de chiffres et d'opérateurs '+', '-' ou '*', elle représente une expression arithmétique. Ecrire un algorithme qui évalue cette expression arithmétique et affiche le résultat. Tous les opérateurs ont même priorité, chaque opérande est formé d'un seul chiffre. On dispose de la fonction* `ConvCarEnt` *qui convertit un caractère chiffre dans sa valeur décimale.*

Exemple : Fichier lu : `1+7-5*3-7+4`

Résultat : `6`

Exercice 172 *Coupe du monde de football 2002*
Dans un fichier structuré centralisé à la fédération de football sont référencés tous les joueurs licenciés susceptibles d'être sélectionnés. Ce fichier « joueurs.dat » contient pour chacun des joueurs les informations suivantes :

- *nom*
- *prénom*
- *âge*
- *ville ou il joue actuellement.*

Le sélectionneur de l'équipe de France René Lemaire a créé chez lui un petit fichier texte « `selectionneur.txt`*» dans lequel figure uniquement le nom des joueurs sélectionnés.*

```
Zidane
Petit
...
```

On voudrait à partir de ces deux fichiers créer le fichier structuré « `selectionnes.dat` *» contenant les informations du fichier «* `joueurs.dat` *» mais uniquement pour les joueurs sélectionnés.*

1. *Définir le type* `unJoueur`
2. *Ecrire la fonction* `creeFichierSelectionnes()` *qui à partir des deux fichiers «* `joueurs.dat` *» et «* `selectionneur.txt` *» crée le fichier «* `selectionnes.dat` *».*
3. *Ecrire la fonction* `ageMoyen()` *qui renvoie l'âge moyen des joueurs sélectionnés.*
4. *Ecrire le programme principal qui crée le fichier* `selectionnes.dat` *et affiche l'âge moyen des joueurs sélectionnés.*
5.

On suppose que les noms écrits par le sélectionneur dans le fichier texte sont écrits de la même manière que dans le fichier de joueurs.

Exercice 173 *Etablir la relation de récurrence permettant le calcul récursif de la moyenne des n éléments du tableau t. La formule du calcul de la moyenne étant :*

$$\overline{t_n} = \frac{\sum_{i=1}^{n} t_i}{n}$$

Etablir l'algorithme complet d'un module récursif `moyenne(...)` *calculant la moyenne des n éléments du tableau t en utilisant la relation de récurrence précédente.*

Exercice 174 *Multiplication de 2 entiers par additions successives*

Ecrire une fonction récursive dépendant de deux entiers donnés, qui calcule leur produit selon la formule de récurrence suivante.

$$k * x = x + (k - 1) * x \tag{13.7}$$

On prendra soin de trouver un cas simple pour que la fonction récursive s'arrête

Exercice 175 *Tri et récursivité*

1. *Ecrire deux procédures permettant de fusionner deux tableaux d'entiers dans les deux cas suivants :*
   - *T1 et T2 sont triés*
   - *T1 et T2 ne sont pas triés.*

   Expliquer dans chaque cas le principe utilisé.
2. *Ecrire une fonction récursive* `SOMME_REC(T,n)` *qui calcule la somme des éléments d'un tableau.*
3. *Ecrire deux procédures récursives qui affichent les éléments d'un tableau*
   - `AFF_GD(T,n)` *affiche les éléments de la gauche vers la droite en commençant par le premier de gauche ;*
   - `AFF_DG(T,n)` *affiche les éléments de la droite vers la gauche en commençant par le dernier élément*

Exercice 176 *Fichiers et structures de données*

On se propose d'écrire un logiciel permettant de construire le fichier des étudiants de ALGO2 (création, saisie et sauvegarde des données) avec nom, prénom, date de naissance, Filière etc. Les étudiants seront placés dans un tableau, chargé en mémoire depuis un fichier, en début de session, puis sauvegardé dans un fichier à la sortie du programme.

1. *Faire la déclaration des types utilisés (tableau, fichier...)*
2. *Ecrire la procédure saisie permettant de saisir les coordonnées des étudiants, et de les placer dans un tableau.*
3. *Ecrire une procédure Pascal permettant de sauvegarder le tableau d'étudiants dans un fichier séquentiel.*
4. *Ecrire une procédure Pascal permettant de lire tout le fichier d'étudiants de ALGO2 et d'en faire un tableau, placé en mémoire centrale.*
5. *Ecrire une procédure Pascal permettant de rechercher séquentiellement, dans le fichier des étudiants, une personne à partir de son nom.*

6. *Écrire une procédure Pascal permettant de rechercher séquentiellement, dans le tableau des étudiants, placé en mémoire, une personne à partir de son nom.*
7. *Ecrire un programme complet, en Pascal , utilisant les procédures définies précédemment, et proposant un menu à l'utilisateur pour lui demander ce qu'il veut : recherche dans le fichier, recherche dans le tableau, saisie de nouveaux étudiant... Le programme chargera le fichier dans un tableau, au départ, proposera le menu à l'utilisateur, et sauvegardera le tout en fin de session.*

Exercice 177 *Écrire en C une fonction récursive calculant le n-ieme terme de la suite de Lucas. Cette suite est définie pour $n > 0$ par :*

$$\begin{cases} L_1 = 2 \\ L_2 = 1 \\ L_k = L_{k-1} + L_{k-2}; \end{cases} \tag{13.8}$$

Exercice 178 *Afin de calculer efficacement a^b avec b positif, l'idée des algorithmes dits d'*exponentiation *binaire est de décomposer b en base 2*

$$b_i = \sum_{i=0}^{k} b_i \times 2^i \quad b_i \in \{O, 1\} \tag{13.9}$$

On peut ensuite remarquer que :

$$a^b = a^{\sum_{i=0}^{k} b_i \times 2^i} = \prod_{i=0}^{k} \left(a^{(2^i)}\right)^{b_i} \tag{13.10}$$

Afin de calculer a^b, il suffit donc d'être capable de calculer récursivement les ($a^{(2^i)}$ qui peuvent s'obtenir récursivement par élevation successive au carré. En déduire une fonction permettant de calculer a^b

13.0.5 Complexité

Exercice 179 *Tri par insertion. Une méthode pour trier un ensemble consiste à prendre les éléments un par un et à les insérer dans un paquet déjà trié (pensez à un joueur triant les cartes que l'on vient de lui distribuer). Quand tous les éléments ont été insérés, l'ensemble est triée.*

1. *Ecrire l'algorithme correspondant, en pseudo-code, pour trier un tableau de n entiers.*
2. *Déterminer la complexité dans le pire cas. Cela se fait en deux étapes :*
   (a) *Trouver un majorant du nombre de pas de calcul efectués par l'algorithme (valable quel que soit le tableau donné en entrée).*
   (b) *Exhiber un tableau sur lequel l'algorithme exectue effectivement le nombre de pas de calcul donné par la borne précédente.*
3.

Exercice 180 *Déterminer un algorithme qui teste si un tableau de taille n est un « tableau de permutation »tous les éléments sont distincts et compris entre 1 et n).*

1. *Donner un premier algorithme naif qui soit quadratique.*
2. *Donner un second algorithme linéaire utilisant un tableau auxiliaire.*
3.

Exercice 181 *Objectif : faire réfléchir les élèves sur la construction d'un algorithme, les faire réfléchir aussi sur la spécification d'un problème Il s'agit d'écrire en pseudo-langage un algorithme permettant à une machine d'éplucher un nombre suffisant de pommes de terre qui se trouvent dans un panier. Il faut tenir compte du fait que le panier peut être vide à un moment donné. indication : actions prédéfinies pour la machine*

```
éplucher-une-pdt
remplir-panier
```

Voici 14 versions écrites par des étudiants (pas très doués !) ne connaissant que le schéma `tantque` *et non* `répéter`

- *Critiquer ces différentes versions (discussion par rapport aux valeurs de départ, terminaison de l'algorithme, validité du programme, des résultats,...). Réfléchir à ce qui se passe dans le déroulement des différents programmes : quel est le résultat de* `éplucher-une-pdt` *si le panier est vide, quel est le résultat de* `remplir-panier` *si le panier n'est pas vide,... le nombre de pdt à éplucher est-il supérieur à la contenance du panier ? ...*
- *Ecrire un algorithme correct !*

Version 1

```
tantque panier non vide faire
tantque nbre-de-pdt insuffisant faire éplucher-une-pdt ;"
```

Version 2

```
tantque panier non vide faire
début
éplucher-une-pdt ;
éplucher-une-pdt ;
fin ;
```

Version 3

```
tantque (panier non vide) et (nbre-de-pdt insuffisant) faire
éplucher-une-pdt ;
```

Version 4

```
tantque nbre-de-pdt insuffisant faire
si panier non vide alors éplucher-une-pdt sinon remplir-panier
;
```

Version 5

```
tantque nbre-de-pdt insuffisant faire
tantque panier non vide faire éplucher-une-pdt ;
```

Version 6

```
tantque nbre-de-pdt insuffisant faire
si panier non vide alors faire éplucher-une-pdt ;
```

Version 7

```
tantque panier plein faire
si nbre-de-pdt insuffisant alors éplucher-une-pdt sinon stop ;
```

Version 8

```
tantque panier plein faire
si nbre-de-pdt insuffisant alors éplucher-une-pdt sinon
début
remplir-panier ;
éplucher-une-pdt
fin ;
```

Version 9

```
si panier non vide
alors tantque nbre-de-pdt insuffisant faire éplucher-une-pdt
sinon remplir-panier ;
```

Version 10

```
tantque (nbre-de-pdt insuffisant) ou (panier plein) faire éplucherune-
pdt ;
```

Version 11

```
tantque panier plein faire éplucher-une-pdt ;
si nbre-de-pdt insuffisant alors remplir-panier ;
```

Version 12

```
si panier vide alors remplir-panier ;
éplucher-une-pdt ;
tantque nbre-de-pdt insuffisant faire
début
si panier non vide alors éplucher-une-pdt ;
si panier non vide alors éplucher-une-pdt sinon
début
remplir-panier ;
éplucher-une-pdt ;
fin ;
fin ;
```

Version 13

```
tantque nbre-de-pdt insuffisant faire
si panier plein alors éplucher-une-pdt sinon
```

```
début
remplir panier ;
éplucher-une-pdt ;
fin ;
```

Version 14

```
si panier plein
alors début
tantque nbre-de-pdt insuffisant faire éplucher-une-pdt ;
fin
sinon début
remplir-panier ;
tantque nbre-de-pdt insuffisant faire éplucher-une-pdt ;
fin ;
```

Exercice 182 *Ecrire un algorithme qui détermine le minimum et le maximum de N valeurs non triées dans un tableau de longueur N. Quelle est la complexité de cet algorithme ? Peut-on faire mieux ? Pourquoi ?*

Exercice 183 *Ecrire un programme qui lit trois variables N (un entier positif), a (un reel positif) et x (un reel dans* $]-1,1[$*) et qui calcule R avec N*

$$R = 1 + \sum_{k=1}^{N} a(a-1)\dots(a-k+1)(-1)^k \frac{x^k}{k!}$$

k = 1 Vous supposerez que la fonction factorielle et l'exponentiation n'existent pas dans le langage de programmation utilise. Quelle est la complexite du programme en nombre de multiplications et de divisions (en fonction de N) ?

13.0.5.1 LISTES

Exercice 184 *Recherche dans une liste Écrire une fonction de recherche dans une liste (non triée).*

Exercice 185 *Algorithmes sur les piles Une "pile" est une liste dans laquelle on se contraint à toujours faire des insertions et des suppréssions au début de la liste. Seul le premier élément est dit accessible directement (on ne s'autorise pas à aller regarder plus loin que le premier élément dans la liste à un instant donné). On garde donc un ordre particulier sur les éléments qui est "premier entré, dernier sorti".*

- *Écrire une fonction qui ajoute un élément dans une pile.*
- *Écrire une fonction qui renvoie la valeur de l'élément au sommet de la pile.*
- *Écrire une fonction qui supprime le sommet de la pile.*

Exercice 186 *Une "file" est une liste dans laquelle on insère un élément en début de liste et dans laquelle on supprime un élément à la fin de la liste. De manière analogue aux piles on ne s'autorise à regarder que le dernier élément de la file pour avoir un ordre sur les éléments qui est "premier entré, premier sorti".*

- *Écrire une fonction qui ajoute un élément dans une file.*
- *Écrire une fonction qui renvoie la valeur de l'élément accessible de la file.*
- *Écrire une fonction qui supprime le premier élément (l'élément accessible) de la file.*

Exercice 187 *Les listes et les structures dynamiques se prêtent généralement assez bien aux versions récursives des algorithmes de parcours et de recherche. Une version récursive des algorithmes sur les listes est obtenue en général en considérant la liste comme composée de deux entités : l'élément que l'on est en train de considérer et la liste des éléments suivants. Une version récursive des algorithmes considère donc les éléments un par un suivant le traitement que l'on veut faire dans la fonction et si le traitement n'a pas pu être fait (l'élément n'est pas le bon) se rappelle elle-même avec le reste de la liste comme argument.*

- *Écrire une version récursive de l'algorithme de recherche dans une liste non triée.*
- *Écrire une version récursive de l'algorithme d'insertion dans une liste triée.*

Exercice 188 *Supposons une liste d'entiers stockées dans un tableau de* `NB_MAX` *entiers. Le but de l'exercice est d'écrire l'algorithme qui construit une liste chainée stockant les valeurs du tableau. La liste chaînée est définie de la manière suivante :*

```
Type Maillon = Enreg(valeur : Entier, suivant : pointeur(Maillon))
```

Ecrire une proccédure qui construit une liste chainée stockant les n premières valeurs du tableau.

Exercice 189 *Soit une liste simplement chainée dont chaque maillon est défini de la manière suivante :*

```
Type Maillon = Enreg(valeur : Entier, suivant : pointeur(Maillon))
```

Question : Ecrire une procédure qui inverse la liste simplement chainée. Par exemple, la liste composée consécutivement des entiers 23, 52, 31, 45, 59 sera inversée de la manière suivante : 59 ,45, 31, 52, 23.

Exercice 190 *Dans cet exercice, nos souhaitons représenter une discothèque personnelle en utilisant une structure dynamique. La structure proposée est représentées dans la figure 4.*

1. *Déclarer les différents types nécessaires pour représenter la structure de données de la figure 4.*
2. *Ecrire une procédure d'ajout d'une catégorie de musique en fin de liste.*
3. *Ecrire une procédure d'ajout en début de liste d'un artiste et du nombre de CDs dans une catégorie musicale donnée.*
4. *ecrire une fonction qui retourne le nombre total de disques dans la discothèque.*
5. *Ecrire une procédure qui affiche la catégorie musicale d'un artiste donnée.*
6. *Modifier les types existant et ajouter les types nécessaires pour stocker le titre des CD pour chaque artiste.*

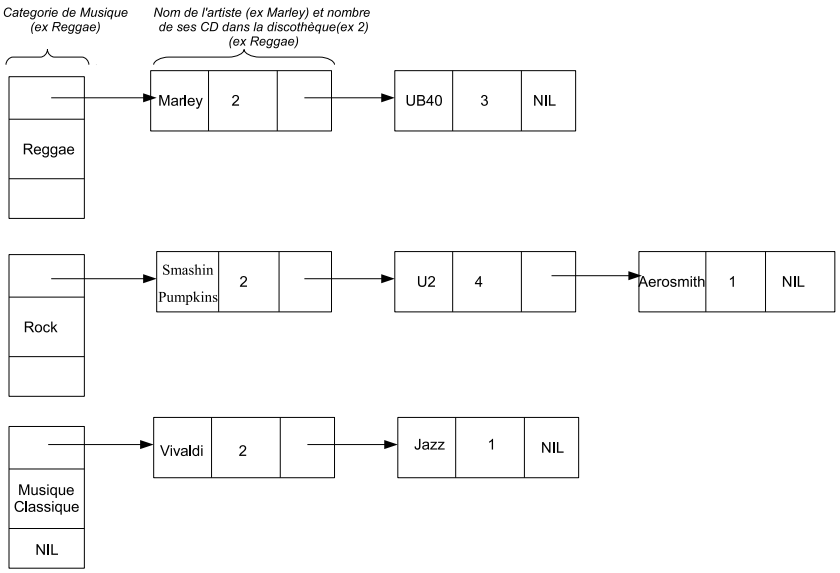


FIGURE 13.1 – Représentation d'une structure de données d'une discothèque.

Exercice 191 *On considère deux séquences d'entiers positifs, classées en ordre croissant, et représentées dans des fichiers. Un même nombre peut apparaître plusieurs fois dans une séquence. Ecrire un algorithme qui calcule le nombre d'éléments communs aux deux séquences. Exemple :*

```
séquence 1 : 1 1 3 4 11 11 11 25 28 30 30
séquence 2 : 1 2 3 5 5 11 11 24 25 29 30 34 39
résultat affiché : nombre d'éléments en commun : 6
```

Si un nombre est présent plusieurs fois dans les deux listes, il sera compté autant de fois qu'il apparaît dans les deux listes. Chaque séquence ne sera lue qu'une seule fois

13.0.5.2 Arbres

Exercice 192 *La hauteur d'un arbre est la plus longue chaîne depuis la racine jusqu'aux feuilles. On compte +1 à chaque fois que l'on passe par un n.ud de l'arbre. Écrire une fonction qui calcule la hauteur d'un arbre binaire (on pourra utiliser une fonction récursive de type post-fixée). La fonction à écrire prend comme argument l'adresse du sommet de l'arbre et renvoi la hauteur de cet arbre.*

Exercice 193 *On considère des arbres binaires dont les noeuds sont étiquetés par des entiers. Donnez :*

- *une fonction* `somme` *prenant un tel arbre et rendant la somme des valeurs de tous ses noeuds,*
- *une fonction additionne prenant en argument un arbre T' et un entier a et rendant un arbre T 0 obtenu en ajoutant a à la valeur de chaque noeud de T (pouvez-vous le faire en laissant l'arbre T lui-même in- tact ?).*

Exercice 194 *Donner une fonction calculant le minimum des valeurs contenues dans un arbre de recherche. Même question pour le maximum.*

Exercice 195 *On considère un arbre de recherche T contenant des entiers. Si v est la valeur attachée à un noeud de cet arbre, on appelle successeur de v dans T la plus petite valeur strictement supérieure à v contenue dans T. Donner un algorithme calculant le successeur de v dans T .*

Exercice 196 *On veut donner un algorithme de suppression d'un élément dans un arbre binaire de recherche.*

Commencer par donner un algorithme `supprimemaximum` *enlevant de l'arbre le noeud de valeur maximale et rendant un arbre de recherche.*

Donner ensuite un algorithme supprimeracine enlevant la racine. Pour ce dernier algorithme, vous pourrez distinguer deux cas :

- *si le sous-arbre gauche ou droit est vide,*
- *si les deux sous-arbres sont non-vides : vous pouvez alors considérer le maximum du sous-arbre gauche.*

En utilisant `supprimeracine` *donner finalement l'algorithme* `supprime`*.*

Exercice 197 *Chaînages*

On souhaite gérer une liste triée d'entiers par ordre croissant. Pour cela on utilise un vecteur unique `L` *d'enregistrements. Chaque entrée* `L[i]` *est un couple.* `L[i].cont` *est le contenu de*

l'enregistrement d'indice `i`. `L[i].suiv` *est l'indice de l'enregistrement suivant l'enregistrement d'indice* `i`. *On dispose aussi d'un entier* `PRE` *qui indique l'indice du premier entier de la liste. Le nombre* `0` *sert à indiquer l'élément fin de liste. Si l'élément d'indice i est le dernier alors* `L[i].suiv` *est égal à* `0`. *Si la liste est vide* `PRE` *est alors égal à* `0`

1. *Proposer un type permettant de représenter une liste d'entiers*
2. *Ecrire une procédure* `ajouteTete(L,n,x,PRE)` *qui ajoute l'entier* `x` *en tête de la liste* `L` *sans se préoccuper de l'ordre.*
3. *Ecrire une procédure* `ajouteQueue(L,n,x,PRE)` *qui ajoute l'entier* `x` *en queue de la liste* `L` *sans se préoccuper de l'ordre.*
4. *Ecrire une procédure* `ajouteListe(L,n,x,PRE)` *qui ajoute l'entier* `x` *à sa bonne place dans une liste d'entiers triée* `L`.
5. *Ecrire une procédure* `supprimeElement(L,x,PRE)` *qui supprime l'entier x de la liste triée d'entiers* `L`. *Pour la suppression on ne gérera pas les cellules vides, mais seulement les chaînages.*
6. *Ecrire une procédure récursive* `affichePremDern (L,PRE)` *permettant d'afficher les éléments de la liste du premier au dernier.*
7. *Ecrire une procédure récursive* `afficheDernPrem (L,PRE)` *permettant d'afficher les éléments de la liste du dernier au premier.*
8. *Ecrire une procédure récursive* `afficheMiroir(L,PRE)` *permettant d'afficher la liste* `L` *et son image miroir en même temps .*
9. *Ecrire une fonction récursive* `estTriee(T,n)` *qui renvoie* vrai *si un tableau de n entiers est trié et* faux *sinon.*

NB :

- *L'ajout consiste à placer l'élément à ajouter en fin de tableau et à le chaîner aux éléments déjà présents dans la liste.*
- *La liste* `L` *peut être vide dans toutes les sous-programmes*

Exercice 198 *Fusion de listes*

Ecrire un programme permettant de fusionner en ordre croissant deux listes chaînées d'entiers dans les deux cas suivants :

1. *Les listes sont triées par ordre croissant.*
2. *Les deux listes ne sont pas triées.*

Exercice 199 *Séparation de liste*

1. *Soit une liste chaînée L de nombres entiers. Séparer L en deux listes L1 et L2 où L1 est une liste de nombre strictement inférieur à zéro et L2 une liste de nombres positifs ou nuls.*
2. *Lire une liste L de nombres entiers, les classer par ordre dans une liste L1 et parcourir la nouvelle pour supprimer tous les éléments négatifs.*

Exercice 200 *Représentation chaînée d'un polynôme*

1. *Donner une structure S qui permet de représenter les polynômes avec des listes chaînées. Ne tenir en compte que les éléments non nuls.*
2. *Donner un algorithme qui permet d'effectuer le produit de deux polynômes représentés avec la structure S.*

Exercice 201 *Changement de base d'un entier Soit un entier naturel positif A écrit dans une base $d \in C$ où $C = \{2, 3, 4, 5, 6, 7, 8, 9\}$. On se propose d'écrire A dans la base $a \in C$. Pour cela on doit définir une nouvelle structure de données S pour représentant A dans les différentes bases.*

1. *Décrire la structure de Données S.*
2. *Donner l'algorithme qui permet de passer de la base d à la base a.*

Exercice 202 *Soit* `T` *un tableau contenant des entiers . Soit* `DEB` *un pointeur indiquant le premier entier de la liste* `T`. *Soit* `SUIVANT` *un tableau linéaire de pointeurs, de même longueur que* `T` *fournissant un chaînage des éléments de T. On se propose de lire n éléments et de les insérer par ordre croissant dans la liste* `T`. *Les éléments sont placés au fur et à mesure de leur lecture dans le tableau* `T` *(le* `i-eme` *élément lu est placé dans* `T[i]` *c'est à dire à la suite des éléments déjà placés dans* `T` *et c'est le chaînage par le tableau* `SUIVANT` *qui assure le classement). On suppose au départ que la liste* `T` *est vide c'est à dire que* `DEB=0`.

Ecrire une procédure `INSERER(SUIVANT,DEB,A)` *insérant un nombre* `A` *lu en entrée dans la liste* `T` *; elle consistera en :*

(a) *Placer* `A` *à la suite des éléments déjà présents dans* `T`
   - *Chercher l'emplacement dans la liste* `T` *en suivant le chaînage indiqué par* `SUIVANT`, *à partir de* `DEB`
   - *Effectuer les chaînages*

(b) *En déduire un programme de TRI d'une suite d'éléments entiers.*

(c) *)Ecrire une procédure* `CHERCHE(SUIVANT,DEB,K)` *cherchant le nombre* `K` *se trouvant dans la liste* `T`, *en suivant le chaînage indiqué par SUIVANT, à partir de* `DEB`. *Si* `K`

est trouvé, la procédure retourne la position de `K` *dans* `T`*, sinon* `CHERCHE` *renvoie un message du style " K n'existe pas*

Exercice 203 *Ecrire des programmes qui décrivent les opérations d'insertion et de suppression dans les listes pour la représentation doublement chaînée.*

Exercice 204 *Implémentation des piles et files par les tableaux*

1. *Piles Pour implémenter une pile à l'aide d'un tableau, on a besoin de trois variables :*
   - *un pointeur de tête de pile de type entier* `TPILE`
   - *un tableau* `PILE` *de type* `TAB = TABLEAU[1..TALLLEMAX]` *de type élément*
   - *et taille maximale* `TAILLEMAX` *de la pile.*

   On demande de définir les primitives de gestion de pile suivantes :

   `Pilevide(PILE,TPILE)` *fonction qui renvoie vrai si la pile est vide et faux sinon :*

   `Pilepeine(PILE,TAILLEMAX,TPILE)` *fonction qui renvoie vrai si la pile est pleine et faux sinon*

   `Empiler(PILE,TPILE,A)` *procédure qui empile l'élément A au sommet de la pile*

   `Depiler(PILE,TPILE)` *fonction qui renvoie l'élément de tête de pile*
2. *Files Pour implémenter une file à l'aide d'un tableau, on a besoin de quatre variables :*
   - *un pointeur de tête de file de type* `TFILE`
   - *un pointeur de queue de file de type* `QFILE`
   - *un tableau* `FIL` *de type* `TAB = TABLEAU[1..TAILLEMAX]` *de type élément et de taille maximale de* `TAILLEMAX` *de la file*

   on demande de définir les primitives de gestion d'une file suivantes :

   `Filevide(FIL,TFILE,QFILE)` *fonction qui retourne vrai si la file est vide et faux sinon*

   `Filepleine(FIL,TFILE,QFILE,TAILLEMAX)` *qui retourne vrai si la file est pleine et faux sinon*

   `Enfiler(FIL,TFILE,QFILE,A)` *procédure qui place l'élément A en tête de file*

   `Defiler(FIL,TFILE,QFILE)` *fonction qui retourne l'élément de tête de file*

b)

Conclusion

Annexe A

Titre

A.1 Articulation d'un programme

```
program ... ; { Tête du programme }
uses ... ; { Utilisation des bibliothèques }
const ... ; { Déclaration des constantes }
type ... ; { Déclaration des types }
var ... ; { Déclaration des variables}
procedure ... ; { Procédures }
function ... ; { Fonctions }
begin { programme principal }
instruction ;
...
end. {fin de programme }
```

A.2 Procédure

```
procedure NomdelaProcédure ( paramètre1 ; ... )
var nomdevariable : typedevariable ; { déclaration facultative }
begin
instruction ;
...
end;
```

A.3 Mots réservés

A.3.1 Mots réservés

absolute	exports	label	Resident
and	external	library	set
array	far	mod	shl
asm	file	near	shr
assembler	for	nil	string
begin	forward	not	then
case	function	object	to
const	goto	of	type
constructor	if	or	unit
destructor	implementation	packed	until
div	in	private	uses
do	index	procedure	var
downto	inherited	program	virtual
else	inline	public	while
end	interface	record	with
export	interrupt	repeat”	xor

Tableau A.1 – Mots réservés

A.3.2 Unité Crt

AssignCrt	GotoXY	NormVideo	TextColor
ClrEol	HighVideo	NoSound	TextMode
ClrScr	InsLine	ReadKey	WhereX
Delay	KeyPressed	Sound	WhereY
DelLine	LowVideo	TextBackground	

Tableau A.2 – Unité Crt

A.3.3 Unité DOS

A.3.4 Unité graph

```
program CarreMagique (input, output);
```

DiskFree	FindNext	GetIntVec	PackTime	SetVerify
DiskSize	FSearch	GetTime	SetCBreak	SwapVectors
DosExitCode	Fsplit	GetVerify	SetDate	UnpackTime
DosVersion	GetCBreak	Intr	SetFAttr	
Exec	GetDate	Keep	SetFTime	
FExpand	GetFAttr	line	SetIntVec	
FindFirst	GetFTime	MsDos	SetTime	

TABLEAU A.3 – Unité DOS

Arc	GetFillSettins	ImageSize	SetAllPalette
Bar	GetGraphMode	InitGraph	SetAspectRatio
Bar3D	GetImage	InstallUserDriver	SetBkColor
Circle	GetLineSettings	InstallUserFont	SetColor
ClearDevice	GetMaxColor	Line	SetFillPattern
ClearViewPort	GetMaxMode	LineRel	SetFillStyle
CloseGraph	GetMaxX	LineTo	SetGraphBufSize
DetectGraph	GetMaxY	MoveRel	SetGraphMode
Drawpoly	GetModeName	MoveTo	SetLineStyle
Ellipse	GetModeRange	OutText	SetPalette
FillEllipse	GetPalette	OutTextXY	SetRGBPalette
FillPoly	GetPaletteSize	PieSlice	SetTextJustify
FloodFill	GetPixel	PutImage	SetTextStyle
GetArcCoords	GetTextSettings	PutPixel	SetUserCharSize
GetAspectRatio	GetViewSettings	Rectangle	SetViewPort
GetBkColor	vGetX	RegisterBGIdriver	SetVisualPage
GetColor	GetY	RegisterBGIfont	SetWriteMode
GetDefaultPalette	GraphDefaults	RestoreCrtMode	TextHeight
GetDriverName	GraphErrorMsg	Sector	TextWidth
GetFillPattern	GraphResult	SetActivePage	

TABLEAU A.4 – Unité graph

```
label 999;                                (* - *)
const NMax = 100;
type  T = array [1..NMax, 1..NMax] of integer;

var a: T;
    n: integer;

procedure Init (n: integer);
```

Type	Description
byte	0..255 (8 bits non signé)
shortint	-128..127 (8 bits signé)
word	0..65535 (16 bits non signé)
integer	-32768..32767 (16 bits signé)
longint	-2147483648..2147486647 (32 bits signé)
real	2.9e-39..1.7e38 (64 bits signé) 11-12 chiffres
double	5.0e-324..1.7e308 15 chiffres
boolean	booléen qui vaut true ou False
file...of	fichier de...
string	chaîne de caractère (maximum 255)
string[num]	chaîne de caractère de longueur num
char	caractère (les 256 caractères ASCII)
pointer	pointeur

TABLEAU A.5 – Types de variables

```
    var i, j: integer;

    begin
    for i := 1 to n do
        for j := 1 to n do
            a[i, j] := 0;
    end;

function Pred (i: integer): integer;
    begin
    if i > 1 then
        Pred := i - 1;
    else
        Pred := n;
    end;

procedure Magique (n: integer);

    var i, j, k: integer;

    begin
    i := n; j := n div 2 + 1
```

```
    for k := 1 to n * n do
        begin
        while a[i, j] <> 0 do
            begin
            i := Pred (Pred (i));
            j := Pred (j);
            end;
        a[i, j] := k;
        i := 1 + i mod n;
        j := 1 + j mod n;
        end;
    end;

procedure Erreur;
    begin
    writeln ('Taille impossible.');
    goto 999;
    end;

procedure Lire (var n: integer);
    begin
    write('Taille du carre'' magique, svp?::  ');
    readln(n);
    if (n <= 0) or (n > NMax) or not odd(n) then Erreur;
    end;

procedure Imprimer (n: integer);
    var i, j: integer;
    begin
    for i := 1 to n do
        begin
        for j := 1 to n do
            write (a[i, j] : 4);
        writeln;
        end;
    end;
```

```
begin
Lire(n);
Init(n);
Magique(n);
Imprimer(n);
999:                                    (* - *)
end.
```

Bibliographie

[1] J. Shinagawa, Y. Kurosaki, F. Zhang, C. Parker, S. E. Brown, D. Jerome, J. B. Christensen, et K. Bechgaard. Superconducting state of the organic conductor $(tmtsf)_2clo_4$. *Physical Review Letters* **98**(14), 147002 (2007).